SERVICIO NACIONAL DE APRENDIZAJE – SENA

República de Colombia



MANUAL TÉCNICO

ProyecTwin

Plataforma de análisis de similitud para proyectos formativos de aprendices SENA

Versión del sistema: MVP web (React + Laravel) y aplicación Android 1.0 — el repositorio no declara un número de versión semántico único (ver § 2.1)

Equipo de desarrollo: Autores registrados en el historial de Git del repositorio: «Oscar» y «YDanB-312». La nómina completa, la ficha y los datos institucionales no constan en los archivos del proyecto (ver § 1.9 y Anexo F)

Ciudad y año: Popayán, Cauca — Colombia · Octubre de 2026

Documento generado a partir de la auditoría técnica del repositorio realizada el 16 de octubre de 2026. Las afirmaciones técnicas se sustentan en el código fuente, los archivos de configuración y las verificaciones ejecutadas que se relacionan en los anexos.

Ficha de identificación del documento

Tabla 1. Ficha técnica del documento

Campo	Valor
Nombre del proyecto	ProyecTwin (MVP) — plataforma de análisis de similitud para proyectos formativos
Nombre del documento	Manual Técnico de ProyecTwin
Identificador documental	PT-MT-001
Versión del manual	1.0
Fecha de elaboración	16 de octubre de 2026
Estado del documento	Completo para revisión del equipo; generado de forma automatizada y pendiente de validación humana final
Ámbito técnico	Backend Laravel 10, frontend React 19, aplicación Android (Kotlin), base de datos MySQL, motor de similitud, seguridad, pruebas y despliegue del MVP
Equipo de desarrollo	«Oscar» y «YDanB-312» (autores de los 46 commits del repositorio). Nómina completa, roles individuales y ficha de formación: no constan en los archivos del proyecto
Programa de formación	No confirmado en el repositorio. El sistema gestiona programas SENA (p. ej., ADSO) como datos de su catálogo, lo que no acredita el programa del equipo desarrollador
Responsable de elaboración	Auditoría y generación documental automatizada (asistida por OpenCode) sobre el repositorio real; revisión y correcciones a cargo del equipo del proyecto
Repositorio y rama	https://github.com/YDanB-312/ProyecTwin-MVP — rama master
Revisión auditada	46 commits; último commit registrado: 3b15f3f (13/10/2026). Al momento de la auditoría existían cambios sin confirmar, documentados en el Anexo F
Entorno de verificación	Windows, PHP 8.2.12 (XAMPP), Composer 2.10.2, Node 26.3.1, MariaDB 10.4.32, Microsoft Word disponible para validación del DOCX

Fuente: Elaboración propia (auditoría del repositorio ProyecTwin-MVP, 16/10/2026).

Nota de alcance documental (discrepancia entre fuentes)

El material de referencia entregado junto con la solicitud de este manual incluía el documento «M.U- PopayanAllTour -SdeI.pdf», un manual de usuario de **PopayanAllTour**, un proyecto distinto (aplicación turística de Popayán) que no guarda relación con el código de este repositorio y cuyo código fuente no existe en el entorno de trabajo. Siguiendo la regla de prioridad de fuentes —el código y los archivos de configuración determinan lo que está implementado—, este manual documenta exclusivamente el sistema **ProyecTwin-MVP** y registra la discrepancia en el Anexo F. Ninguna afirmación técnica de este documento proviene del PDF de PopayanAllTour.

Este documento es una **verificación de ingeniería basada en evidencia** (código, configuración y ejecución local). No constituye una auditoría de certificación, una prueba de penetración ni una declaración de conformidad normativa.

Control de versiones del documento

El siguiente registro documenta la elaboración de este manual. Las filas finales quedan reservadas para que el equipo del proyecto registre sus propias revisiones.

Tabla 2. Control de versiones del manual técnico

Versión	Fecha	Descripción del cambio	Responsable
0.1	16/10/2026	Auditoría inicial del repositorio, inventario de evidencias y definición de la estructura documental	Generación documental asistida (OpenCode)
0.5	16/10/2026	Ejecución del entorno local (backend, frontend, base de datos), pruebas verificadas y capturas reales de la interfaz	Generación documental asistida (OpenCode)
0.9	16/10/2026	Redacción de los ocho capítulos, figuras, tablas, referencias y anexos	Generación documental asistida (OpenCode)
1.0	16/10/2026	Versión completa entregada para revisión: campos de índice y listas activados, control de calidad documental ejecutado	Generación documental asistida (OpenCode)
—	—	(Reservado: correcciones y validación del equipo antes de la entrega institucional)	Equipo ProyecTwin

Fuente: Elaboración propia.

Tabla de contenido

La tabla siguiente es un campo automático de Word. Si aparece sin diligenciar, actualícela con «Referencias → Actualizar tabla» (o F9 sobre el campo).

Ficha de identificación del documento i
Nota de alcance documental (discrepancia entre fuentes) i
Control de versiones del documento ii
Tabla de contenido iii
Listas de figuras y tablas ix
Índice de figuras ix
Índice de tablas x
Glosario de términos y siglas xiii
CAPÍTULO 1. PRESENTACIÓN DEL PROYECTO 1
1.1. Introducción 1
1.2. Presentación general de ProyecTwin 1
1.3. Resumen ejecutivo 2
1.4. Necesidad que atiende la aplicación 3
1.5. Solución tecnológica propuesta 3
1.6. Objetivo general del manual técnico 4
1.7. Objetivos específicos 4
1.8. Alcance del manual 4
1.9. Limitaciones conocidas 5
1.10. Público objetivo 5
1.11. Convenciones de lectura y nomenclatura documental 6
CAPÍTULO 2. ASPECTOS TÉCNICOS 7
2.1. Descripción general de la solución 7
2.2. Arquitectura real del sistema 8
2.3. Componentes principales 8

2.4. Lenguajes de programación 9
2.5. Frameworks y bibliotecas 9
2.6. Herramientas de desarrollo 10
2.7. Motor de base de datos y persistencia 11
2.8. API e integraciones 11
2.9. Servicios externos 11
2.10. Entorno de ejecución 12
2.11. Requisitos mínimos y recomendados de hardware 12
2.12. Requisitos de software y versiones compatibles 12
2.13. Dependencias principales y su función 13
CAPÍTULO 3. DISEÑO Y MODELADO 14
3.1. Requisitos funcionales 14
3.2. Requisitos no funcionales 15
3.3. Matriz de trazabilidad de requisitos 16
3.4. Casos de uso relevantes 17
3.5. Actores y permisos 17
3.6. Diagramas UML pertinentes 18
3.7. Modelo de componentes y estados 19
3.8. Diagramas de secuencia de procesos importantes 20
3.9. Modelo entidad-relación 21
3.10. Modelo de datos alternativo 22
3.11. Decisiones de diseño identificadas 22
CAPÍTULO 4. BASE DE DATOS 23
4.1. Motor y versión 23
4.2. Configuración de conexión 23
4.3. Estructura de la base de datos 23
4.4. Tablas, colecciones o estructuras persistentes 24

4.5. Diccionario de datos 24
4.5.1. `general_users` 24
4.5.2. `knowledge_networks` 25
4.5.3. `training_programs` 25
4.5.4. `instructors` 25
4.5.5. `class_groups` 26
4.5.6. `apprentices` 26
4.5.7. `admins` 26
4.5.8. `projects` 26
4.5.9. `similarities` 27
4.5.10. `notifications` 27
4.5.11. `bug_reports` 28
4.5.12. `apprentice_projects` 28
4.5.13. `comments` 28
4.5.14. `motor_configs` 29
4.5.15. `audit_logs` 29
4.5.16. `password_reset_tokens` 29
4.5.17. `project_histories` 29
4.5.18. `sessions` 30
4.5.19. `personal_access_tokens` (gestionada por Sanctum) 30
4.6. Tipos de datos 30
4.7. Claves primarias y foráneas 31
4.8. Relaciones y restricciones 31
4.9. Índices 32
4.10. Migraciones y scripts de inicialización 32
4.11. Copia de seguridad y restauración 33
4.12. Integridad y validación de datos 33

CAPÍTULO 5. INSTALACIÓN Y CONFIGURACIÓN 35
5.1. Requisitos previos 35
5.2. Instalación de herramientas 35
5.3. Obtención del código fuente 35
5.4. Estructura inicial del proyecto 36
5.5. Instalación de dependencias 36
5.6. Variables de entorno 36
5.7. Configuración de servicios 37
5.8. Configuración de la base de datos 37
5.9. Inicialización y carga de datos 37
5.10. Ejecución en desarrollo 37
5.11. Verificación del funcionamiento 38
5.12. Compilación y construcción de producción 38
5.13. Despliegue 39
5.14. Solución de errores frecuentes 40
CAPÍTULO 6. REGISTRO TÉCNICO DEL DESARROLLO Y PROCEDIMIENTOS 41
6.1. Organización del código fuente 41
6.2. Árbol de directorios relevantes 41
6.3. Archivos principales y sus responsabilidades 42
6.4. Punto de entrada de la aplicación 42
6.5. Rutas y navegación 42
6.6. Componentes de interfaz 43
6.7. Lógica de negocio 43
6.8. Controladores, servicios, módulos y clases 44
6.9. Acceso y transformación de datos 45
6.10. Procesos principales 45
6.11. Operaciones CRUD implementadas 47

6.12. Endpoints de la API 47
6.13. Autenticación y manejo de sesiones 48
6.14. Gestión de errores 48
6.15. Comandos frecuentes 49
6.16. Convenciones de nombres 49
6.17. Control de versiones 49
6.18. Procedimiento recomendado para modificar y mantener el código 50
CAPÍTULO 7. SEGURIDAD Y CALIDAD 52
7.1. Arquitectura de seguridad 52
7.2. Autenticación 52
7.3. Autorización y roles 53
7.4. Validación de entradas 54
7.5. Gestión de errores 54
7.6. Protección de datos 54
7.7. Gestión de secretos 55
7.8. Dependencias y riesgos de seguridad 55
7.9. Pruebas existentes 56
7.10. Pruebas ejecutadas durante la auditoría 56
7.11. Resultados y evidencias 57
7.12. Limitaciones identificadas 57
7.13. Riesgos técnicos y medidas de mitigación 58
7.14. Plan de mejora 59
7.15. Control de calidad y normas ISO (análisis de pertinencia) 59
CAPÍTULO 8. INTERFACES Y FUNCIONALIDADES IMPLEMENTADAS 62
8.1. Metodología de verificación visual 62
8.2. Módulo público (portal, acceso y recuperación) 62
8.3. Módulo del aprendiz 64

8.4. Módulo del instructor 68
8.5. Módulo del administrador 72
8.6. Modo oscuro y experiencia móvil 76
8.7. Aplicación Android (análisis estático) 80
8.8. Resumen de estado por módulo 80
REFERENCIAS BIBLIOGRÁFICAS 81
Normas técnicas (ISO) 81
Documentación oficial de las tecnologías 82
Documentos internos del proyecto (fuentes primarias) 82
ANEXO A. ÁRBOL DE ARCHIVOS RELEVANTES 84
ANEXO B. CATÁLOGO COMPLETO DE ENDPOINTS 86
ANEXO C. MATRIZ DE TRAZABILIDAD AMPLIADA 89
ANEXO D. COMANDOS DE REFERENCIA 90
ANEXO E. EVIDENCIAS DE VERIFICACIÓN DE LA AUDITORÍA 91
ANEXO F. REGISTRO DE LIMITACIONES, DISCREPANCIAS Y DATOS NO CONFIRMADOS 92

Listas de figuras y tablas

Las listas se generan a partir de los campos de numeración de cada leyenda (actualizar si aparecen vacías).

Índice de figuras

Figura 1. Arquitectura por capas de ProyecTwin (clientes, servidor de aplicación y datos) 8
Figura 2. Casos de uso por actor (reconstruidos del código y la interfaz real) 17
Figura 3. Diagrama de secuencia: «el aprendiz abre Mis Propuestas» (GET /v1/projects) 19
Figura 4. Máquina de estados de una propuesta de proyecto 20
Figura 5. Modelo entidad-relación (núcleo del esquema implementado) 21
Figura 6. Topología de despliegue prevista (servicio único en Railway + MySQL gestionado) .. 39
Figura 7. Flujo del motor de similitud (señales, umbral y resultados vigentes/históricos) 46
Figura 8. Cadena de autenticación y autorización (SPA y móvil) con los eslabones de defensa 52
Figura 9. Portal público de ProyecTwin con el resumen del motor y la demo de similitud 63
Figura 10. Pantalla de inicio de sesión por nombre de usuario 63
Figura 11. Panel del aprendiz: métricas personales, acciones rápidas y propuestas recientes con su grado de similitud 64
Figura 12. Listado de propuestas del aprendiz con filtros por estado y grado de similitud 65
Figura 13. Detalle de una propuesta: datos, equipo, historial y acciones según estado 65
Figura 14. «Mi ficha»: unión por código, código propio y compañeros 66
Figura 15. Mis similitudes: pares detectados con porcentaje y grado (el histórico se conserva) 67
Figura 16. Detalle de similitud: comparación del par, señales del motor y pasajes coincidentes 67
Figura 17. Bandeja de alertas del aprendiz con estados de lectura 68
Figura 18. Panel del instructor: cola de revisión, fichas a cargo y actividad 69
Figura 19. Cola de revisión de propuestas con filtros por estado y ficha 69
Figura 20. Detalle de propuesta para revisión: contenido completo, equipo y observaciones 70
Figura 21. Similitudes del alcance del instructor agrupadas por ficha 71
Figura 22. Gestión de fichas del instructor (creación, edición y roster) 71
Figura 23. Panel administrativo: métricas globales y colas de gestión 72
Figura 24. Gestión de usuarios: búsqueda, filtros, estado y credenciales 73
Figura 25. Detalle de usuario: perfil, poderes administrativos y actividad 73
Figura 26. Bitácora de auditoría inmutable con filtros y paginación 74
Figura 27. Configuración del motor de similitud (umbral y ventana del corpus) 75
Figura 28. Supervisión global de similitudes con recálculo del corpus 75

Figura 29. Bandeja de soporte: reportes de falla y su ciclo de atención 76
Figura 30. Panel del aprendiz en modo oscuro (persistencia por cookie y contraste verificado por axe en la suite) 77
Figura 31. Portal público en vista móvil (390×844) 78
Figura 32. Listado de propuestas del aprendiz en vista móvil (tarjetas en lugar de tabla) 79

Índice de tablas

Tabla 1. Ficha técnica del documento i
Tabla 2. Control de versiones del manual técnico ii
Tabla 3. Glosario técnico xiii
Tabla 4. Público objetivo y uso esperado del manual 5
Tabla 5. Artefactos del repositorio 7
Tabla 6. Componentes principales y su responsabilidad 8
Tabla 7. Lenguajes utilizados (con evidencia) 9
Tabla 8. Frameworks y bibliotecas principales 10
Tabla 9. Herramientas de desarrollo y operación 10
Tabla 10. Dependencias de servicios externos 11
Tabla 11. Entornos de ejecución soportados por el proyecto 12
Tabla 12. Requisitos de hardware (estimación técnica) 12
Tabla 13. Matriz de software: declarado, verificado y compatibilidad 12
Tabla 14. Dependencias relevantes para operación y mantenimiento 13
Tabla 15. Requisitos funcionales (reconstruidos y verificados) 14
Tabla 16. Requisitos no funcionales (reconstruidos y verificados) 15
Tabla 17. Trazabilidad requisito → implementación → prueba 16
Tabla 18. Detalle de casos de uso principales 17
Tabla 19. Matriz de permisos por actor (síntesis del middleware y los scopes) 17
Tabla 20. Decisiones de diseño con evidencia y compromiso (trade-off) 22
Tabla 21. Tablas de la base de datos y su propósito 23
Tabla 22. Diccionario de datos — general_users 24
Tabla 23. Diccionario de datos — knowledge_networks 25
Tabla 24. Diccionario de datos — training_programs 25
Tabla 25. Diccionario de datos — instructors 25
Tabla 26. Diccionario de datos — class_groups 26

Tabla 27. Diccionario de datos — apprentices 26
Tabla 28. Diccionario de datos — admins 26
Tabla 29. Diccionario de datos — projects 26
Tabla 30. Diccionario de datos — similarities 27
Tabla 31. Diccionario de datos — notifications 27
Tabla 32. Diccionario de datos — bug_reports 28
Tabla 33. Diccionario de datos — apprentice_projects 28
Tabla 34. Diccionario de datos — comments 28
Tabla 35. Diccionario de datos — motor_configs 29
Tabla 36. Diccionario de datos — audit_logs 29
Tabla 37. Diccionario de datos — password_reset_tokens 29
Tabla 38. Diccionario de datos — project_histories 29
Tabla 39. Diccionario de datos — sessions 30
Tabla 40. Tipos de datos empleados (convención Laravel 10 → MySQL) 30
Tabla 41. Claves foráneas declaradas (estado final) 31
Tabla 42. Índices declarados explícitamente 32
Tabla 43. Verificación de integridad ejecutada durante la auditoría 33
Tabla 44. Requisitos previos del entorno de desarrollo 35
Tabla 45. Lista de verificación posterior a la instalación 38
Tabla 46. Errores frecuentes, causa y solución 40
Tabla 47. Archivos clave del sistema 42
Tabla 48. Rutas del frontend por área 43
Tabla 49. Componentes de interfaz destacados 43
Tabla 50. Controladores de la API y su responsabilidad (una línea) 44
Tabla 51. Cobertura CRUD por recurso (según rutas y controladores) 47
Tabla 52. Distribución de endpoints por recurso 48
Tabla 53. Errores del backend y tratamiento en la interfaz 48
Tabla 54. Comandos de operación y desarrollo 49
Tabla 55. Convenciones observadas en el código 49
Tabla 56. Resultado de los verificadores de dependencias (16/10/2026) 55
Tabla 57. Inventario de pruebas del repositorio 56
Tabla 58. Verificaciones ejecutadas y resultados reales 56
Tabla 59. Registro de riesgos (observados y potenciales) 58
Tabla 60. Normas de referencia: estado verificado y relación con el proyecto 59

Tabla 61. Estado de implementación y verificación por módulo 80
Tabla 62. Catálogo completo de endpoints de la API (84 rutas) 86
Tabla 63. Trazabilidad requisito → evidencia en código → prueba → estado 89
Tabla 64. Comandos verificados o derivados de la configuración del repositorio 90
Tabla 65. Resumen de evidencias ejecutadas 91
Tabla 66. Registro de limitaciones y discrepancias 92

Glosario de términos y siglas

Definiciones de los términos y siglas empleados en este manual, alineadas con las tecnologías que el sistema utiliza realmente.

Tabla 3. Glosario técnico

Término	Definición
ADSO	Análisis y Desarrollo de Software: programa de formación del SENA presente en el catálogo del sistema (no acreditado como programa del equipo).
Aprendiz / Instructor / Administrador	Los tres roles funcionales del sistema, definidos en la columna rol de general_users y exigidos por el middleware rol.
API REST	Interfaz HTTP con recursos y verbos estándar. ProyecTwin expone su API bajo el prefijo /v1 mediante Laravel.
Archivado de similitud	Transición de un par detectado a vigente = false para conservarlo como evidencia histórica cuando el contenido cambia o la propuesta se rechaza.
Bearer token	Token de acceso enviado en la cabecera Authorization. Sanctum lo emite para clientes que no usan cookie (app Android y pruebas automatizadas).
BM25	Función de ranking de recuperación de información usada por el motor como primera señal (peso 0,40).
Corpus (del motor)	Conjunto de propuestas aprobadas del mismo programa, dentro de la ventana temporal configurada, contra el que se compara cada propuesta enviada.
CSRF	Cross-Site Request Forgery. Laravel lo mitiga con cookie XSRF-TOKEN y la cabecera X-XSRF-TOKEN en cada escritura de la SPA.
Cookie httpOnly	Cookie de sesión no legible desde JavaScript; es el mecanismo de sesión principal de la SPA web.
Eloquent	ORM de Laravel: cada tabla tiene un modelo (app/Models) y relaciones declaradas.
Ficha (class group)	Grupo de formación SENA. Tiene código de unión, instructor a cargo, programa, y estados activo, finalizado y anulada.
Huella de envío	Resumen sha1 del contenido de una propuesta (projects.huella_envio) que impide reenviar sin cambios reales.
LSA	Latent Semantic Analysis (análisis semántico latente). Señal temática del motor implementada en PHP puro (app/Similarity/Lsa.php).
Middleware	Componente que intercepta la petición antes del controlador. ProyecTwin usa auth:sanctum, cuenta.activa, CambioContrasenaObligatorio y rol.
Migración	Archivo versionado que crea o modifica la estructura de la base de datos (database/migrations).
Motor de similitud	Conjunto de clases PHP (SimilitudService, Recomputador, Lsa, Pasajes) que compara propuestas y produce pares con porcentaje, señales y pasajes.
Par de similitud	Fila de similarities con dos propuestas (id_proyecto_1 < id_proyecto_2), porcentaje 0–100, detalles y bandera vigente.
Pasajes	Fragmentos textuales coincidentes entre dos propuestas, estilo herramienta antiplagio (hasta 5 pares por comparación).
Pivote equipo	Tabla apprentice_projects: relación N:M entre aprendices y propuestas; define quién participa en cada proyecto.
PWA	Progressive Web App: el frontend incluye manifest.webmanifest para instalación básica.
Regla «Classroom»	Regla de negocio implementada en Project::puedeEscribir: solo se participa (editar, comentar, gestionar equipo) dentro de la ficha activa; fuera de ella, solo lectura.
Sanctum	Paquete de Laravel para autenticar SPA (cookie) y clientes móviles (token) contra la misma API.
Scoping	Filtrado de datos por alcance del usuario (ficha, autoría, asignación) codificado en los scope* de los modelos (p. ej. Project::scopeParaUsuario).
Seeder	Clase que carga datos de demostración (DatabaseSeeder) o de volumen (VolumenSeeder).
SPA	Single Page Application: el frontend React se carga una vez y consume la API por fetch.
Umbral	Porcentaje mínimo a partir del cual un par se considera similar; se configura en

Término	Definición
	motor_configs (por defecto 0,30 para el seed).
Ventana temporal	Cantidad de meses hacia atrás que delimita el corpus del motor (motor_configs.meses, por defecto 12).
Vitest / Playwright / PHPUnit	Herramientas de prueba usadas por el proyecto: unitarias de frontend, extremo a extremo de navegador y pruebas de backend, respectivamente.
Word / DOCX	Formato de este manual; los índices y listas usan campos automáticos que Word actualiza al abrir el documento.

Fuente: Elaboración propia con base en el código auditado.

CAPÍTULO 1. PRESENTACIÓN DEL PROYECTO

1.1. Introducción

ProyecTwin es una plataforma desarrollada en el marco de la formación del Servicio Nacional de Aprendizaje (SENA) cuyo propósito es apoyar la **gestión de las propuestas de proyecto formativo** de los aprendices y el análisis de **similitud entre propuestas**. El sistema permite registrar propuestas, someterlas a revisión por parte de los instructores, compararlas contra un corpus de propuestas aprobadas mediante un motor de similitud implementado íntegramente en el backend, y conservar la trazabilidad de todo el ciclo mediante una bitácora de auditoría y un historial funcional.

Este manual técnico documenta el estado real del sistema a partir de una auditoría del repositorio `ProyecTwin-MVP` realizada el 16 de octubre de 2026. La auditoría incluyó la lectura del código fuente y de los archivos de configuración, la ejecución del entorno completo en un equipo local (backend, frontend y base de datos), la ejecución de las suites de pruebas y la captura de pantallas reales de la interfaz. Cada afirmación relevante se vincula con su evidencia en los anexos.

El documento está dirigido a desarrolladores, instructores técnicos y evaluadores académicos. Está organizado en ocho capítulos que cubren desde la presentación funcional hasta el detalle de instalación, operación, mantenimiento, seguridad y calidad.

1.2. Presentación general de ProyecTwin

ProyecTwin materializa el ciclo completo de una propuesta de proyecto formativo:

- **El aprendiz** crea y edita su propuesta (título, resumen, palabras clave, área de aplicación, objetivo general y objetivos específicos), gestiona el equipo de trabajo cuando la propuesta es grupal, y la envía a revisión. Al enviarla, el motor de similitud la compara contra el corpus aprobado del mismo programa de formación.
- **El instructor** revisa las propuestas de sus fichas, aprueba o rechaza con observaciones, gestiona sus fichas de formación (roster, credenciales, estados) y consulta las similitudes detectadas en su alcance.
- **El administrador** gestiona usuarios y credenciales institucionales, los catálogos de redes de conocimiento y programas, las fichas, las propuestas, las similitudes, la configuración del motor de similitud, la bitácora de auditoría y los reportes de soporte.

El sistema se compone de tres artefactos de software que conviven en un mismo repositorio: una **API REST** (Laravel 10 sobre PHP), una **aplicación web** tipo SPA (React 19 sobre Vite) y una **aplicación móvil Android** (Kotlin con Jetpack Compose). La web y la API se despliegan juntas en un único contenedor Docker; la app móvil se distribuye por separado y consume la misma API mediante token de acceso.

Un rasgo distintivo del proyecto es que el **motor de similitud es 100 % local**: no depende de servicios externos de inteligencia artificial ni de librerías de procesamiento de lenguaje natural; combina cuatro señales (coincidencia de palabras con BM25, n-gramas de caracteres, tema latente con LSA y cobertura de términos) en una puntuación de 0 a 100 con umbral y ventana temporal configurables.

1.3. Resumen ejecutivo

- **Arquitectura verificada:** monorepo con API Laravel 10 (/v1, 84 rutas), SPA React 19 (52 rutas navegables de 55 declaradas), app Android (Kotlin 2.2, Compose) y base de datos MySQL/MariaDB (20 tablas físicas).
- **Seguridad por diseño:** autenticación Sanctum con cookie httpOnly para la SPA y token Bearer para móvil; autorización por roles (aprendiz, instructor, admin) más *scoping* por ficha; validación en cada controlador; contraseñas con hash; bitácora de auditoría inmutable y campos sensibles ocultos por defecto.
- **Motor de similitud:** cuatro señales ponderadas, umbral configurable (0,05–0,95) y ventana temporal (1–60 meses) desde el panel de administración; pares vigentes e históricos; coincidencia textual por pasajes.
- **Calidad verificada en esta auditoría:** 244 pruebas de backend superadas (1.156 aserciones), 60 pruebas unitarias de frontend superadas, ESLint sin hallazgos, arranque completo del sistema y 24 capturas reales de interfaz; 54 especificaciones E2E existen en el repositorio aunque no se re-ejecutaron completas (ver § 7.10).
- **Estado de dependencias:** composer audit reporta 42 avisos de seguridad en 11 paquetes (varios de severidad alta) y npm audit reporta 3 vulnerabilidades altas en la cadena de herramientas de desarrollo. Existen actualizaciones disponibles; se recomienda planificarlas (ver § 7.8).
- **Despliegue:** Dockerfile multi-etapa con nginx + PHP-FPM y publicación en Railway con healthcheck /up; migraciones automáticas con reintentos y seed opcional en el primer despliegue.

- **Riesgos principales:** Laravel 10 y PHP 8.2 alcanzaron el final de su vida útil o están cerca de hacerlo; ausencia de integración continua (CI); sesiones de archivo que se pierden en cada redespliegue (su migración a base de datos está en curso, sin confirmar en Git); y estrategia de copias de seguridad no implementada en el repositorio.

1.4. Necesidad que atiende la aplicación

En los procesos de formación con proyectos, la revisión de propuestas plantea tres necesidades recurrentes: (1) **originalidad**: detectar coincidencias significativas entre propuestas para orientar acompañamiento sin emitir juicios automáticos; (2) **trazabilidad**: saber quién hizo qué y cuándo, tanto sobre la evolución funcional de cada propuesta como sobre las acciones administrativas; y (3) **centralización**: que aprendices, instructores y administración trabajen sobre una misma fuente de verdad en lugar de hojas de cálculo o documentos dispersos.

ProyecTwin atiende esas necesidades con reglas explícitas: las propuestas viven en estados auditables (borrador → pendiente → aprobado/rechazado), la detección de similitud se ejecuta en el momento del envío contra un corpus acotado por programa y ventana temporal, y cada acción relevante queda registrada en una bitácora inmutable. Este manual **no atribuye resultados medidos** (como reducciones de tiempo o mejoras cuantificadas) porque el repositorio no contiene estudios de impacto ni métricas de uso verificables.

1.5. Solución tecnológica propuesta

La solución implementada es una arquitectura **cliente–servidor en tres clientes** sobre una API común:

- **API REST Laravel** en /v1, con autenticación doble (cookie de sesión para la SPA, token para móvil y pruebas), middleware propio (cuenta.activa, rol, cambio de contraseña obligatorio), controladores con validación y autorización, y 16 modelos Eloquent.
- **SPA React 19 + Vite 8** con enrutado por rol, contextos de sesión/tema/motor, cliente HTTP propio con CSRF, caché de lectura de 15 s y manejo centralizado de sesión expirada.
- **App Android Kotlin + Jetpack Compose** con arquitectura MVVM, Retrofit + kotlinx-serialization, token cifrado en el dispositivo (AES-256-GCM con Android Keystore) y paridad funcional con la web.
- **Base de datos MySQL/MariaDB** con 18 tablas de dominio, restricciones de integridad y migraciones versionadas.
- **Empaquetado único** (nginx + PHP-FPM + estáticos de la SPA) para el despliegue web, con la app móvil fuera del alcance del contenedor.

El detalle de cada componente, su versión y su evidencia se desarrolla en el Capítulo 2.

1.6. Objetivo general del manual técnico

Documentar de forma completa, verificable y reproducible la arquitectura, la base de datos, la instalación y configuración, los procedimientos de desarrollo, las medidas de seguridad y las interfaces implementadas en ProyecTwin, de modo que un desarrollador ajeno al equipo pueda comprender el sistema, ponerlo en funcionamiento y mantenerlo sin depender del conocimiento de sus autores originales.

1.7. Objetivos específicos

1. Describir la arquitectura real del sistema y sus componentes con base en el código fuente auditado.
2. Documentar el modelo de datos completo: tablas, columnas, tipos, claves, relaciones, índices y restricciones.
3. Establecer un procedimiento de instalación y ejecución reproducible en un entorno limpio, con comandos verificados y resultados esperados.
4. Explicar la organización del código, los flujos de negocio principales y las convenciones de desarrollo.
5. Analizar la seguridad implementada, las pruebas existentes y las ejecutadas durante la auditoría, con sus limitaciones explícitas.
6. Evaluar la pertinencia de normas ISO de referencia frente al proyecto, distinguiendo práctica observada de evidencia de cumplimiento.
7. Documentar las interfaces reales de la aplicación con evidencia visual capturada del sistema en ejecución.
8. Registrar limitaciones, riesgos y discrepancias entre fuentes para orientar el plan de mejora.

1.8. Alcance del manual

Cubre: el backend API (Laravel), el frontend web (React), la aplicación móvil Android, la base de datos, el motor de similitud, la configuración de despliegue (Docker/Railway), el entorno de pruebas y los procedimientos de operación y mantenimiento.

No cubre: la infraestructura física de la institución, la administración del dominio institucional, la capacitación pedagógica de los usuarios finales (existe material en el repositorio: `docs/guion-presentacion.md`), la auditoría contable o legal del proyecto, ni una evaluación de rendimiento bajo carga (no se ejecutaron pruebas de estrés).

1.9. Limitaciones conocidas

Las siguientes limitaciones se identificaron durante la auditoría y se mantienen visibles de forma deliberada (el detalle y su tratamiento están en el Anexo F):

- **Datos institucionales incompletos:** el repositorio no contiene la ficha de formación, la nómina completa del equipo, cargos ni el centro de formación; la portada y la ficha documental lo declaran como dato faltante en lugar de inventarlo.
- **Discrepancia de fuentes:** el PDF de referencia entregado pertenece a otro proyecto (PopayanAllTour) y no fue utilizable como fuente funcional; ver la nota de alcance en los preliminares.
- **Pruebas E2E no re-ejecutadas completas:** la suite E2E (54 especificaciones) existe y está documentada, pero en esta auditoría solo se ejecutaron capturas controladas del sistema; no se corrió la suite completa (justificación y método en § 7.10).
- **App móvil sin compilación verificada en esta auditoría:** el SDK de Android está presente, pero no se generó el APK ni se capturaron pantallas del dispositivo; su análisis es estático (código y configuración).
- **Entorno verificado ≠ producción:** todas las verificaciones se ejecutaron en un equipo Windows con PHP 8.2.12 y MariaDB 10.4.32; el contenedor de producción usa PHP 8.3 (imagen oficial) y la base de datos gestionada de Railway.
- **Documentación histórica desactualizada:** varios archivos del repositorio (README-ALCANCE.md, ALINEACION-FRONTEND-BACKEND.md, CHANGELOG.md, conteos del README.md del frontend) describen etapas anteriores del proyecto; este manual señala las discrepancias donde afectan afirmaciones vigentes.
- **Trabajo en curso sin confirmar:** al cierre de la auditoría existían cambios sin confirmar en Git (incluida una migración de sesiones en base de datos y archivos de despliegue); se documentaron como estado del árbol de trabajo, no como versión liberada.
- **Sin integración continua:** no existen flujos de CI en el repositorio; las verificaciones dependen de ejecución manual.

1.10. Público objetivo

Tabla 4. Público objetivo y uso esperado del manual

Perfil	Uso esperado
Desarrollador nuevo en el equipo	Comprender arquitectura, estructura del código y flujos; poner el sistema en marcha; modificar y extender con seguridad.
Desarrollador de mantenimiento	Consultar diccionario de datos, contratos de API, convenciones y procedimientos de cambios.
Instructor técnico / evaluador académico SENA	Verificar la correspondencia entre lo implementado y lo documentado; evaluar calidad, seguridad y trazabilidad.

Perfil	Uso esperado
Administrador de sistemas	Desplegar, configurar variables, migrar la base de datos y operar el servicio (capítulo 5).
Auditor de calidad / seguridad	Punto de partida para revisiones formales: hallazgos, riesgos, pruebas y limitaciones (capítulo 7).

Fuente: Elaboración propia.

1.11. Convenciones de lectura y nomenclatura documental

- **Código fuente en línea:** formato monoespaciado para rutas, archivos, comandos, tablas y campos.
- **Rutas de archivo:** relativas a la raíz del repositorio (backend/…, frontend/…, Movil/…) salvo indicación contraria.
- **Clasificación de afirmaciones:** *verificado* (comprobado en código, configuración o ejecución durante la auditoría); *parcialmente verificado* (evidencia indirecta o incompleta); *inferido* (deducido del código sin prueba ejecutada); *no verificado* (declarado por documentos del proyecto pero no comprobado en esta auditoría).
- **Referencias normativas:** las ediciones de las normas se verificaron en el catálogo oficial de ISO el 16/10/2026 (§ 7, control de calidad).
- **Figuras y tablas:** numeradas de forma consecutiva y generadas con campos automáticos de Word; las listas de figuras y tablas al inicio del documento se diligencian al actualizar los campos.
- **Prioridad de fuentes:** código y configuración > documentos técnicos internos vigentes > documentación oficial de las tecnologías > material histórico (marcado como tal).
- **Cifras:** cuando un documento del proyecto afirma un conteo y la auditoría obtuvo otro, prevalecen los conteos obtenidos por lectura directa y ejecución, señalando la diferencia.

CAPÍTULO 2. ASPECTOS TÉCNICOS

2.1. Descripción general de la solución

ProyecTwin es una solución cliente–servidor. En el repositorio conviven tres aplicaciones y los archivos de despliegue:

Tabla 5. Artefactos del repositorio

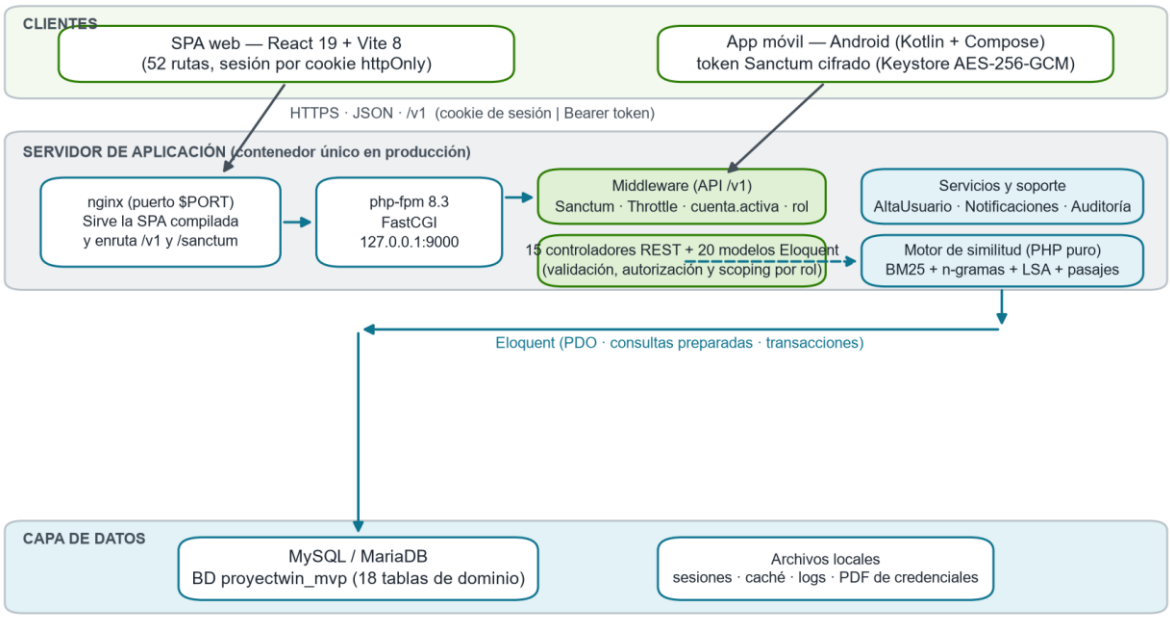
Ruta	Qué contiene	Estado
backend/	API REST Laravel 10 (PHP): rutas /v1, controladores, modelos, migraciones, seeders, motor de similitud, pruebas PHPUnit.	Completo y en ejecución
frontend/	SPA React 19 + Vite 8: 47 páginas, 54 componentes, clientes de API, suite E2E (54 especificaciones) y unitarias.	Completo y en ejecución
Movil/	Aplicación Android Kotlin + Jetpack Compose (MVVM), Retrofit, DataStore, ~40 pantallas.	Completo; compilación no verificada en esta auditoría
docker/ + Dockerfile + railway.json	Empaquetado de producción (nginx + PHP-FPM) y despliegue en Railway.	Trabajo reciente, aún sin confirmar en Git al cierre de la auditoría
docs/	Diagramas y documentos internos (arquitectura, despliegue, guion de presentación, normativa visual).	Parcialmente desactualizado (ver § 1.9)

Fuente: Inventario del repositorio (16/10/2026).

No existe un número de versión semántico único del sistema: la app Android declara `versionName 1.0` (`Movil/app/build.gradle.kts`), mientras que la web y la API no exponen versión en el código (el `package.json` del frontend es `0.0.0`). El nombre canónico del producto en la interfaz y los metadatos es **ProyecTwin** (los identificadores técnicos usan variantes `proyectwin`/`proyecttwin`, p. ej. la base de datos `proyectwin_mvp`).

2.2. Arquitectura real del sistema

Figura 1. Arquitectura por capas de ProyecTwin (clientes, servidor de aplicación y datos)



Fuente: Elaboración propia a partir del código y la configuración del repositorio.

La Figura 1 resume la arquitectura verificada. Los clientes (SPA y app Android) consumen la misma API REST bajo `/v1` mediante HTTPS y JSON. En producción, un único contenedor ejecuta nginx (sirve los estáticos compilados de la SPA y enruta `/v1` y `/sanctum` a PHP-FPM) y Laravel resuelve las peticiones atravesando middleware de seguridad antes de llegar a los controladores; estos usan modelos Eloquent y servicios de dominio (credenciales, notificaciones, auditoría, motor de similitud) sobre MySQL. La app Android no forma parte del contenedor: se distribuye aparte y apunta a la misma API.

En desarrollo, Vite sirve la SPA en el puerto 5173 y **proxya** `/v1` y `/sanctum` hacia `http://127.0.0.1:8000` (`frontend/vite.config.js`), de modo que navegador y API comparten origen y la cookie de sesión viaja sin fricción CORS. Esta decisión (mismo origen en desarrollo y producción) explica por qué el cliente HTTP del frontend usa rutas relativas y cookies httpOnly en lugar de tokens en almacenamiento web.

2.3. Componentes principales

Tabla 6. Componentes principales y su responsabilidad

Componente	Tecnología	Responsabilidad	Ubicación
API REST	Laravel 10	84 rutas bajo /v1; autenticación, autorización por rol y alcance, reglas de	backend/routes/api-v1.php

Componente	Tecnología	Responsabilidad	Ubicación
		negocio, motor de similitud, auditoría.	
Autenticación	Laravel Sanctum 3	Sesión por cookie httpOnly para la SPA; tokens Bearer para móvil y pruebas.	backend/app/Http/Controllers/Api/AuthController.php
Motor de similitud	PHP puro (sin IA externa)	Puntuación 0–100 con 4 señales, umbral y ventana configurables; pares vigentes/históricos y pasajes.	backend/app/Services/SimilitudService.php, backend/app/Similarity/
Servicio de notificaciones	PHP/Laravel	Punto único de creación de avisos internos (similitud, revisión, observación, mensaje, sistema).	backend/app/Services/NotificacionesService.php
Auditoría e historial	PHP/Laravel	Bitácora inmutable de acciones administrativas y línea de tiempo funcional de cada propuesta.	backend/app/Support/Auditoria.php, HistorialProyecto.php
Credenciales institucionales	PHP + Dompdf	Alta con usuario/contraseña temporal, envío por correo y exportación de PDF.	backend/app/Services/AltaUsuario.php, backend/app/Support/CredencialesPdf.php
SPA web	React 19 + React Router 7	43 rutas protegidas por rol, guardas de sesión y error, temas claro/oscuro, PWA básica.	frontend/src/App.jsx, frontend/src/pages/
Cliente HTTP web	JavaScript (fetch)	CSRF, cookie de sesión, caché de lectura 15 s, deduplicación y manejo de 401.	frontend/src/lib/api.js, frontend/src/lib/recursos.js
App móvil	Kotlin + Compose (MVVM)	Paridad funcional con la web para los tres roles; token cifrado en el dispositivo.	Movil/app/src/main/java/com/example/proyectwin/
Contenedor de despliegue	Docker (nginx + php-fpm 8.3)	Empaqueta SPA + API en un servicio con healthcheck y migraciones al arranque.	Dockerfile, docker/nginx.conf, docker/entrypoint.sh

Fuente: Elaboración propia con base en el código auditado.

2.4. Lenguajes de programación

Tabla 7. Lenguajes utilizados (con evidencia)

Lenguaje	Uso en el sistema	Evidencia
PHP (≥ 8.1)	Backend completo: API, motor de similitud, comandos y pruebas.	backend/composer.json ("php": "^8.1"); ejecución verificada con 8.2.12
JavaScript (ES2022+)	SPA React, clientes de API, pruebas unitarias y E2E.	frontend/package.json; archivos .js/.jsx
Kotlin 2.2	Aplicación Android completa (Compose, ViewModels, repositorios).	Movil/gradle/libs.versions.toml
SQL (MySQL)	Esquema relacional, consultas de scopes, integridad y seeders.	Migraciones y modelos Eloquent
HTML5 / CSS3	Interfaz web: tokens de diseño, CSS Modules, PWA.	frontend/src/tokens/index.css, módulos CSS
JSON	Contratos de la API, columnas JSON (detalles, objetivos_especificos) y configuración.	Respuestas de API y migraciones

Fuente: Revisión del repositorio (16/10/2026).

2.5. Frameworks y bibliotecas

La Tabla siguiente consolida las dependencias principales por capa, con la **versión declarada** en los archivos de manifiesto y la **versión detectada** en el entorno de verificación (instalada por

el gestor de paquetes). Las restricciones declaradas y las versiones resueltas pueden diferir porque los manifiestos usan rangos (^).

Tabla 8. Frameworks y bibliotecas principales

Capa	Paquete	Declarada	Detectada	Propósito
Backend	laravel/framework	^10.0	10.x	Framework base de la API.
Backend	laravel/sanctum	^3.2	3.x	Autenticación SPA (cookie) y tokens móviles.
Backend	barryvdh/laravel-dompdf	*	2.x	Generación del PDF de credenciales.
Backend	guzzlehttp/guzzle	^7.2	7.10.0	Cliente HTTP interno (correo/notificaciones).
Backend (dev)	phpunit/phpunit	^10.0	10.x	Suite de pruebas del backend (244 casos).
Backend (dev)	fakerphp/faker, mockery, collision, pint, sail	^1.9 / ^1.4 / ^7 / ^1 / ^1.18	—	Datos falsos, dobles de prueba, salida de pruebas, estilo de código y contenedores de desarrollo.
Frontend	react, react-dom	^19.2.5	19.2.6	Biblioteca de interfaz.
Frontend	react-router-dom	^7.15.1	7.18.3	Enrutado de la SPA con guardas por rol.
Frontend	phosphor-react	^1.4.1	1.4.x	Iconografía.
Frontend (dev)	vite	^8.0.10	8.2.2	Servidor de desarrollo y empaquetado.
Frontend (dev)	@playwright/test + @axe-core/playwright	^1.62.1 / ^4.13.0	1.62.1	Pruebas E2E y auditoría de accesibilidad.
Frontend (dev)	vitest + @testing-library/*	^4.1.11 / ^16.3 / ^7.0 / ^14.6	4.1.11	Pruebas unitarias de componentes y utilidades.
Frontend (dev)	eslint + plugins	^10.2.1	10.x	Análisis estático (0 hallazgos en esta auditoría).
Móvil	Kotlin + Jetpack Compose	2.2.10 / BOM 2026.02.01	—	UI declarativa y lógica de la app Android.
Móvil	Hilt	2.60.1	—	Inyección de dependencias.
Móvil	Retrofit + OkHttp + kotlinx-serialization	2.9.0 / 4.12.0 / 1.6.3	—	Consumo de la API y serialización JSON.
Móvil	DataStore Preferences	1.1.4	—	Persistencia local de sesión.

Fuente: `backend/composer.json`, `frontend/package.json`, `Movil/gradle/libs.versions.toml`; versiones detectadas con `composer show`, `npm ls` (16/10/2026).

2.6. Herramientas de desarrollo

Tabla 9. Herramientas de desarrollo y operación

Herramienta	Versión en la auditoría	Uso
Composer	2.10.2	Gestión de dependencias PHP; instalación verificada del backend.
npm / Node.js	11.16.0 / 26.3.1	Dependencias y scripts del frontend; instalación verificada.
Vite	8.2.2	Servidor de desarrollo (proxy a la API), build de producción y preview.
PHP + extensiones	8.2.12 (XAMPP)	Ejecución local del backend; extensiones clave presentes: pdo_mysql, mbstring, dom, openssl, tokenizer, xml, fileinfo, curl, bcmath, zip.
MariaDB (XAMPP)	10.4.32	Base de datos local compatible con MySQL.
Git	Repositorio con 46 commits	Control de versiones (rama master; remoto GitHub).
Gradle	9.4.1 (wrapper)	Construcción de la app Android (no ejecutada en esta auditoría).
PHPUnit / Vitest / Playwright	10.x / 4.1.11 / 1.62.1	Pruebas de backend, unitarias y E2E.
ESLint	10.x	Análisis estático del frontend (flat config).
Microsoft Word	Instalado en el equipo	Actualización de campos y validación del presente

Herramienta	Versión en la auditoría	Uso
		documento.

Fuente: Verificación directa del entorno (16/10/2026).

2.7. Motor de base de datos y persistencia

La persistencia principal es **MySQL**, con compatibilidad efectiva con **MariaDB** (la verificación se hizo sobre MariaDB 10.4.32; el driver es `pdo_mysql`). No se detectó ningún otro motor de datos: los archivos de configuración declaran `DB_CONNECTION=mysql` y el esquema usa 18 tablas de dominio creadas por migraciones Laravel con codificación `utf8mb4` y colación `utf8mb4_unicode_ci`. Algunos datos viven en columnas `JSON` (detalles del motor, objetivos específicos, bitácora); no se usan bases NoSQL ni almacenamiento en la nube.

Persistencias secundarias verificadas: archivos locales para caché y **sesiones** (`SESSION_DRIVER=file` según `.env.example`; existe una migración reciente, aún sin confirmar en Git, que crea la tabla `sessions` para migrar este comportamiento a base de datos), logs en `storage/logs` y generación de PDF en memoria con Dompdf (sin almacenamiento permanente de los PDF). Las fotografías de perfil se guardan como cadenas en `general_users.foto_url` (data URL procesada en el cliente a 256 px), decisión aceptable para el MVP pero con impacto en el tamaño de la base de datos (riesgo R-09, § 7.13).

2.8. API e integraciones

El sistema expone su propia **API REST** (no consume API de terceros para su función principal):

- Prefijo único /v1 registrado en backend/app/Providers/RouteServiceProvider.php; **84 rutas** en backend/routes/api-v1.php (10 públicas, 4 de sesión, 70 autenticadas, de las cuales 22 restringen a rol:admin o rol:admin,instructor).
- Formato JSON con errores homogéneos {message: …} y códigos 401/403/404/409/422/502; paginación opt-in (?paginado=1); inclusión controlada de relaciones con ?included= validada por lista blanca en cada modelo.
- Integraciones salientes: **correo** para credenciales y recuperación de contraseña (MAIL_MAILER configurable; en desarrollo escribe al log) y **Webhook no**: no hay integraciones de pago, mapas ni IA externa.
- El catálogo completo de endpoints está en el Anexo B y su capítulo de referencia es el 6.12.

2.9. Servicios externos

Tabla 10. Dependencias de servicios externos

Servicio	¿Obligatorio?	Uso y observaciones
Servidor SMTP	Opcional	Solo para envío real de credenciales y recuperación; en

Servicio	¿Obligatorio?	Uso y observaciones
(correo)		desarrollo/log y en producción sin SMTP la cuenta se crea y el error queda registrado (credenciales_error).
Railway (hosting)	Opcional	Plataforma de despliegue prevista en railway.json; el software puede ejecutarse en cualquier host con Docker.
Google Fonts	Sí en la web pública	index.html carga Inter, JetBrains Mono y Sora desde fonts.googleapis.com; sin conexión, el navegador usa fuentes de reserva.
Android Keystore	Sí en la app	Protege el token de sesión en el dispositivo (servicio local del sistema operativo, no red).
APIs de terceros / IA	No	El motor de similitud es local; no se detectaron claves ni llamadas a servicios de IA.

Fuente: Revisión de configuración y código (16/10/2026).

2.10. Entorno de ejecución

Tabla 11. Entornos de ejecución soportados por el proyecto

Entorno	Composición	Evidencia
Desarrollo (verificado en esta auditoría)	Windows + XAMPP (PHP 8.2.12, MariaDB 10.4.32) + php artisan serve (:8000) + Vite dev (:5173, proxy a la API).	Ejecución y capturas del 16/10/2026
Producción web (previsto)	Contenedor Docker: php:8.3-fpm-alpine + nginx + tini; SPA compilada servida por nginx; migraciones al arranque.	Dockerfile, docker/entrypoint.sh
Móvil	Android 8.0 (API 26) o superior; compilación con SDK 36 y Gradle 9.4.1.	Movil/app/build.gradle.kts
Navegadores web	Navegadores modernos de escritorio y móviles (la build de Vite 8 apunta a navegadores «Baseline widely available»).	documentación oficial de Vite 8 y vite.config.js

Fuente: Auditoría del repositorio y ejecución local.

2.11. Requisitos mínimos y recomendados de hardware

El repositorio no declara requisitos de hardware. Los siguientes valores son **recomendaciones de estimación técnica** elaboradas a partir de la carga observada durante la auditoría (instalación completa, servidor de desarrollo, base de datos y suite de pruebas), no requisitos oficiales del proyecto.

Tabla 12. Requisitos de hardware (estimación técnica)

Perfil	Mínimo	Recomendado
Equipo de desarrollo (web)	4 GB de RAM, 4 GB de disco libre, CPU de 2 núcleos	8–16 GB de RAM, SSD, CPU de 4 núcleos (la instalación de dependencias y las suites de prueba se benefician del disco SSD)
Equipo de desarrollo (móvil)	Además de lo anterior: 8 GB de RAM y 10 GB libres para SDK/emulador	16 GB de RAM
Producción web (contenedor)	512 MB de RAM, 1 vCPU (MVP)	1 GB de RAM y 1–2 vCPU con base de datos gestionada aparte

Fuente: Estimación propia durante la auditoría (no consta en el repositorio).

2.12. Requisitos de software y versiones compatibles

Tabla 13. Matriz de software: declarado, verificado y compatibilidad

Software	Declarado / exigido	Verificado en auditoría	Observaciones de compatibilidad
PHP	^8.1 (composer.json)	8.2.12 (local); imagen de producción 8.3	PHP 8.1 ya no recibe soporte; la rama 8.2 termina su soporte de seguridad el 31/12/2026 y la 8.3 el 31/12/2027 (fuente: php.net, consulta 16/10/2026).
Composer	2.x	2.10.2	Instalación reproducible con composer install.

Software	Declarado / exigido	Verificado en auditoría	Observaciones de compatibilidad
MySQL / MariaDB	MySQL 5.7+ (README)	MariaDB 10.4.32	Esquema compatible; strict activo y utf8mb4_unicode_ci.
Node.js	No declarado explícitamente; Vite 8 exige ≥ 20.19/22.12	26.3.1	La imagen Docker compila con Node 22.
npm	—	11.16.0	Lockfile package-lock.json presente.
Navegadores	Modernos	Chromium (capturas)	PWA instalable opcional (manifest.webmanifest).
Android	minSdk 26 / target 36	SDK presente; compilación no ejecutada	Requiere JDK 17 para el daemon de Gradle (descargado por toolchain).
Word	Para editar el manual	Disponible	Los índices del documento son campos automáticos.

Fuente: Manifiestos del proyecto y verificación local (16/10/2026); ciclo de vida de PHP según php.net.

2.13. Dependencias principales y su función

Además de la tabla de frameworks (§ 2.5), estas son las dependencias cuyo comportamiento es determinante para operar el sistema:

Tabla 14. Dependencias relevantes para operación y mantenimiento

Dependencia	Capa	Función en el sistema	Nota de operación
laravel/sanctum	Backend	Sesiones SPA + tokens móviles.	La expiración del token se controla con SANCTUM_EXPIRATION (1440 minutos por defecto en .env.example).
barryvdh/laravel-dompdf	Backend	PDF de credenciales institucionales.	Requiere extensiones PHP dom, mbstring y fileinfo.
guzzlehttp/guzzle	Backend	Transporte HTTP para correo.	Aparece en composer audit con avisos vigentes (ver § 7.8).
react-router-dom	Frontend	Enrutado con guardas por rol.	Centraliza redirecciones por rol en src/constants/routes.js.
vite	Frontend	Dev server con proxy y build de producción.	El proxy /v1→127.0.0.1:8000 evita CORS y preflight en desarrollo.
@playwright/test	Frontend	E2E y capturas automatizadas.	Descarga navegadores propios (npx playwright install).
vitest	Frontend	Pruebas unitarias de componentes.	Entorno jsdom; se ejecuta con npm run test:unit.
Hilt / Retrofit / DataStore	Móvil	DI, red y sesión persistente.	El token se cifra con AES-256-GCM y la clave vive en Android Keystore.

Fuente: Auditoría del repositorio.

CAPÍTULO 3. DISEÑO Y MODELADO

Este capítulo documenta los requisitos y modelos del sistema tal como están implementados. La clasificación usada es: **documentado** (declarado en documentos vigentes del proyecto), **inferido** (reconstruido por la auditoría a partir del código) y **verificado** (comprobado en ejecución durante esta auditoría). El proyecto no conserva un documento formal de requisitos aprobados; por transparencia, la matriz completa de requisitos se presenta como reconstrucción evidenciada desde el código, sus pruebas y su ejecución.

3.1. Requisitos funcionales

Tabla 15. Requisitos funcionales (reconstruidos y verificados)

ID	Requisito	Prioridad	Criterio de aceptación (evidencia)	Estado
RF-01	Autenticar usuarios por nombre de usuario y contraseña, con sesión persistente y cierre de sesión.	Alta	Login/logout funcionan por rol; CambioContrasenaObligatorio bloquea el resto de la API en primer ingreso (CredencialesTest).	Verificado
RF-02	Recuperar contraseña por correo con token temporal (60 min) y cambio de contraseña autenticado.	Alta	Flujo completo con broker de Laravel; respuesta genérica ante correo inexistente (PasswordRecoveryTest).	Verificado
RF-03	Gestionar cuentas y perfiles (nombre, apellido, documento, correo, foto, rol, estado) con alta administrativa y credenciales temporales.	Alta	Alta, suspensión, cambio de rol, eliminación con guardas (AuthorizationTest, DeleteGuardsTest).	Verificado
RF-04	Exportar las credenciales institucionales en PDF (por usuario, selección o ficha) y reenviarlas por correo.	Media	PDF generado con Dompdf; reenvío solo con contraseña temporal vigente (CredencialesCorreoTest).	Verificado
RF-05	Gestionar fichas de formación: crear con código de unión generado, asignar instructor y programa, cambiar de estado (activo/finalizado/anulada), editar roster y eliminar con salvaguardas.	Alta	Código único servidor-generado; anulación libera número conservando historial (FichaNumeroUnicoTest, FichaAprendizTest).	Verificado
RF-06	Permitir al aprendiz unirse o salir de una ficha por código, con notificaciones al instructor.	Media	Unión/salida con validaciones 404/422 y avisos (ApprenticeFichaTest).	Verificado
RF-07	Registrar propuestas de proyecto con ciclo borrador → enviado → aprobado/rechazado, edición según estado y bloqueo tras la aprobación.	Alta	Transiciones con historial; reenvío exige cambios reales por huella sha1 (FlujoPropuestaTest, LogicaNegocioTest).	Verificado
RF-08	Gestionar el equipo de una propuesta (agregar/quitar integrantes con reglas de ficha y estado).	Media	Pivote apprentice_projects con invariantes; no se puede quitar al creador (409) (EquipoTest).	Verificado
RF-09	Detectar similitudes entre la propuesta enviada y el corpus aprobado del mismo programa, con porcentaje, señales y pasajes.	Alta	SimilarityController@detect y Recomputador::detectar; umbral y ventana desde configuración (SimilitudServiceTest).	Verificado
RF-10	Conservar evidencia histórica de similitudes (pares vigentes vs. archivados) y recalibrar el corpus globalmente.	Media	Archivado al rechazar/reenviar; recálculo admin (SimilitudesVigentesTest, comando similitud:recalcular).	Verificado

ID	Requisito	Prioridad	Criterio de aceptación (evidencia)	Estado
RF-11	Generar notificaciones internas por eventos (revisión, similitud, observación, ficha, soporte) y gestionar su lectura.	Media	NotificacionesService como punto único; reglas de spoofing cerradas (AuthorizationTest).	Verificado
RF-12	Registrar bitácora de auditoría inmutable y línea de tiempo funcional por propuesta.	Media	audit_logs solo lectura vía API; project_histories visible en detalle (AuditLogTest).	Verificado
RF-13	Ofrecer soporte interno: reportar fallas, clasificarlas y resolverlas con respuesta del administrador.	Media	Flujo completo reporte → revisión → cierre con aviso (SoporteTest).	Verificado
RF-14	Administrar catálogos institucionales: redes de conocimiento y programas de formación.	Media	CRUD con borrado protegido por dependencias (ReglasNegocioFase2Test).	Verificado
RF-15	Publicar un portal público con resumen del sistema y demo del motor de similitud sin sesión.	Baja	/public/resumen y /public/demo-similitud (máx. 300 caracteres; solo títulos y porcentajes) (home-demo E2E).	Verificado
RF-16	Disponer de una aplicación Android con paridad funcional para los tres roles.	Media	Código y configuración completos; compilación no verificada en esta auditoría.	Parcialmente verificado
RF-17	Configurar el motor de similitud (umbral 0,05–0,95 y ventana 1–60 meses) desde el panel administrativo.	Alta	PUT /v1/config-similitud con auditoría config_motor (ValidacionesTest).	Verificado
RF-18	Consultar perfiles de otros usuarios con alcance limitado (compañeros de ficha, instructor, aprendices propios).	Baja	Endpoint perfil con datos mínimos; sin PII (NormalizacionSeguridadTest).	Verificado

Fuente: Reconstrucción de la auditoría a partir del código, los tests y la ejecución (16/10/2026). Requisitos inferidos, no un documento original de requisitos.

3.2. Requisitos no funcionales

Tabla 16. Requisitos no funcionales (reconstruidos y verificados)

ID	Requisito	Evidencia	Estado
RNF-01	Seguridad de acceso: autenticación obligatoria en toda operación privada y autorización por rol y alcance.	Middleware auth:sanctum + cuenta.activa + rol; scopes por ficha; 15 pruebas de AuthorizationTest.	Verificado
RNF-02	Protección de datos personales: documentos, usernames y metadatos de credenciales no se exponen fuera de las vistas administrativas.	GeneralUser::$hidden + CierreAuditoriaTest (regresión BUG-001) + verificación de código del 16/10/2026.	Verificado
RNF-03	Trazabilidad: toda acción administrativa relevante y toda transición de propuesta quedan registradas con actor, fecha y detalle.	audit_logs (inmutable) y project_histories; AuditLogTest y HistorialProyecto.	Verificado
RNF-04	Integridad transaccional en operaciones compuestas (altas con perfiles, envíos con motor, borrados en cascada).	DB::transaction + lockForUpdate; AtomicidadTest.	Verificado
RNF-05	Accesibilidad web: cumplimiento de WCAG A/AA en las rutas del sistema (claro y oscuro).	Especificaciones accesibilidad*.spec.js (axe) e infraestructura de navegación; no re-ejecutadas en esta auditoría.	Parcialmente verificado
RNF-06	Adaptabilidad responsiva en móvil (375 px) y tableta (768 px) sin desbordes.	Especificación responsive.spec.js; capturas móviles reales tomadas en la auditoría.	Parcialmente verificado
RNF-07	Rendimiento percibido: lecturas con caché corta (15 s), deduplicación de peticiones y paginación opcional.	frontend/src/lib/api.js; ?paginado=1 en listados densos.	Verificado (por código)
RNF-08	Compatibilidad de despliegue: empaquetado reproducible y sin	Dockerfile multi-etapa; solo SMTP/Google Fonts son externos y opcionales.	Verificado (por

ID	Requisito	Evidencia	Estado
	dependencia de servicios externos para funcionar.		configuración)
RNF-09	Mantenibilidad: convenciones de código, análisis estático y suites de prueba automatizadas.	244 pruebas backend superadas; 60 unitarias superadas; ESLint 0 hallazgos.	Verificado
RNF-10	Auditabilidad de seguridad: los cambios de credenciales, correo y configuración quedan auditados y los tokens se revocan cuando corresponde.	ChangeEmailTest; auditorías cambiar_clave, cambiar_correo, config_motor.	Verificado
RNF-11	Protección frente a abuso básico: límite de tasa en la API y bloqueo de cuentas suspendidas.	ThrottleRequests:api (límite configurable); middleware cuenta.activa; NormalizacionSeguridadTest.	Verificado
RNF-12	Privacidad del contenido entre pares: un aprendiz no ve propuestas, comentarios ni pasajes ajenos.	VisibilityTest, SimilarityScopeTest, privacidad E2E; verificación de código del 16/10/2026.	Verificado

Fuente: Reconstrucción de la auditoría a partir del código, los tests y la ejecución.

3.3. Matriz de trazabilidad de requisitos

La matriz siguiente vincula cada requisito con su implementación principal y sus pruebas específicas. La versión ampliada (con archivos y líneas) está en el Anexo C.

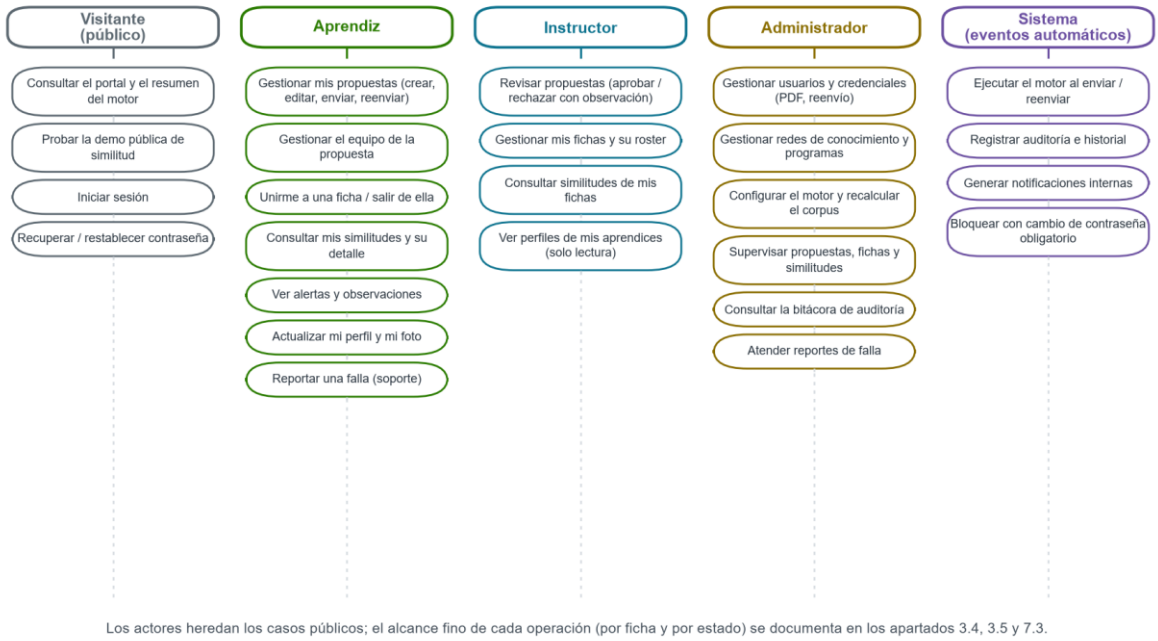
Tabla 17. Trazabilidad requisito → implementación → prueba

Requisito	Implementación principal	Prueba representativa
RF-01, RF-02	AuthController, PersonalAccessToken, broker password_reset_tokens	CredencialesTest, PasswordRecoveryTest
RF-03, RF-04	GeneralUserController, AltaUsuario, CredencialesPdf	DeleteGuardsTest, CredencialesCorreoTest
RF-05, RF-06	ClassGroupController, ApprenticeController	FichaNumeroUnicoTest, ApprenticeFichaTest
RF-07, RF-08	ProjectController, ApprenticeProjectController, Project::puedeEscribir	FlujoPropuestaTest, EquipoTest
RF-09, RF-10, RF-17	SimilarityController, Recomputador, SimilitudService, MotorConfigController	SimilitudServiceTest, SimilitudesVigentesTest, SimilarityPerspectiveTest
RF-11	NotificationController, NotificacionesService	AuthorizationTest (notificaciones)
RF-12	AuditLogController, Auditoria, HistorialProyecto	AuditLogTest, CierreAuditoriaTest
RF-13	BugReportController, SoporteTest	SoporteTest
RF-14	KnowledgeNetworkController, TrainingProgramController	ReglasNegocioFase2Test
RF-15	MotorConfigController@resumen, SimilarityController@demo	home-demo.spec.js (E2E)
RF-16	Código Movil/ (22 ViewModels, 12 repositorios)	Sin pruebas automatizadas propias en el repositorio
RF-18	GeneralUserController@perfil	VisibilityTest, AccesoContraparteTest
RNF-01, RNF-02, RNF-12	Middlewares, scopes y $hidden de GeneralUser	AuthorizationTest, CierreAuditoriaTest
RNF-03, RNF-04, RNF-10	Transacciones, Auditoria, lockForUpdate	AtomicidadTest, ChangeEmailTest
RNF-05, RNF-06	Tokens de diseño, DataTable, layout responsive	accesibilidad*.spec.js, responsive.spec.js (no re-ejecutadas)
RNF-07…RNF-11	api.js, ThrottleRequests, Dockerfile, suites de prueba	Verificación de ejecución § 7.10

Fuente: Auditoría del repositorio.

3.4. Casos de uso relevantes

Figura 2. Casos de uso por actor (reconstruidos del código y la interfaz real)



Fuente: Elaboración propia a partir de las rutas, los guardas y las interfaces verificadas.

La Figura 2 agrupa los casos de uso por actor. A continuación se detallan los tres flujos más representativos; el resto se describe en la matriz de interfaces del Capítulo 8.

Tabla 18. Detalle de casos de uso principales

Caso	Actor	Flujo principal	Reglas y errores
CU-03 Enviar propuesta	Aprendiz (creador o integrante)	1) Edita el borrador · 2) Pulsar Enviar · 3) El backend valida campos mínimos · 4) Calcula huella y compara con la anterior · 5) Ejecuta el motor contra el corpus · 6) Notifica a instructor y coincidencias · 7) Registra historial y auditoría.	403 sin ficha activa o fuera de la ficha; 422 por campos faltantes, reenvío sin cambios o transición inválida.
CU-04 Revisar propuesta	Instructor de la ficha / asignado	1) Abre la cola de revisión · 2) Lee el detalle con historial y observaciones · 3) Aprueba o rechaza con observación · 4) El sistema notifica al aprendiz y registra historial y auditoría.	403 sobre propuestas ajenas; 422 al editar aprobadas; la observación de rechazo queda como comentario.
CU-05 Consultar similitudes	Aprendiz / Instructor / Admin	1) Abre Similitudes · 2) El listado aplica scoping (propias / de sus fichas / global) · 3) El detalle muestra el par A/B, señales y pasajes · 4) El histórico conserva pares archivados.	Aprendiz solo ve pares de sus propuestas; contrapartes de otros alcances se muestran en solo lectura.

Fuente: Auditoría del código (`ProjectController`, `SimilarityController`) y pruebas de la suite backend.

3.5. Actores y permisos

Tabla 19. Matriz de permisos por actor (síntesis del middleware y los scopes)

Recurso / operación	Visitante	Aprendiz	Instructor	Administrador
Portal y demo pública del motor	Sí	Sí	Sí	Sí
Iniciar sesión / recuperar	Sí	Sí	Sí	Sí

Recurso / operación	Visitante	Aprendiz	Instructor	Administrador
contraseña				
Ver y editar el propio perfil	—	Sí	Sí	Sí
Propuestas: ver	—	Solo propias y de su ficha	Sus fichas + mismo programa (lectura)	Todas
Propuestas: crear/editar/enviar	—	Propias, según estado y ficha activa	No crea; aprueba/rechaza en revisión	Intervención excepcional
Equipo de propuesta	—	Según regla de participación	—	Todas
Fichas: unirse/salir	—	Por código	—	—
Fichas: gestionar roster y estado	—	—	Sus fichas	Todas
Similitudes: ver	—	Solo de sus propuestas	De sus fichas	Todas (vigentes e históricas)
Similitudes: recalcular/editar	—	—	—	Sí
Motor: configurar umbral/ventana	—	—	—	Sí
Usuarios y credenciales	—	—	Alta de aprendices en sus fichas	CRUD completo + PDF
Catálogos (redes/programas)	Lectura	Lectura	Lectura	CRUD completo
Notificaciones / alertas	—	Propias	Propias + avisos a sus aprendices	Propias + supervisión
Auditoría (bitácora)	—	—	—	Lectura total
Reportes de soporte	—	Crea y consulta los suyos	Crea y consulta los suyos	Responde y cierra

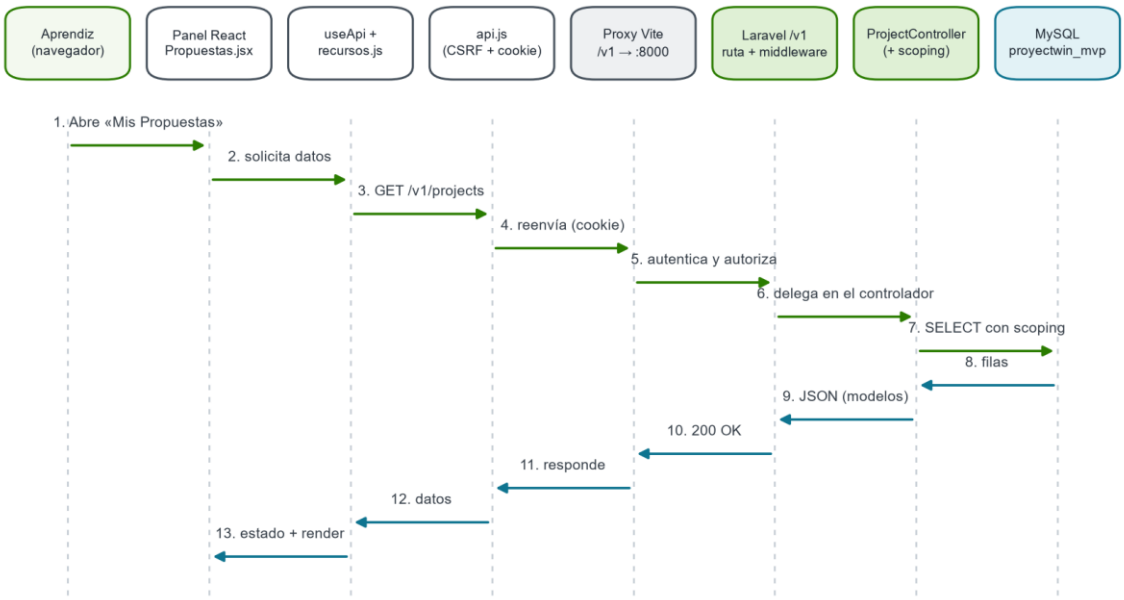
Fuente: `routes/api-v1.php`, middlewares y scopes de modelos (auditoría).

3.6. Diagramas UML pertinentes

Los diagramas se generaron a partir del comportamiento real del código y se referencian donde se explican con detalle:

- **Secuencia de una petición de lectura** (Figura del capítulo 3.8): recorrido completo desde la pantalla hasta MySQL y regreso, con los archivos intervinientes.
- **Estados de una propuesta** (sección 3.7 y Capítulo 6): máquina de estados borrador → pendiente → aprobado/rechazado con reenvío e intervención administrativa.
- **Modelo entidad-relación** (sección 3.9 y Capítulo 4): esquema relacional implementado.
- **Casos de uso** (Figura anterior): actores y operaciones.
- **Despliegue** (Capítulo 5): topología del contenedor y servicios.

Figura 3. Diagrama de secuencia: «el aprendiz abre Mis Propuestas» (GET /v1/projects)



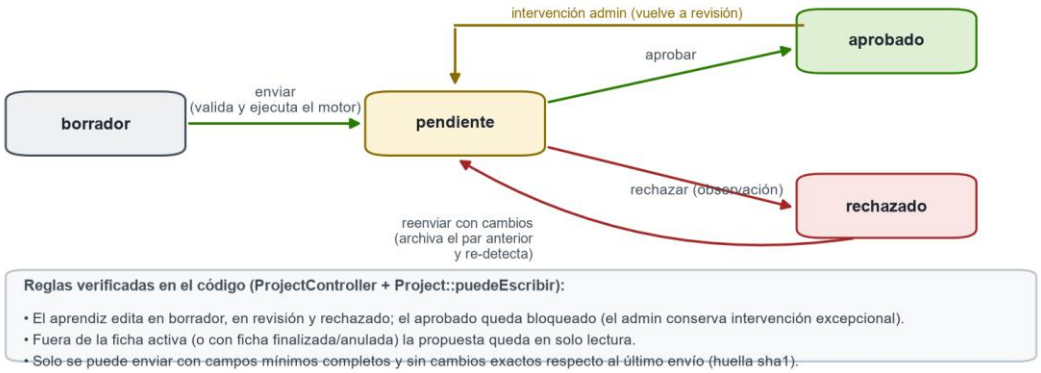
Fuente: Elaboración propia a partir de `Propuestas.jsx`, `useApi.js`, `api.js`, `routes/api-v1.php` y `ProjectController`.

La Figura 3 refleja el flujo verificado con líneas de código identificables: la página llama al hook `useApi` (`frontend/src/pages/aprendiz/Propuestas/Propuestas.jsx`), este usa los contratos de `frontend/src/lib/recursos.js`, y el cliente `frontend/src/lib/api.js` añade credenciales/CSRF. En el backend, la ruta vive en `backend/routes/api-v1.php` y el scoping se aplica en `Project::scopeParaUsuario` (`backend/app/Models/Project.php`).

3.7. Modelo de componentes y estados

Componentes: el sistema se descompone en los artefactos descritos en § 2.1–2.3. En el backend se distinguen: controladores (15), servicios de dominio (4), soporte transversal (6 clases en `app/Support`), motor de similitud (4 clases en `app/Similarity` y `app/Services`), middleware propio (3) y comandos artisan (3). En el frontend: páginas (47), componentes reutilizables (54 archivos `.jsx`, con implementaciones compartidas `*Base` por familia funcional), contextos (sesión, tema y motor), y una capa de datos única (`lib/api.js`). La correspondencia pantalla→componente está en el Capítulo 8.

Figura 4. Máquina de estados de una propuesta de proyecto



Fuente: Elaboración propia a partir de las transiciones implementadas y probadas en `ProjectController`.

La Figura 4 formaliza el ciclo de vida de la propuesta. Las transiciones están impuestas por el controlador y por `Project::puedeEscribir`, y cada cambio relevante se refleja en `project_histories` y en la bitácora cuando corresponde. El envío valida campos mínimos, computa la huella `sha1` del contenido y ejecuta el motor; el reenvío exige que la huella cambie (anti-reenvío vacío) y archiva las detecciones anteriores.

3.8. Diagramas de secuencia de procesos importantes

Proceso «enviar propuesta» (RF-07 + RF-09), pasos verificados en `ProjectController::enviar`:

9. Verificación de sesión, rol y alcance (puedeEscribir) y de estado (borrador o rechazado).
10. Validación de campos mínimos (camposFaltantes: título ≥ 5, descripción ≥ 20, área ≥ 3, objetivo ≥ 15, al menos 2 objetivos específicos ≥ 8 caracteres).
11. Bloqueo de concurrencia con lockForUpdate sobre la propuesta.
12. Cálculo de la huella sha1 del contenido y comparación con huella_envio; si no cambió, se responde 422.
13. Transacción: archivar() de los pares previos si la propuesta venía rechazada, cambio a pendiente y detectar() de similitudes contra el corpus.
14. Notificación al instructor y a los creadores de propuestas coincidentes; registro de historial (enviada/reenviada) y auditoría enviar_propuesta.

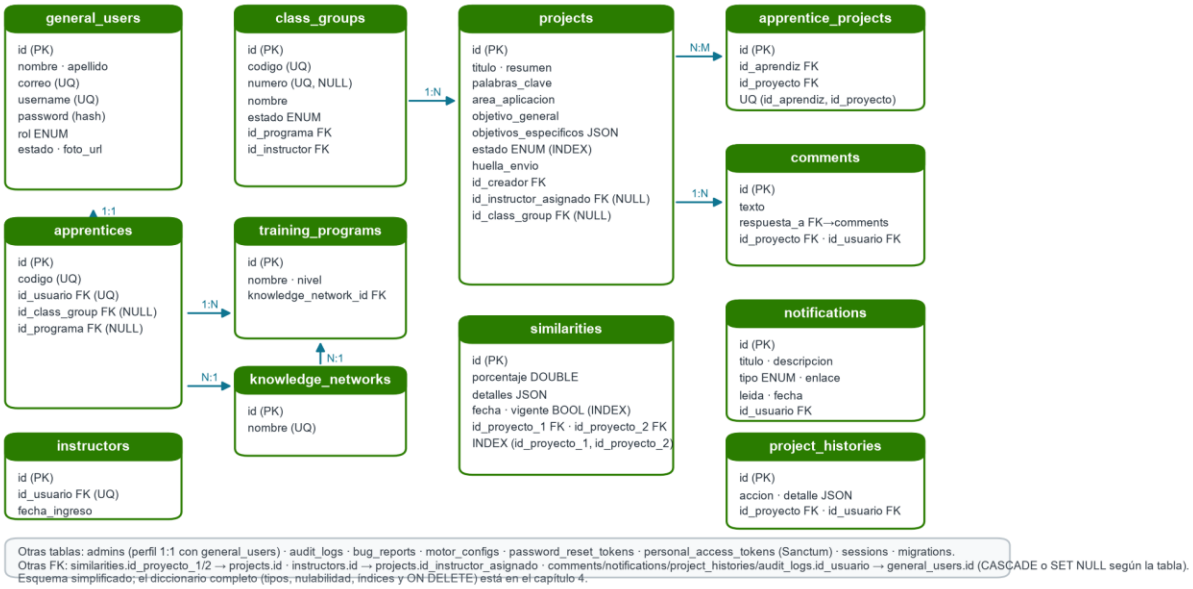
Proceso «aprobar/rechazar» (`ProjectController::update`): el instructor de la ficha o el asignado aplica la transición `pendiente → aprobado|rechazado` (con observación obligatoria en rechazo, guardada como comentario). La aprobación no re-ejecuta el motor (decisión de

diseño verificada en el código); el rechazo archiva los pares vigentes y notifica al equipo. El administrador puede intervenir una propuesta aprobada, lo que la devuelve a `pendiente`, re-detecta similitudes y queda auditado como `devuelta_revision`.

Proceso «unirme a ficha» (`ApprenticeController::unirmeAFicha`): validación del código (404 si no existe; 422 si la ficha no está activa o ya es la propia), transacción que crea/actualiza la fila de aprendiz con derivación del programa de la ficha, y notificaciones al instructor nuevo y al anterior si hubo cambio. La salida limpia `id_class_group` e `id_programa` y notifica igualmente.

3.9. Modelo entidad-relación

Figura 5. Modelo entidad-relación (núcleo del esquema implementado)



Fuente: Elaboración propia a partir de las migraciones reales de `backend/database/migrations`.

La Figura 5 presenta el núcleo del esquema. El sistema es relacional puro: 18 tablas de dominio con claves foráneas declaradas y restricciones de integridad (el diccionario completo, con tipos y reglas ON DELETE, está en el Capítulo 4). Las relaciones estructurales más importantes son: general_users 1:1 con apprentices/instructors/admins; apprentices N:1 con class_groups y training_programs; class_groups N:1 con training_programs e instructors; training_programs N:1 con knowledge_networks; projects N:1 con creador, instructor asignado y ficha, y N:M con apprentices a través de apprentice_projects; similarities como relación N:M reflexiva entre projects (con par normalizado `id_proyecto_1 < id_proyecto_2`); comments

autorreferenciada para hilos; y las tablas de trazabilidad (notifications, bug_reports, audit_logs, project_histories) colgando de usuarios y propuestas.

3.10. Modelo de datos alternativo

No aplica: la persistencia es exclusivamente relacional. Se usan columnas `JSON` para estructuras polimórficas simples (`objetivos_especificos` de las propuestas, `detalles` de similitudes y bitácora, `detalle` del historial), lo que constituye un uso híbrido dentro del mismo motor y no un cambio de modelo de datos. No hay base documental, ni caché distribuida, ni almacenamiento externo de datos.

3.11. Decisiones de diseño identificadas

Tabla 20. Decisiones de diseño con evidencia y compromiso (trade-off)

Decisión	Justificación observada	Compromiso / riesgo asociado
Sesión por cookie httpOnly con SPA en mismo origen	Elimina el almacenamiento de tokens en el navegador y el preflight CORS (documentado en vite.config.js y docs/deploy-railway.md).	Requiere proxy en desarrollo y despliegue conjunto web+API; la app móvil usa token aparte.
Token Bearer para móvil con cifrado en dispositivo	Sanctum permite ambos modos sobre la misma API.	Dos rutas de autenticación que mantener; expiración por variable de entorno.
Motor de similitud local en PHP puro	Sin dependencias de IA de terceros, reproducible y sin costos (comentario en SimilitudService).	Calidad de detección acotada al corpus; sin embeddings semánticos modernos.
Umbral y ventana en tabla de fila única motor_configs	Ajuste operativo sin tocar código, auditado.	Requiere recalcular el corpus tras cambios; pares antiguos se reanalizan.
Huella sha1 anti-reenvío	Evita duplicar propuestas y envíos vacíos (BUG-004 de la auditoría previa).	Cambios triviales de formato cuentan como cambios.
Separar auditoría de seguridad (audit_logs) del historial funcional (project_histories)	Distinta audiencia y distinta inmutabilidad (comentado en los soportes).	Dos mecanismos de escritura que mantener.
Regla «Classroom» de solo lectura fuera de la ficha	Protege el trabajo una vez termina la ficha; probado en InteraccionPropuestaTest.	Usuarios sin ficha conservan propuestas pero no pueden editarlas.
Inclusión de relaciones por lista blanca (?included=)	Control explícito de lo que se expone y carga; evita N+1 arbitrarios.	Sin selección de columnas por relación (mitigado ocultando campos sensibles).
Migraciones consolidadas antes de la entrega	Esquema reproducible desde cero (commit eeff159).	Bases creadas antes de la consolidación podrían requerir migrate:fresh.
Componentes compartidos *Base en el frontend	Una sola implementación para las tres fachadas de rol (comentado en el README del frontend).	Acoplamiento entre rol y presentación a gestionar con cuidado.

Fuente: Código, comentarios del repositorio y auditoría previa (`AUDITORIA-INTEGRAL.md`).

CAPÍTULO 4. BASE DE DATOS

4.1. Motor y versión

El sistema usa **MySQL** como motor relacional (driver `pdo_mysql`). La verificación de esta auditoría se realizó sobre **MariaDB 10.4.32** (compatible), y la configuración de conexión es la estándar de Laravel: `utf8mb4` / `utf8mb4_unicode_ci`, modo estricto activado (`strict => true`) y motor InnoDB (por defecto de MySQL, ya que la configuración no lo fuerza). El entorno de producción previsto (Railway) suministra la base como servicio gestionado de la misma familia. **No existen** bases NoSQL, réplicas ni *sharding*.

4.2. Configuración de conexión

La configuración efectiva se define por variables de entorno (`backend/.env`, no versionado) y `backend/config/database.php`. Las variables (solo nombres; los valores reales no se publican en este documento) son: `DB_CONNECTION` (`mysql`), `DB_HOST`, `DB_PORT` (3306), `DB_DATABASE` (`proyectwin_mvp` en la plantilla), `DB_USERNAME` y `DB_PASSWORD`. En el despliegue, el proveedor inyecta estas variables y el contenedor reintenta la migración hasta 24 veces al arrancar para tolerar el levantamiento de la base (`docker/entrypoint.sh`).

Nota de auditoría: para verificar el sistema sin tocar la base de datos del equipo, la auditoría creó una base separada `proyectwin_mvp_manual` y ejecutó sobre ella migraciones, seeders y pruebas. La base original `proyectwin_mvp` no fue modificada.

4.3. Estructura de la base de datos

El esquema está compuesto por **20 tablas físicas**: 18 tablas de dominio (creadas por las migraciones del repositorio), la tabla `migrations` (control de Laravel) y `personal_access_tokens` (creada por Sanctum desde el paquete). La migración más reciente (`sessions`) se encuentra en el árbol de trabajo sin confirmar en Git al cierre de la auditoría y se documenta como tal. Composición por dominio:

Tabla 21. Tablas de la base de datos y su propósito

#	Tabla	Propósito	Origen
1	general_users	Cuenta única de acceso (aprendiz, instructor o admin) con credenciales y datos personales.	Migración 2026_04_09_023859
2	knowledge_networks	Catálogo de redes de conocimiento.	Migración 2026_04_09_024300
3	training_programs	Catálogo de programas de formación (pertenece a una red).	Migración 2026_04_09_024326
4	instructors	Perfil de instructor (1:1 con general_users).	Migración 2026_04_09_024350
5	class_groups	Fichas de formación con código de unión y estados activo/finalizado/anulada.	Migración 2026_04_09_024400
6	apprentices	Perfil de aprendiz (1:1 con general_users), con ficha y	Migración

#	Tabla	Propósito	Origen
		programa.	2026_04_09_024516
7	admins	Perfil de administrador (1:1 con general_users).	Migración 2026_04_09_030055
8	projects	Propuestas de proyecto formativo con estados y huella de contenido.	Migración 2026_04_09_030220
9	similarities	Pares de propuestas con similitud detectada (vigentes e históricos).	Migración 2026_04_09_030704
10	notifications	Avisos internos por usuario (dominio propio, no la tabla estándar de Laravel).	Migración 2026_04_09_031912
11	bug_reports	Reportes de falla y solicitudes de soporte.	Migración 2026_04_09_032118
12	apprentice_projects	Pivote N:M del equipo de cada propuesta.	Migración 2026_04_09_032359
13	comments	Observaciones y respuestas anidadas sobre propuestas.	Migración 2026_04_09_032402
14	motor_configs	Parámetros del motor (umbral y ventana); fila única.	Migración 2026_04_10_000002
15	audit_logs	Bitácora inmutable de acciones sensibles.	Migración 2026_04_11_000002
16	password_reset_tokens	Tokens del flujo de recuperación de contraseña.	Migración 2026_04_11_000003
17	project_histories	Línea de tiempo funcional de cada propuesta.	Migración 2026_10_06_000002
18	sessions	Sesiones en base de datos (mejora de despliegue; migración en curso al cierre de la auditoría).	Migración 2026_10_16_000001 (sin confirmar en Git)
19	personal_access_tokens	Tokens Sanctum (móvil y pruebas).	Paquete laravel/sanctum 3.x
20	migrations	Control interno de migraciones ejecutadas.	Framework Laravel

Fuente: `backend/database/migrations` y verificación en MariaDB (16/10/2026).

4.4. Tablas, colecciones o estructuras persistentes

Además de las tablas, el sistema persiste: sesiones y caché en archivos (en producción se prevé `sessions` en base de datos), logs en `storage/logs/laravel.log` y fotografías de perfil como cadenas en `general_users.foto_url`. No hay colecciones documentales ni almacenamiento de objetos externo.

4.5. Diccionario de datos

Diccionario completo de las 18 tablas de dominio, en su **estado final consolidado** (las migraciones eliminadas durante la refactorización quedaron fusionadas en los archivos de creación). Convenciones: `UQ` = restricción única; `FK` = clave foránea; los tipos se expresan en su forma DDL de MySQL. Ninguna tabla declara comentarios DDL (los comentarios citados provienen del código fuente).

4.5.1. `general_users`

Cuenta base del sistema (los perfiles viven en `apprentices`, `instructors` o `admins`). Implementa la autenticación (`Authenticatable`) y la recuperación de contraseña por correo.

Tabla 22. Diccionario de datos — general_users

Columna	Tipo	Nulo	Predet.	Notas

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
nombre	VARCHAR(255)	No	—	
apellido	VARCHAR(255)	No	—	
tipo_documento	VARCHAR(20)	Sí	NULL	p. ej. CC en los datos demo
numero_documento	VARCHAR(40)	Sí	NULL	UQ; oculto en JSON por $hidden
correo	VARCHAR(255)	No	—	UQ; usado para recuperación
username	VARCHAR(255)	Sí	NULL	UQ; credencial de acceso (login por usuario)
password	VARCHAR(255)	No	—	Hash bcrypt
must_change_password	TINYINT(1)	No	0	Primer ingreso con contraseña temporal
password_temporal	TEXT	Sí	NULL	Cifrada con Crypt; solo para exportar el PDF
credenciales_enviadas_en	TIMESTAMP	Sí	NULL	Bitácora de envío de credenciales
credenciales_error	VARCHAR(500)	Sí	NULL	Último error de envío (si hubo)
foto_url	TEXT	Sí	NULL	Data URL o URL; procesada a 256 px en el cliente
rol	ENUM('aprendiz','instructor','admin')	No	—	Rol único del sistema
estado	TINYINT(1)	No	1	1 activo / 0 suspendido
remember_token	VARCHAR(100)	Sí	NULL	Recordar sesión
created_at / updated_at	TIMESTAMP	Sí	NULL	Auditoría de fila

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.2. `knowledge_networks`

Catálogo institucional de redes de conocimiento; agrupa programas.

Tabla 23. Diccionario de datos — knowledge_networks

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
nombre	VARCHAR(255)	No	—	UQ
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.3. `training_programs`

Programas de formación (ADSO, Multimedia, etc.) vinculados a una red. La columna histórica `num_trimestres` fue retirada y no debe reintroducirse (comentario en la migración).

Tabla 24. Diccionario de datos — training_programs

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
nombre	VARCHAR(255)	No	—	
nivel	VARCHAR(255)	No	—	p. ej. «Tecnólogo»
knowledge_network_id	BIGINT UNSIGNED	No	—	FK → knowledge_networks.id (RESTRICT)
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.4. `instructors`

Perfil 1:1 del rol instructor.

Tabla 25. Diccionario de datos — instructors

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
fecha_ingreso	DATE	No	—	Normalizada a Y-m-d por el modelo
id_usuario	BIGINT UNSIGNED	No	—	UQ; FK → general_users.id (CASCADE)
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.5. `class_groups`

Fichas de formación. El código de unión es una credencial: lo genera el servidor.

Tabla 26. Diccionario de datos — class_groups

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
codigo	VARCHAR(255)	No	—	UQ; formato xxx-xxxx
numero	VARCHAR(255)	Sí	NULL	UQ; al anular se libera (NULL) conservando historial
nombre	VARCHAR(255)	No	—	
estado	ENUM('activo','finalizado','anulada')	No	'activo'	Gobierna la participación de aprendices
id_programa	BIGINT UNSIGNED	No	—	FK → training_programs.id (RESTRICT)
id_instructor	BIGINT UNSIGNED	No	—	FK → instructors.id (RESTRICT)
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.6. `apprentices`

Perfil 1:1 del rol aprendiz; puede existir sin ficha. Hallazgo histórico: el borrado de una ficha elimina la fila del aprendiz (CASCADE) y con ella sus vínculos de equipo; la auditoría previa lo documentó y el flujo de «salir de ficha» mitiga el caso normal anulando la referencia.

Tabla 27. Diccionario de datos — apprentices

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
codigo	VARCHAR(255)	No	—	UQ; formato AP-###
id_class_group	BIGINT UNSIGNED	Sí	NULL	FK → class_groups.id (CASCADE al borrar ficha)
id_usuario	BIGINT UNSIGNED	No	—	UQ; FK → general_users.id (CASCADE)
id_programa	BIGINT UNSIGNED	Sí	NULL	FK → training_programs.id (RESTRICT); sin ficha puede ser NULL
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.7. `admins`

Perfil 1:1 del rol administrador.

Tabla 28. Diccionario de datos — admins

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
id_usuario	BIGINT UNSIGNED	No	—	UQ; FK → general_users.id (CASCADE)
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.8. `projects`

Propuestas de proyecto formativo; nacen como borrador y admiten campos incompletos hasta el envío.

Tabla 29. Diccionario de datos — projects

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
titulo	VARCHAR(255)	Sí	NULL	Obligatorio para enviar

Columna	Tipo	Nulo	Predet.	Notas
				(validación de negocio)
resumen	TEXT	Sí	NULL	
palabras_clave	TEXT	Sí	NULL	
area_aplicacion	VARCHAR(255)	Sí	NULL	
objetivo_general	TEXT	Sí	NULL	
objetivos_especificos	JSON	Sí	NULL	Cast a arreglo en el modelo
estado	ENUM('borrador','pendiente','aprobado','rechazado')	No	'pendiente'	INDEX
huella_envio	VARCHAR(40)	Sí	NULL	sha1 del contenido enviado
id_creador	BIGINT UNSIGNED	No	—	FK → general_users.id (CASCADE)
id_instructor_asignado	BIGINT UNSIGNED	Sí	NULL	FK → instructors.id (SET NULL)
id_class_group	BIGINT UNSIGNED	Sí	NULL	FK → class_groups.id (RESTRICT: no se borra ficha con propuestas)
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.9. `similarities`

Pares de propuestas con similitud detectada; guarda siempre el id menor primero. Índice normal (no único) sobre `(id_proyecto_1, id_proyecto_2)`: el mismo par puede reaparecer como versiones vigente/histórica; la serialización del motor se logra con bloqueo sobre `motor_configs`.

Tabla 30. Diccionario de datos — similarities

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
porcentaje	DOUBLE	No	—	0–100
detalles	JSON	Sí	NULL	Señales, pesos, términos y pasajes
fecha	DATE	Sí	NULL	
id_proyecto_1	BIGINT UNSIGNED	No	—	FK → projects.id (CASCADE)
id_proyecto_2	BIGINT UNSIGNED	No	—	FK → projects.id (CASCADE)
vigente	TINYINT(1)	No	1	1 = contenido actual; 0 = evidencia histórica. INDEX
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.10. `notifications`

Avisos internos por usuario. Es una tabla del dominio (no la tabla nativa de notificaciones de Laravel).

Tabla 31. Diccionario de datos — notifications

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
titulo	VARCHAR(255)	No	—	
descripcion	TEXT	Sí	NULL	
tipo	ENUM('similitud','observacion','revision','mensaje','sistema')	No	—	Validado también en el controlador
enlace	VARCHAR(255)	Sí	NULL	Formato proyecto:{id}, ficha:{id}, reporte:{id}
leida	TINYINT(1)	No	0	
fecha	DATE	No	—	
id_usuario	BIGINT UNSIGNED	No	—	FK → general_users.id (CASCADE)
created_at /	TIMESTAMP	Sí	NULL	

Columna	Tipo	Nulo	Predet.	Notas
updated_at				

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.11. `bug_reports`

Reportes de falla del usuario; también funciona como bandeja de soporte por conflictos de ficha.

Tabla 32. Diccionario de datos — bug_reports

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
titulo	VARCHAR(255)	Sí	NULL	Autogenerado si no llega («Conflicto con la ficha N»)
numero_ficha	VARCHAR(30)	Sí	NULL	Ficha involucrada (soporte)
motivo	VARCHAR(120)	Sí	NULL	Motivo corto del soporte
descripcion	TEXT	No	—	
tipo	ENUM('sistema','proyecto','datos','bug_ui','error_datos','rendimiento','seguridad','otro')	No	—	
estado	ENUM('pendiente','en_revision','resuelto','cerrado','rechazado')	No	'pendiente'	El alta siempre inicia en pendiente
respuesta	TEXT	Sí	NULL	Respuesta con la que el admin cierra
fecha	DATE	No	—	
id_usuario	BIGINT UNSIGNED	No	—	FK → general_users.id (CASCADE)
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.12. `apprentice_projects`

Pivote del equipo de trabajo: qué aprendices participan en qué propuesta. UQ compuesta `(id_aprendiz, id_proyecto)`: una participación única por propuesta. El creador no puede retirarse de su propio equipo (regla 409).

Tabla 33. Diccionario de datos — apprentice_projects

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
id_aprendiz	BIGINT UNSIGNED	No	—	FK → apprentices.id (CASCADE)
id_proyecto	BIGINT UNSIGNED	No	—	FK → projects.id (CASCADE)
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.13. `comments`

Observaciones sobre propuestas, con respuestas anidadas.

Tabla 34. Diccionario de datos — comments

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
texto	TEXT	No	—	

Columna	Tipo	Nulo	Predet.	Notas
id_proyecto	BIGINT UNSIGNED	No	—	FK → projects.id (CASCADE)
id_usuario	BIGINT UNSIGNED	No	—	FK → general_users.id (CASCADE); firmado por el servidor
respuesta_a	BIGINT UNSIGNED	Sí	NULL	FK autorreferente → comments.id (CASCADE: borra el hilo completo)
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.14. `motor_configs`

Parámetros del motor de similitud. Tabla de fila única (id = 1).

Tabla 35. Diccionario de datos — motor_configs

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK (convención: solo existe id 1)
umbral	DOUBLE	No	0.2	Porcentaje mínimo; el seeder lo fija en 0.30. Rango validado 0,05–0,95
meses	INT	No	12	Ventana del corpus. Rango validado 1–60
created_at / updated_at	TIMESTAMP	Sí	NULL	

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.15. `audit_logs`

Bitácora de acciones sensibles: inmutable (solo inserción y consulta por API).

Tabla 36. Diccionario de datos — audit_logs

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED AUTO_INCREMENT	No	—	PK
id_usuario	BIGINT UNSIGNED	Sí	NULL	FK → general_users.id (SET NULL): si se borra el actor, la fila se conserva
accion	VARCHAR(255)	No	—	INDEX; p. ej. crear_usuario, enviar_propuesta, config_motor
entidad	VARCHAR(255)	Sí	NULL	Composición de INDEX con entidad_id
entidad_id	BIGINT UNSIGNED	Sí	NULL	Referencia lógica polimórfica (sin FK)
detalle	JSON	Sí	NULL	Cast a arreglo
ip	VARCHAR(255)	Sí	NULL	IP del actor (tras proxy, según TrustProxies)
created_at / updated_at	TIMESTAMP	Sí	NULL	INDEX en created_at para consultas por fecha

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.16. `password_reset_tokens`

Tokens del flujo de recuperación de contraseña (broker de Laravel).

Tabla 37. Diccionario de datos — password_reset_tokens

Columna	Tipo	Nulo	Predet.	Notas
email	VARCHAR(255)	No	—	PK; almacena el valor de general_users.correo
token	VARCHAR(255)	No	—	Hash del token enviado por correo
created_at	TIMESTAMP	Sí	NULL	Vigencia: 60 minutos; throttling 60 s

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.17. `project_histories`

Evolución funcional y visible de cada propuesta (separada de la bitácora de seguridad).

Tabla 38. Diccionario de datos — project_histories

Columna	Tipo	Nulo	Predet.	Notas
id	BIGINT UNSIGNED	No	—	PK

Columna	Tipo	Nulo	Predet.	Notas
	AUTO_INCREMENT			
id_proyecto	BIGINT UNSIGNED	No	—	FK → projects.id (CASCADE)
id_usuario	BIGINT UNSIGNED	Sí	NULL	FK → general_users.id (SET NULL)
accion	VARCHAR(255)	No	—	creada, enviada, reenviada, actualizada, aprobada, rechazada, devuelta_revision
detalle	JSON	Sí	NULL	Cast a arreglo
created_at / updated_at	TIMESTAMP	Sí	NULL	INDEX compuesto (id_proyecto, created_at)

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.18. `sessions`

Almacenamiento de sesiones en base de datos (mejora de despliegue para no perder sesiones al redesplegar). Al cierre de la auditoría esta migración estaba en el árbol de trabajo sin confirmar en Git; su adopción requiere además `SESSION_DRIVER=database` en el entorno.

Tabla 39. Diccionario de datos — sessions

Columna	Tipo	Nulo	Predet.	Notas
id	VARCHAR(255)	No	—	PK
user_id	BIGINT UNSIGNED	Sí	NULL	INDEX (no FK declarada)
ip_address	VARCHAR(45)	Sí	NULL	
user_agent	TEXT	Sí	NULL	
payload	LONGTEXT	No	—	Contenido serializado de la sesión
last_activity	INT	No	—	INDEX

Fuente: Migración correspondiente en `backend/database/migrations`.

4.5.19. `personal_access_tokens` (gestionada por Sanctum)

Tabla estándar del paquete `laravel/sanctum` (no versionada en `database/migrations`; el paquete la registra automáticamente). Almacena los tokens Bearer: `id` (PK), `tokenable_type/tokenable_id` (morfología hacia `general_users`), `name`, `token` (UQ, hash sha256), `abilities`, `last_used_at`, `expires_at` y timestamps. La expiración se controla con `SANCTUM_EXPIRATION` (1440 minutos en la plantilla).

4.6. Tipos de datos

Tabla 40. Tipos de datos empleados (convención Laravel 10 → MySQL)

Declaración en la migración	Tipo MySQL resultante	Uso en el esquema
id() / bigIncrements	BIGINT UNSIGNED AUTO_INCREMENT	Claves primarias de todas las tablas
foreignId()	BIGINT UNSIGNED	Claves foráneas
string('x')	VARCHAR(255)	Nombres, títulos, códigos, enums textuales de bajo volumen
text() / longText	TEXT / LONGTEXT	Descripciones, objetivos, respuestas, payload de sesión
json()	JSON	objetivos_especificos, detalles, detalle (histórico/auditoría)
enum(...)	ENUM	rol, estados de ficha/propuesta/reporte, tipos de notificación
boolean()	TINYINT(1)	estado, leida, vigente, must_change_password
float()	DOUBLE	porcentaje y umbral (nota: Laravel mapea float a DOUBLE en MySQL)
integer()	INT	meses del motor, last_activity
date()	DATE	fecha de ingreso, fecha de similitud/aviso/reporte
timestamps()	TIMESTAMP NULL	created_at / updated_at

Declaración en la migración	Tipo MySQL resultante	Uso en el esquema
timestamp() / rememberToken()	TIMESTAMP / VARCHAR(100)	Marcas de credenciales y «recordarme»

Fuente: Migraciones reales (auditoría).

4.7. Claves primarias y foráneas

Tabla 41. Claves foráneas declaradas (estado final)

Tabla	Columna	Referencia	ON DELETE
training_programs	knowledge_network_id	knowledge_networks.id	RESTRICT (implícito)
instructors	id_usuario	general_users.id	CASCADE
class_groups	id_programa	training_programs.id	RESTRICT (implícito)
class_groups	id_instructor	instructors.id	RESTRICT (implícito)
apprentices	id_class_group	class_groups.id	CASCADE
apprentices	id_usuario	general_users.id	CASCADE
apprentices	id_programa	training_programs.id	RESTRICT
admins	id_usuario	general_users.id	CASCADE
projects	id_creador	general_users.id	CASCADE
projects	id_instructor_asignado	instructors.id	SET NULL
projects	id_class_group	class_groups.id	RESTRICT
similarities	id_proyecto_1 / id_proyecto_2	projects.id	CASCADE en ambas
notifications	id_usuario	general_users.id	CASCADE
bug_reports	id_usuario	general_users.id	CASCADE
apprentice_projects	id_aprendiz	apprentices.id	CASCADE
apprentice_projects	id_proyecto	projects.id	CASCADE
comments	id_proyecto	projects.id	CASCADE
comments	id_usuario	general_users.id	CASCADE
comments	respuesta_a	comments.id	CASCADE (borra el hilo)
audit_logs	id_usuario	general_users.id	SET NULL
project_histories	id_proyecto	projects.id	CASCADE
project_histories	id_usuario	general_users.id	SET NULL

Fuente: Migraciones (auditoría). ON UPDATE no se declara en ninguna FK (comportamiento por defecto).

Las claves primarias son `BIGINT UNSIGNED AUTO_INCREMENT` en todas las tablas salvo `password_reset_tokens` (PK `email`) y `sessions` (PK `id` de tipo VARCHAR).

4.8. Relaciones y restricciones

- **Un perfil por usuario** por rol: instructors.id_usuario, apprentices.id_usuario y admins.id_usuario son únicos.
- **Par de similitud normalizado:** el modelo ordena id_proyecto_1 < id_proyecto_2 antes de guardar (invariante probado en IntegridadDatosTest).
- **Participación única de equipo:** UQ compuesta en apprentice_projects.
- **Número de ficha reutilizable:** class_groups.numero es único pero admite NULL; la anulación libera el número conservando el historial (FichaNumeroUnicoTest).
- **Códigos únicos generados por el servidor:** class_groups.codigo (xxx-xxxx), apprentices.codigo (AP-###), general_users.username (patrón seguro) y numero_documento.

- **Borrados con salvaguardas:** la ficha con propuestas se anula en lugar de eliminarse; el programa con aprendices no se borra (RESTRICT); el usuario no puede autoeliminarse ni dejar el sistema sin administradores activos (409).
- **Retención de evidencia:** las similitudes archivadas nunca se borran al rechazar o reenviar; la bitácora no expone operaciones de escritura vía API.

4.9. Índices

Tabla 42. Índices declarados explícitamente

Tabla	Índice	Objetivo
projects	estado	Filtros de bandeja por estado (añadido en la consolidación).
similarities	(id_proyecto_1, id_proyecto_2) (normal)	Búsqueda de pares por propuesta; reemplazó al UNIQUE al introducir el histórico.
similarities	vigente	Separar pares actuales de evidencia histórica.
audit_logs	accion; (entidad, entidad_id); created_at	Consulta por acción, por entidad afectada y por rango de fechas.
project_histories	(id_proyecto, created_at)	Línea de tiempo por propuesta.
personal_access_tokens	(tokenable_type, tokenable_id); token UQ	Búsqueda y revocación de tokens Sanctum.

Fuente: Migraciones y paquete Sanctum (auditoría).

4.10. Migraciones y scripts de inicialización

Existen 18 migraciones (listadas en § 4.3 con su tabla). No hay migraciones tipo ALTER vigentes: las modificaciones históricas fueron consolidadas dentro de las creaciones (commit `eeff159`), de modo que `migrate:fresh` reproduce el esquema final exacto. La inicialización de datos se realiza con dos seeders:

- **`DatabaseSeeder`** (invocado por defecto): escenario completo de demostración — 15 usuarios (9 aprendices, 4 instructores, 2 admins), 4 fichas, 3 programas, 2 redes, 16 propuestas en todos los estados, equipos, similitudes (incluye un par histórico deliberado), notificaciones, historiales, 6 reportes de soporte, comentarios y auditorías. Ejecuta además el motor real (Recomputador::recalcular) para dejar datos coherentes.
- **`VolumenSeeder`** (manual): añade 5 instructores, 5 fichas, 40 aprendices y 80 propuestas para probar listados y paginación con volumen. El README lo recomienda únicamente para pruebas de carga visual.

```
# Preparación completa del esquema y datos demo (verificado en la auditoría)
php artisan migrate:fresh --seed        # 18 migraciones + escenario demo
php artisan db:seed --class=VolumenSeeder   # (opcional) datos de volumen
php artisan similitud:recalcular        # recalibra el corpus con el umbral vigente
php artisan similitud:calibrar          # sugiere umbral por programa (opcional)
php artisan perfiles:docentes           # backfill de perfiles de instructor faltantes
```

Fragmento 1 — Archivo: Comandos verificados contra `backend/README.md` y ejecución real del 16/10/2026.

4.11. Copia de seguridad y restauración

No se encontró ningún procedimiento de copia de seguridad implementado en el repositorio (ni scripts de respaldo, ni configuración de retención). Se documentan a continuación **procedimientos recomendados** —no implementados— para el entorno MySQL/MariaDB y para el servicio gestionado de Railway:

```
# Respaldo lógico recomendado (entorno local o servidor con acceso CLI)
mysqldump -u USUARIO -p --single-transaction --routines --triggers \
  proyectwin_mvp > respaldo_proyectwin_AAAA-MM-DD.sql

# Restauración
mysql -u USUARIO -p proyectwin_mvp < respaldo_proyectwin_AAAA-MM-DD.sql
```

Fragmento 2 — Archivo: Procedimiento recomendado (no existe en el repositorio).

- En Railway, el plugin MySQL gestiona copias propias del proveedor; deben verificarse en el panel y complementarse con un volcado lógico programado si el dato es crítico.
- Frecuencia sugerida: diaria en periodos de uso activo; retención mínima de 30 días; prueba de restauración trimestral.
- Antes de aplicar cambios estructurales (migraciones nuevas), ejecutar un respaldo manual y verificar php artisan migrate --pretend cuando sea viable.

4.12. Integridad y validación de datos

La integridad se protege en tres niveles: (1) **restricciones de base de datos** (claves, únicos, restricciones de borrado y modo estricto); (2) **validaciones de la aplicación** (reglas `validate()` en cada controlador, listas blancas de campos, invariantes de dominio como el orden del par o la participación única); y (3) **transacciones y bloqueos** en las operaciones compuestas.

Como evidencia, se ejecutaron sobre la base recién sembrada (`proyectwin_mvp_manual`, 16/10/2026, modo solo lectura) diez consultas de integridad; todas devolvieron cero incidencias:

Tabla 43. Verificación de integridad ejecutada durante la auditoría

Chequeo	Resultado
Pivotes de equipo con proyecto o aprendiz inexistente	0
Comentarios y notificaciones con usuario inexistente	0
Propuestas con ficha inexistente	0
Pares de similitud vigentes duplicados	0
Porcentajes fuera del rango 0–100	0
Usuarios sin username y aprendices sin codigo	0
Pares con orden invertido (id_proyecto_1 >= id_proyecto_2)	0
Distribución de propuestas resultante del seed	1 borrador · 10 pendientes · 3 aprobadas · 2 rechazadas
Pares de similitud resultantes del motor	7 vigentes · 3 históricos

Fuente: Consultas ejecutadas por la auditoría sobre `proyectwin_mvp_manual` (16/10/2026).

Limitación: la auditoría no ejecutó pruebas de concurrencia (dos envíos simultáneos sobre la misma propuesta o ficha). El código mitiga la concurrencia con `lockForUpdate`, pero la garantía no fue comprobada empíricamente en esta revisión.

CAPÍTULO 5. INSTALACIÓN Y CONFIGURACIÓN

Este capítulo describe el procedimiento completo para poner ProyecTwin en funcionamiento desde un entorno limpio. **Todos los comandos de las secciones 5.5 a 5.11 fueron ejecutados y verificados** por la auditoría del 16/10/2026 sobre Windows con XAMPP; los comandos de contenedores y despliegue (5.12–5.13) provienen de los archivos del repositorio y se ajustan a su contenido, pero **el empaquetado Docker no pudo ejecutarse en el equipo de auditoría** (Docker no está instalado), lo que se señala donde corresponde.

5.1. Requisitos previos

Tabla 44. Requisitos previos del entorno de desarrollo

Componente	Requisito	Verificado con
PHP	8.1 o superior con extensiones: pdo_mysql, mbstring, openssl, dom, tokenizer, xml, curl, fileinfo, bcmath	PHP 8.2.12
Composer	2.x	2.10.2
MySQL o MariaDB	MySQL 5.7+ / MariaDB equivalente, en ejecución y accesible por TCP	MariaDB 10.4.32 (puerto 3306)
Node.js + npm	20.19+/22.12+ (requisito de Vite 8); recomendado LTS reciente	Node 26.3.1 + npm 11.16.0
Git	Para clonar el repositorio	Repositorio clonado
Navegador moderno	Chrome/Edge/Firefox actuales	Chromium (Playwright)
(Opcional) Android Studio + SDK 36	Solo para compilar la app Movil/	SDK presente; compilación no ejecutada
(Opcional) Docker	Solo para construir el contenedor de producción	No disponible en la auditoría

Fuente: Manifiestos del proyecto y entorno de la auditoría.

5.2. Instalación de herramientas

Windows (entorno verificado): instalar XAMPP (o cualquier distribución con PHP 8.1+ y MySQL/MariaDB), Composer y Node.js LTS. En la auditoría se usó el PHP de XAMPP (`C:\xampp\php`) con las extensiones requeridas ya habilitadas. Verificar con:

```
php -v
composer --version
node -v
npm -v
```

Fragmento 3 — Archivo: Comandos de verificación (ejecutados).

Linux/macOS: usar el gestor de paquetes de la distribución (`apt`, `brew`, etc.) para PHP, Composer y Node; instalar MySQL/MariaDB por separado. Los comandos del resto del capítulo son idénticos salvo las diferencias de rutas señaladas.

5.3. Obtención del código fuente

```
git clone https://github.com/YDanB-312/ProyecTwin-MVP.git
cd ProyecTwin-MVP
```

Fragmento 4 — Archivo: Repositorio remoto declarado en git (`origin`). La rama de trabajo es `master`.

5.4. Estructura inicial del proyecto

Tras clonar, el repositorio presenta esta estructura de primer nivel (el árbol detallado está en el Anexo A):

```
ProyecTwin-MVP/
├── backend/          # API Laravel (composer.json, app/, routes/api-v1.php, database/,
tests/)
├── frontend/         # SPA React + Vite (package.json, src/, e2e/, public/)
├── Movil/            # App Android Kotlin + Compose (gradle/, app/src/main/)
├── docker/           # nginx.conf y entrypoint.sh del contenedor de producción
├── docs/             # documentación interna (arquitectura, despliegue, guion, normativa
visual)
├── Dockerfile        # empaquetado multi-etapa del servicio web
├── railway.json      # configuración de despliegue (healthcheck /up)
└── AUDITORIA-INTEGRAL.md / HISTORIAL-FIXES.md   # informes internos
```

Fragmento 5 — Archivo: Árbol real del repositorio (16/10/2026).

5.5. Instalación de dependencias

Backend. Desde `backend/`:

```
composer install
```

Fragmento 6 — Archivo: Ejecutado en la auditoría: descargó y descubrió paquetes sin errores.

Frontend. Desde `frontend/`:

```
npm install
```

Fragmento 7 — Archivo: Ejecutado en la auditoría: 227 paquetes instalados en ~32 s.

Resultado esperado: `backend/vendor/autoload.php` y `frontend/node_modules/` existen. Ambos directorios están excluidos del control de versiones (`.gitignore`).

5.6. Variables de entorno

El backend usa `backend/.env` (no versionado). Crear a partir de la plantilla y generar la clave:

```
cd backend
copy .env.example .env        # Windows (o: cp .env.example .env)
php artisan key:generate
```

Fragmento 8 — Archivo: Ejecutado en la auditoría (clave generada correctamente).

Variables relevantes (solo nombres y valores de plantilla; **no se publican secretos**): `APP_NAME=ProyecTwin-MVP`, `APP_URL`, `DB_CONNECTION=mysql`, `DB_HOST=127.0.0.1`, `DB_PORT=3306`, `DB_DATABASE=proyectwin_mvp`, `DB_USERNAME`, `DB_PASSWORD`, `FRONTEND_URL=http://localhost:5173`, `SANCTUM_STATEFUL_DOMAINS` (orígenes del SPA), `SANCTUM_EXPIRATION=1440`, `MAIL_MAILER=log` (en desarrollo el correo se escribe en `storage/logs/laravel.log`), `API_RATE_LIMIT`, `APP_TIMEZONE=America/Bogota`. El archivo de ejemplo incluye comentarios con las recomendaciones para producción (SMTP verificado, cookies seguras).

5.7. Configuración de servicios

Base de datos. Crear la base con codificación completa (comando `mysql` de XAMPP o del servidor instalado):

```
mysql -u root -e "CREATE DATABASE IF NOT EXISTS proyectwin_mvp CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;"
```

Fragmento 9 — Archivo: Ejecutado en la auditoría (se creó además una base separada para no alterar datos existentes).

Verificar que el servicio de MySQL/MariaDB esté en ejecución y que `DB_USERNAME`/`DB_PASSWORD` del `.env` tengan permisos sobre la base. En XAMPP el servicio se inició desde el panel de control; el puerto verificado fue 3306 en `127.0.0.1`.

Correo (opcional). En desarrollo no requiere configuración (`MAIL_MAILER=log`). Para envíos reales (credenciales, recuperación), configurar `MAIL_MAILER=smtp` con un proveedor y remitente verificados; sin SMTP la cuenta se crea igualmente y el error de envío queda registrado en la cuenta (`credenciales_error`).

5.8. Configuración de la base de datos

- Ajustar DB_DATABASE (por defecto proyectwin_mvp) y credenciales en backend/.env.
- **Advertencia operativa:** las pruebas de backend usan la conexión del .env (no hay base de pruebas separada; phpunit.xml trae la configuración SQLite comentada). Ejecutar las pruebas sobre una base de desarrollo, nunca sobre producción.
- El esquema se reconstruye íntegramente con migraciones (§ 5.9); no se requieren scripts SQL manuales.

5.9. Inicialización y carga de datos

Desde `backend/`:

```
php artisan migrate:fresh --seed
```

Fragmento 10 — Archivo: Ejecutado en la auditoría: 18 migraciones + documento base; sembró 15 usuarios, 16 propuestas, 10 similitudes y 30 notificaciones.

El seeder usa el flujo real de la aplicación (incluido el motor de similitud), por lo que los datos quedan coherentes con las reglas de negocio. Para datos de volumen (paginación y filtros):

```
php artisan db:seed --class=VolumenSeeder
```

Fragmento 11 — Archivo: Documentado en el README del backend (opcional).

5.10. Ejecución en desarrollo

Terminal 1 — API (desde `backend/`):

```
php artisan serve --host=127.0.0.1 --port=8000
```

Fragmento 12 — Archivo: Ejecutado y verificado en la auditoría.

Terminal 2 — SPA (desde `frontend/`):

```
npm run dev
```

Fragmento 13 — Archivo: Ejecutado y verificado (Vite en el puerto 5173 con proxy /v1 → :8000).

Abrir `http://localhost:5173`. El frontend **no tiene capa de datos simulada**: todas las pantallas consumen la API real. El proxy de Vite hacia `127.0.0.1:8000` (no `localhost:8000`) está comentado en `vite.config.js` para evitar que Windows resuelva `localhost` a IPv6 mientras `artisan serve` escucha en IPv4.

5.11. Verificación del funcionamiento

Comprobaciones ejecutadas durante la auditoría y su resultado:

Tabla 45. Lista de verificación posterior a la instalación

Comprobación	Procedimiento	Resultado obtenido (16/10/2026)
API en línea	Invoke-WebRequest http://127.0.0.1:8000/up	200 — {"status":"ok"}
SPA en línea	Abrir http://localhost:5173	200 — portal público cargado
Autenticación	Iniciar sesión con las cuentas de demostración del seeder	Redirección al panel por rol (aprendiz/instructor/admin)
Ciclo de datos	Listado de propuestas y similitudes con datos del seed	16 propuestas y 10 pares de similitud visibles según alcance
Datos demo	Credenciales del seeder: mgonzalez/123456, cruiz/123456, a/admin123	Son credenciales **de demostración**; no reutilizar en entornos reales
Suite backend	php artisan test (desde backend/)	244 pruebas superadas, 1.156 aserciones (35,9 s)
Unitarias frontend	npm run test:unit (desde frontend/)	17 archivos, 60 pruebas superadas (12,9 s)
Lint frontend	npm run lint	Sin hallazgos

Fuente: Ejecución real de la auditoría (base de datos separada `proyectwin_mvp_manual`).

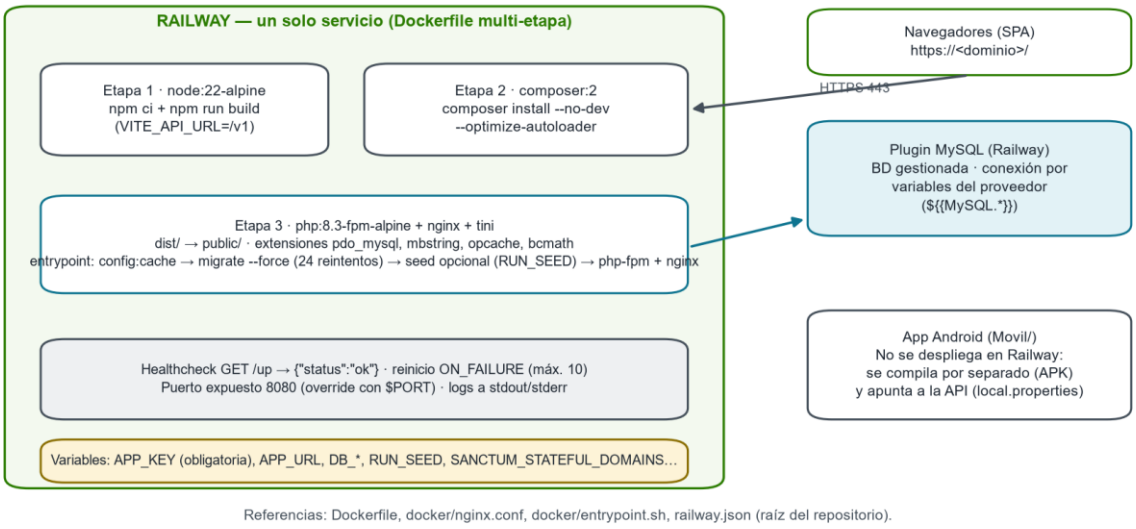
5.12. Compilación y construcción de producción

SPA: desde `frontend/`, `npm run build` genera `dist/` (estáticos optimizados) usando `VITE_API_URL=/v1` (ruta relativa) para que la API se consuma del mismo origen. El resultado se sirve con `npm run preview` (puerto 4173) para validación local.

Backend: en el contenedor se ejecuta `composer install --no-dev --no-scripts --no-interaction --prefer-dist --optimize-autoloader`, y al arrancar: `php artisan package:discover`, `php artisan config:cache`, migraciones con reintentos y, opcionalmente, `db:seed` (si `RUN_SEED=true` y la base está vacía). Estos comandos provienen de `Dockerfile` y `docker/entrypoint.sh`; **no se ejecutó el empaquetado en esta auditoría** (Docker no está disponible en el equipo), por lo que se clasifican como configuración verificada por lectura, no por ejecución.

5.13. Despliegue

Figura 6. Topología de despliegue prevista (servicio único en Railway + MySQL gestionado)



Fuente: Elaboración propia a partir de `Dockerfile`, `docker/` y `railway.json`. No ejecutado en la auditoría.

La Figura 6 resume la topología prevista: un único servicio (nginx + PHP-FPM + SPA compilada) con healthcheck `GET /up`, más una base MySQL gestionada. La guía interna `docs/deploy-railway.md` describe el flujo completo (no re-ejecutado aquí). Resumen de pasos y variables (se publican solo los nombres):

15. Crear el proyecto en Railway e instalar/enlazar la CLI (railway login, railway init).
16. Agregar el plugin MySQL (railway add --database mysql).
17. Definir variables: APP_KEY (obligatoria; el contenedor falla con error explícito si falta), APP_URL, APP_ENV=production, APP_DEBUG=false, credenciales DB_* del plugin, FRONTEND_URL, SANCTUM_STATEFUL_DOMAINS con el dominio real, SESSION_DRIVER, API_RATE_LIMIT, MAIL_* y RUN_SEED=true solo en el primer despliegue.
18. Desplegar (railway up). El contenedor espera la base hasta 24 reintentos, migra y sirve.
19. Verificar: GET /up → {"status":"ok"}; la raíz carga la SPA; las rutas internas no devuelven 404 (fallback SPA).
20. Después del primer despliegue, cambiar RUN_SEED=false para no resembrar en cada arranque.

Limitaciones declaradas de este apartado: el despliegue no fue ejecutado por la auditoría; la guía interna advierte además que con `SESSION_DRIVER=file` cada redespliegue cierra las

sesiones (mitigación en curso: tabla `sessions`, § 4.5.18) y que el correo real requiere configurar SMTP externo.

5.14. Solución de errores frecuentes

Tabla 46. Errores frecuentes, causa y solución

Síntoma	Causa probable	Solución
php no se reconoce como comando	PHP fuera del PATH	Agregar la carpeta de PHP (p. ej. C:\xampp\php) al PATH o usar su ruta completa.
composer install falla por extensiones	Faltan extensiones requeridas	Habilitar pdo_mysql, mbstring, openssl, dom, fileinfo, curl, bcmath, zip en php.ini y reintentar.
Error de conexión a MySQL al migrar	Servicio detenido o credenciales/DB_DATABASE incorrectos	Iniciar MySQL/MariaDB; revisar DB_* en backend/.env; crear la base (§ 5.7).
419 (CSRF) al usar la SPA	Cookie XSRF-TOKEN vencida tras reinicio del backend	Recargar la página; el cliente renueva la cookie automáticamente en la siguiente escritura (api.js).
401 constante desde el navegador	Sesión expirada o SANCTUM_STATEFUL_DOMAINS sin el origen del SPA	Reiniciar sesión; revisar SANCTUM_STATEFUL_DOMAINS y FRONTEND_URL.
CORS bloqueado con API en otro origen	Vite proxy sin usar o API y SPA en orígenes distintos sin configurar	Usar el proxy de Vite (§ 5.10) o configurar config/cors.php con el origen correcto.
403 «Debe cambiar la contraseña»	Cuenta con must_change_password = 1 (primer ingreso)	Completar el cambio de contraseña obligatorio o restablecer credenciales desde administración.
403 «Cuenta suspendida»	Usuario con estado = 0	Reactivar la cuenta desde el panel de administración.
422 al enviar una propuesta	Campos mínimos incompletos o envío sin cambios (huella idéntica)	Completar título/descripción/área/objetivos; si no hubo cambios reales, editar antes de reenviar.
La lista de propuestas no muestra datos	Aprendiz sin ficha activa	Unirse a una ficha con código válido; las propuestas existen pero la edición requiere ficha activa.
Correo de credenciales no llega	MAIL_MAILER=log (desarrollo) o SMTP mal configurado	Revisar storage/logs/laravel.log; configurar SMTP; reenviar desde administración (el error queda en credenciales_error).
npm run dev no conecta con la API	Backend detenido o puerto 8000 ocupado	Verificar php artisan serve en 127.0.0.1:8000; el proxy apunta a ese host/puerto.
Playwright no encuentra navegadores	Binarios no instalados	Ejecutar npx playwright install dentro de frontend/.
PDF de credenciales falla en el servidor	Faltan extensiones dom/mbstring en PHP	Habilitarlas en el entorno de despliegue (el Dockerfile ya las instala).

Fuente: Casos documentados por el proyecto y observados/prevenidos durante la auditoría.

CAPÍTULO 6. REGISTRO TÉCNICO DEL DESARROLLO Y PROCEDIMIENTOS

6.1. Organización del código fuente

El repositorio es un **monorepo** con tres aplicaciones y artefactos de despliegue. Cada aplicación mantiene su propio manifiesto de dependencias y su propio conjunto de pruebas:

- backend/: aplicación Laravel estándar (MVC + servicios), API bajo /v1.
- frontend/: aplicación Vite/React con páginas por rol, componentes compartidos y pruebas unitarias y E2E.
- Movil/: proyecto Gradle de una sola aplicación Android con arquitectura MVVM por capas (data, domain, ui).
- docker/ y raíz: empaquetado de producción y configuración de despliegue.
- docs/: documentación interna del proyecto (útil pero parcialmente histórica; ver § 1.9).

6.2. Árbol de directorios relevantes

```
backend/
├── app/
│   ├── Console/Commands/        # 3 comandos artisan (perfiles:docentes, similitud:*)
│   ├── Http/Controllers/Api/    # 15 controladores de la API
│   ├── Http/Middleware/         # CuentaActiva, RolMiddleware, CambioContrasenaObligatorio
+ base
│   ├── Models/                  # 16 modelos Eloquent (scopes de alcance)
│   ├── Notifications/           # CredencialesUsuario, ResetPasswordNotification
│   ├── Services/                # AltaUsuario, CredencialesCorreo, NotificacionesService,
SimilitudService
│   ├── Similarity/              # Lsa, Pasajes, Recomputador (motor)
│   └── Support/                 # Auditoria, BorradoCascada, Credenciales,
CredencialesPdf, HistorialProyecto, Included
├── config/                      # configuración Laravel (database, sanctum, cors, auth…)
├── database/
│   ├── migrations/              # 18 migraciones (esquema consolidado)
│   └── seeders/                 # DatabaseSeeder (escenario demo) y VolumenSeeder
├── resources/views/pdf/         # plantilla Blade del PDF de credenciales
├── routes/                      # api-v1.php (84 rutas), web.php (/ y /up), console.php
└── tests/                       # 33 clases: 31 Feature + 2 Unit (244 métodos)

frontend/src/
├── pages/{public,aprendiz,instructor,admin}/   # 47 páginas (carpetas index.jsx +
implementación)
├── components/                  # 54 componentes .jsx (incluye guards y *Base compartidos)
├── layouts/                     # DashboardLayout, AuthLayout, LandingLayout
├── contexts/                    # AuthContext, ThemeContext, MotorContext
├── constants/                   # rutas por rol, paginación, estados, contacto
├── lib/                         # api.js (cliente HTTP), recursos.js (contratos),
useApi.js
├── tokens/index.css             # sistema de diseño (tema claro/oscuro)
└── utils/, hooks/               # helpers, validaciones, focus-trap
frontend/e2e/                    # 54 especificaciones E2E + runners + agentes de
navegación
```

```
Movil/app/src/main/java/com/example/proyectwin/
├── data/{api,model,mapper,local,repository}/    # red (Retrofit), modelos, sesión cifrada,
repositorios
├── domain/{repository,usecase}/                 # contratos y casos de uso
├── di/                          # módulos Hilt (red, repositorios)
├── navigation/                  # AppNavigation (4 grafos por rol)
└── ui/{screens,viewmodel,components,theme}/     # ~40 pantallas Compose y 22 ViewModels
```

Árbol de código — Archivo: Árbol real del repositorio (resumido; el íntegro está en el Anexo A).

6.3. Archivos principales y sus responsabilidades

Tabla 47. Archivos clave del sistema

Archivo	Responsabilidad
backend/routes/api-v1.php	Definición de las 84 rutas de la API con sus grupos y middlewares.
backend/app/Providers/RouteServiceProvider.php	Registro del prefijo /v1 y del rate limiter api.
backend/app/Http/Kernel.php	Grupos de middleware (web/api) y alias (cuenta.activa, rol).
backend/app/Services/SimilitudService.php	Motor de puntuación (4 señales ponderadas, normalización, pasajes).
backend/app/Similarity/Recomputador.php	Orquestación: detectar, archivar, recalcular y vigencia de pares.
backend/app/Models/Project.php	Reglas de escritura (puedeEscribir) y scopes de visibilidad.
backend/app/Support/Included.php	Inclusión controlada de relaciones por lista blanca.
backend/app/Support/Auditoria.php	Escritura silenciosa de la bitácora.
frontend/src/App.jsx	Enrutado completo con carga diferida y guards por rol.
frontend/src/lib/api.js	Cliente HTTP: cookie, CSRF, caché 15 s, 401 global.
frontend/src/lib/recursos.js	Contratos tipados por recurso de la API.
frontend/src/layouts/DashboardLayout/DashboardLayout.jsx	Shell de la aplicación privada (sidebar, topnav, campana).
Movil/.../data/api/AuthInterceptor.kt	Inyección del token Bearer en cada petición.
Movil/.../data/local/SessionManager.kt + TokenCipher.kt	Persistencia de sesión con token cifrado (Keystore).
Dockerfile, docker/entrypoint.sh, docker/nginx.conf	Empaquetado y arranque de producción.

Fuente: Auditoría del repositorio.

6.4. Punto de entrada de la aplicación

- **API (web y móvil):** backend/public/index.php → bootstrap/app.php → kernel HTTP → RouteServiceProvider (prefijo /v1). En producción nginx enruta /v1 y /sanctum a este archivo.
- **SPA:** frontend/index.html (script anti-parpadeo de tema + metadatos) → frontend/src/main.jsx (proveedores: tema, motor, router, error boundary, sesión) → App.jsx (rutas).
- **Android:** MainActivity.kt (NavHost y eventos de sesión) con la clase de aplicación ProyecTwinApp (Hilt y migración de token heredado).

6.5. Rutas y navegación

Backend (84 rutas): 10 públicas (login, recuperación, catálogos, configuración del motor y demo), 4 de sesión (`logout`, `me`, correo, contraseña) y 70 autenticadas; 22 de ellas exigen rol administrativo o administrativo/instructor. El catálogo íntegro con método, ruta, acción y middleware está en el Anexo B.

Frontend (55 declaraciones de ruta): 5 públicas (incluida la 404), 7 redirecciones heredadas y 43 rutas protegidas por rol. Cada página carga de forma diferida (`lazy`) bajo un guard doble: `ProtectedRoute` (sesión + rol + cambio de contraseña obligatorio) y `SafeRoute` (barrera de error con salidas de recuperación). Resumen por rol:

Tabla 48. Rutas del frontend por área

Área	Rutas protegidas	Ejemplos representativos
Públicas	—	/ (portal), /login, /recuperar-contrasena, /restablecer-contrasena, 404
Aprendiz	14	/aprendiz/dashboard, /aprendiz/propuestas, /aprendiz/ficha, /aprendiz/similitudes, detalles de proyecto/similitud/ficha
Instructor	11	/instructor/revision-propuestas, /instructor/fichas, /instructor/similitudes, perfiles
Administrador	16	/admin/usuarios, /admin/fichas, /admin/proyectos, /admin/similitudes, /admin/bitacora, /admin/config-similitud, /admin/reportes-fallas
Compartidas	2	/aprendiz/perfil-companero/:id y /instructor/perfil-companero/:id (componente único)

Fuente: `frontend/src/App.jsx` (auditoría).

6.6. Componentes de interfaz

La interfaz usa un sistema de diseño propio en dos temas (claro/oscuro) definido en `frontend/src/tokens/index.css`, con tipografías Inter/Sora y JetBrains Mono, y una regla explícita: el estado nunca depende solo del color (`docs/normativa-visual.md`). Los componentes más reutilizados:

Tabla 49. Componentes de interfaz destacados

Componente	Función
DataTable	Tabla densa con encabezado fijo y modo tarjeta en pantallas pequeñas.
Sidebar / TopNav / GovernmentBar	Navegación por rol, campana de notificaciones, identidad institucional y conmutador de tema.
AlertasBase / PerfilBase / ReportarFallaBase / DetalleSimilitudBase	Implementaciones únicas compartidas por las tres fachadas de rol (patrón *Base).
ScoreDial + GradeBadge + grade.js	Visualización del porcentaje de similitud y su grado S/A/B/C según el umbral vigente del motor (contexto MotorContext).
ObservacionHilo	Render de hilos de observaciones con autor y respuestas anidadas.
HistorialProyecto	Línea de tiempo del project_histories.
ProtectedRoute / SafeRoute / ErrorBoundary	Guardas de acceso y tolerancia a fallos de render.
FormField, Input, Button, Badge, Modal, Lightbox	Bloques base de formularios e interacción, con accesibilidad (aria, foco visible, focus-trap).

Fuente: Auditoría del frontend.

6.7. Lógica de negocio

La lógica de negocio está distribuida en tres lugares claramente identificables:

21. **Modelos** (reglas de dominio reutilizables): Project::puedeEscribir (regla «Classroom») y los scopes de alcance (paraUsuario, paraDetalle, visibleDetalle, contraparteAprobada, vigentes, historicos).
22. **Controladores** (transiciones y validaciones): envío con huella y motor, revisión con observación, gestión de fichas con salvaguardas, credenciales con PDF.

23. **Servicios y soporte** (operaciones transversales): motor (Recomputador, SimilitudService), notificaciones (NotificacionesService), alta institucional (AltaUsuario), auditoría (Auditoria), historial (HistorialProyecto) y borrado en cascada controlado (BorradoCascada).

Fragmento representativo de la regla de participación (**«Classroom»**), que gobierna cuándo un usuario puede editar, comentar o gestionar equipo:

```
// backend/app/Models/Project.php (líneas 201-224)
public function puedeEscribir($user): bool
{
    if (!$user) return false;
    if ($user->rol === 'admin') return true;

    if ($user->rol === 'instructor') {
        $instructor = Instructor::where('id_usuario', $user->id)->first();
        if (!$instructor) return false;
        return (int) $this->id_instructor_asignado === (int) $instructor->id
            || (int) optional($this->classGroup)->id_instructor === (int) $instructor->id;
    }

    $ficha = $this->classGroup;
    if (!$ficha || $ficha->estado !== 'activo') return false;

    $aprendiz = Apprentice::where('id_usuario', $user->id)->first();
    if (!$aprendiz || (int) $aprendiz->id_class_group !== (int) $ficha->id) return false;

    if ((int) $this->id_creador === (int) $user->id) return true;

    return ApprenticeProject::where('id_proyecto', $this->id)
        ->where('id_aprendiz', $aprendiz->id)
        ->exists();
}
```

Fragmento A — Archivo: backend/app/Models/Project.php

6.8. Controladores, servicios, módulos y clases

Tabla 50. Controladores de la API y su responsabilidad (una línea)

Controlador	Responsabilidad
AuthController	Login (usuario o correo), logout, sesión actual, cambio de correo/contraseña y recuperación.
GeneralUserController	Cuentas: CRUD administrativo, perfil público con alcance, credenciales (PDF, reenvío, restablecimiento).
InstructorController / ApprenticeController	Perfiles de instructor y aprendiz con alcance por ficha; unirse/salir de ficha.
ClassGroupController	Fichas: crear con código servidor, roster, estados, credenciales de la ficha y borrado con salvaguardas.
ProjectController	Propuestas: CRUD por rol, envío con validación/huella/motor, revisión, historial.
SimilarityController	Listado y detalle con alcance, detección manual, recálculo global y demo pública.
MotorConfigController	Lectura pública y actualización administrativa del umbral/ventana; resumen público.
KnowledgeNetworkController / TrainingProgramController	Catálogos institucionales (lectura pública; escritura admin).
NotificationController	Avisos internos con reglas anti-suplantación y marcado de lectura.
BugReportController	Soporte: reporte del usuario, bandeja administrativa y cierre con respuesta.
ApprenticeProjectController	Equipo de la propuesta (pivote) con invariantes de ficha y estado.
CommentController	Observaciones y respuestas con regla de participación.

Controlador	Responsabilidad
AuditLogController	Consulta de la bitácora (solo administrador; sin escritura por API).

Fuente: `backend/app/Http/Controllers/Api`.

6.9. Acceso y transformación de datos

- **Eloquent en todos los accesos:** no hay SQL crudo de negocio; los scopes de los modelos encapsulan las reglas de visibilidad y los controladores componen consultas con filtros y ?included=.
- **Inclusión por lista blanca:** cada modelo declara $allowIncluded; Support/Included.php valida la ruta de relación solicitada y aplica with(). Las relaciones naviegan en camelCase en el JSON ($snakeAttributes = false).
- **Ocultamiento por defecto:** los campos sensibles de GeneralUser están en $hidden; solo los endpoints administrativos los revelan explícitamente (makeVisible) cuando la operación lo exige.
- **Casts:** objetivos_especificos, detalles y detalle se castean a arreglo; vigente a booleano; fechas normalizadas por mutadores (setFechaAttribute, setFechaIngresoAttribute).
- **Serialización de dominio:** Similarity ordena el par antes de guardar; Project expone puedeEscribir implícitamente vía permisos calculados en la respuesta de detalle.

6.10. Procesos principales

1) Envío de propuesta y detección de similitud (flujo completo verificado en código y pruebas). El fragmento siguiente muestra el mecanismo anti-duplicados que exige cambios reales entre envíos:

```
// backend/app/Http/Controllers/Api/ProjectController.php (líneas 504-514)
// Huella del contenido que define la propuesta (bloquea reenvíos sin cambios).
private function huellaContenido(Project $project): string
{
    return sha1(json_encode([
        $project->titulo,
        $project->resumen,
        $project->palabras_clave,
        $project->area_aplicacion,
        $project->objetivo_general,
        $project->objetivos_especificos,
    ]));
}
```

Fragmento B — Archivo: backend/app/Http/Controllers/Api/ProjectController.php

2) Motor de similitud. El puntaje combina cuatro señales ponderadas; si el corpus es demasiado pequeño para el análisis latente, el peso del tema se reparte entre las demás (comentario del código):

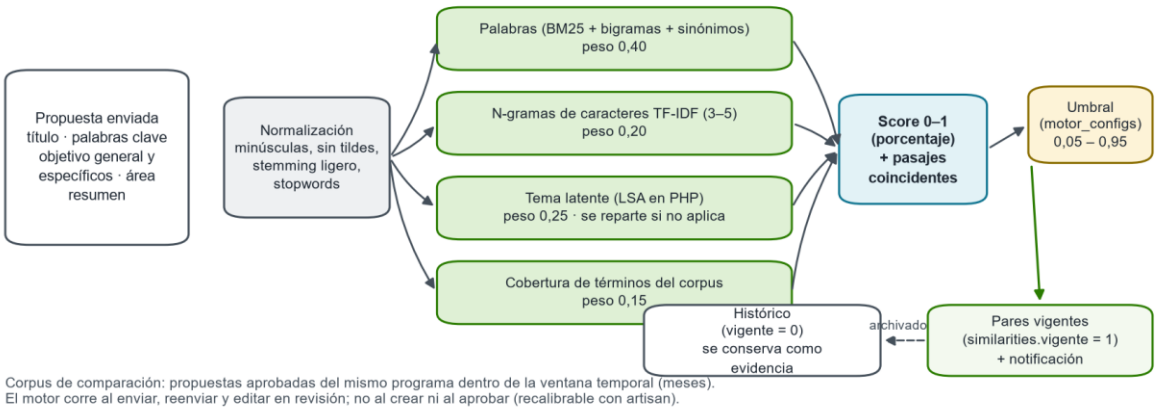
```
// backend/app/Services/SimilitudService.php (líneas 23-44)
// Pesos de las señales (suman 1). Si no hay LSA (corpus pequeño), el peso del
// tema se reparte entre las demás.
private const PESOS = [
    'palabras' => 0.40,
    'caracteres' => 0.20,
    'tema' => 0.25,
    'cobertura' => 0.15,
];

// Parámetros de BM25.
private const BM25_K1 = 1.4;
private const BM25_B = 0.75;

// Campos de la propuesta y su peso (los tokens se repiten según el peso).
private const PESO_CAMPOS = [
    ['titulo', 3],
    ['palabras_clave', 3],
    ['objetivo_general', 2],
    ['objetivos_especificos', 2],
    ['area_aplicacion', 2],
    ['resumen', 1],
];
```

Fragmento C — Archivo: backend/app/Services/SimilitudService.php

Figura 7. Flujo del motor de similitud (señales, umbral y resultados vigentes/históricos)



Fuente: Elaboración propia a partir de `SimilitudService`, `Recomputador` y `motor_configs`.

La Figura 7 sintetiza el proceso interno del motor documentado en § 6.7–6.8.

3) Autenticación y sesión. La SPA obtiene la cookie CSRF antes de la primera escritura y mantiene la sesión en cookie httpOnly; el cliente renueva la cookie si el servidor se reinicia (evitando 419 persistentes):

```
// frontend/src/lib/api.js (líneas 31-48)
let csrfListo = false;

// Pide (o renueva) la cookie CSRF. El Set-Cookie de esta respuesta reemplaza la
```

```
// XSRF-TOKEN anterior, así que también recupera el caso en que el servidor se
// reinicia (p. ej. un despliegue): la sesión muere pero el navegador conserva
// el token viejo y toda escritura respondería 419 hasta limpiar cookies.
async function refrescarCsrf() {
  csrfListo = false;
  try {
    await fetch(URL_CSRF, {
      credentials: "include",
      headers: { Accept: "application/json" },
    });
    csrfListo = true;
  } catch {
    /* se reintentará en la próxima escritura */
  }
}
```

Fragmento D — Archivo: frontend/src/lib/api.js

4) Calibración del corpus (administración). `POST /v1/similarities/recalculate` purga pares obsoletos y recrea los que el umbral vigente detecte; el comando `php artisan similitud:recalcular` hace lo mismo desde consola y `similitud:calibrar` sugiere umbrales por programa con tabla de señales.

6.11. Operaciones CRUD implementadas

Tabla 51. Cobertura CRUD por recurso (según rutas y controladores)

Recurso	Crear	Leer	Actualizar	Eliminar
Usuarios (general-users)	Admin	Admin / propio / perfil público	Propio o admin (campos con lista blanca)	Admin (con guardas)
Redes y programas	Admin	Público	Admin	Admin (con cascada controlada)
Instructores / aprendices	Admin / instructor (aprendiz)	Según alcance	Según rol y alcance	Admin
Fichas (class-groups)	Admin / instructor	Según rol (la ficha propia)	Gestor de la ficha	Anular o eliminar según impacto
Propuestas (projects)	Aprendiz con ficha activa / admin	Según alcance	Según estado y rol (con historial)	Creador o admin
Equipo (apprentice-projects)	Con permiso de escritura	Según visibilidad	(No aplica: se agrega/quita)	Quitar (no al creador)
Similitudes	Motor / admin	Según alcance	Admin	Admin
Notificaciones	Sistema / permitido por alcance	Propias (admin: todas)	Marcar leída (dueño)	Admin
Observaciones (comments)	Con permiso de escritura	Según alcance	Autor o admin	Autor o admin
Reportes (bug-reports)	Cualquier autenticado (el suyo)	Propios / admin	Admin (estado y respuesta)	Admin

Fuente: `boot()`, `routes/api-v1.php` y controladores (auditoría).

6.12. Endpoints de la API

La API expone **84 endpoints** bajo `/v1`. La tabla resume la distribución por recurso; el catálogo detallado (método, ruta, acción y middleware) está en el Anexo B.

Tabla 52. Distribución de endpoints por recurso

Recurso (prefijo)	Nº	Público	Autenticado	Restricción de rol
/auth (login, recuperación, sesión)	7	3	4	—
/general-users (cuentas y credenciales)	11	—	11	Escritura total y credenciales: admin
/knowledge-networks y /training-programs	10	4 (lectura)	6	Escritura: admin
/instructors y /apprentices	12	—	12	Alta/edición según rol; me/ficha solo aprendiz
/class-groups (fichas)	7	—	7	Creación por admin/instructor; credenciales admin
/projects (propuestas)	7	—	7	Todos según alcance
/similarities	7	1 (demo)	6	Recálculo y CRUD manual: admin
/notifications	5	—	5	Según alcance del aviso
/bug-reports (soporte)	5	—	5	Responder: admin
/apprentice-projects (equipo)	4	—	4	Permiso de escritura sobre la propuesta
/comments (observaciones)	5	—	5	Permiso de escritura sobre la propuesta
/config-similitud y /public/*	4	3	1	Actualización: admin

Fuente: `backend/routes/api-v1.php` (auditoría). Los totales por fila incluyen duplicados de verbos por recurso.

6.13. Autenticación y manejo de sesiones

El sistema ofrece **dos mecanismos sobre la misma API**: (a) cookie de sesión httpOnly con CSRF para la SPA servida en el mismo origen, y (b) token Bearer Sanctum para la app Android y las pruebas. En ambos casos, la cadena de middleware aplica `auth:sanctum` → `cuenta.activa` → `CambioContrasenaObligatorio` → `rol:*` (cuando la ruta lo exige). La sesión de la SPA se apoya en el grupo `EnsureFrontendRequestsAreStateful` de Sanctum; el móvil guarda su token cifrado en el dispositivo (AES-256-GCM con Android Keystore) y lo adjunta en cada petición mediante `AuthInterceptor`. El detalle de seguridad está en el Capítulo 7.

6.14. Gestión de errores

Tabla 53. Errores del backend y tratamiento en la interfaz

Código	Significado en este sistema	Ejemplo	Tratamiento en el frontend
401	Sin sesión o token inválido	Cookie expirada	Evento auth:expirada → limpieza de sesión y redirección a login.
403	Sin permiso, cuenta suspendida o cambio de contraseña pendiente	Instructor sobre ficha ajena	Mensaje de solo lectura o redirección a /cambio-obligatorio.
404	Recurso no encontrado	Código de ficha inexistente	Estado vacío con mensaje del backend.
409	Conflicto de invariantes	Quitar al creador del equipo; último admin	Alerta con el mensaje del servidor.
422	Validación de negocio o de campos	Reenvío sin cambios; campos mínimos	Errores por campo (toFieldErrors) o alerta general.
419	CSRF expirado (SPA)	Reinicio del backend	Renovación automática de la cookie y reintento en la próxima escritura.
502	Fallo de servicio externo	Envío de correo de credenciales	Aviso con opción de reenvío; el error queda en la cuenta.

Fuente: Código de controladores, `api.js` y pruebas del proyecto.

En la interfaz, cualquier fallo de render dentro de una página protegida lo captura `SafeRoute` (`ErrorBoundary`) con opciones de volver al panel o reportar una falla; las páginas no se quedan en pantalla blanca. La auditoría interna encontró y documentó casos históricos de errores silenciados en acciones destructivas (corregidos en el cierre de auditoría; ver § 7.12).

6.15. Comandos frecuentes

Tabla 54. Comandos de operación y desarrollo

Contexto	Comando	Para qué sirve
Backend	php artisan serve	Servidor de desarrollo en el puerto 8000.
Backend	php artisan migrate:fresh --seed	Reconstruir esquema y datos demo (destructivo).
Backend	php artisan test	Suite completa de pruebas (MySQL del .env).
Backend	php artisan similitud:recalcular	Recalibrar el corpus con el umbral vigente.
Backend	php artisan similitud:calibrar	Sugerir umbral por programa (con etiquetas opcionales).
Backend	php artisan perfiles:docentes	Crear perfiles de instructor faltantes.
Backend	php artisan db:seed --class=VolumenSeeder	Cargar datos de volumen para probar listados.
Frontend	npm run dev	Servidor de desarrollo con proxy a la API.
Frontend	npm run build / npm run preview	Build de producción y servidor de vista previa (4173).
Frontend	npm run test:unit	Pruebas unitarias (Vitest).
Frontend	npm run lint	Análisis estático (ESLint).
Frontend	npm test / npm run test:e2e	E2E de Playwright (reseedea la base antes de cada archivo).
Frontend	npm run test:full	Gate completo E2E (aislado + agentes + cobertura + negativos + volumen).
Móvil	gradlew assembleDebug (desde Movil/)	Compilar APK de depuración (no ejecutado en la auditoría).

Fuente: README de backend y frontend; verificado lo ejecutado en la auditoría.

6.16. Convenciones de nombres

Tabla 55. Convenciones observadas en el código

Ámbito	Convención	Ejemplo
Tablas y columnas (BD)	snake_case en español	class_groups.numero, id_instructor_asignado
Modelos Eloquent	PascalCase singular	ClassGroup, ProjectHistory
Controladores	PascalCase + sufijo Controller	SimilarityController
Rutas de la API	kebab-case en plural	/v1/class-groups/{id}/aprendices
Scopes de modelos	camelCase con prefijo scope	scopeParaUsuario, scopeVigentes
Páginas del frontend	Carpeta PascalCase con index.jsx de reexportación	pages/aprendiz/Propuestas/Propuestas.jsx
Componentes React	PascalCase por carpeta	components/DataTable/DataTable.jsx
Hooks y utilidades	use* / camelCase	useApi, formatearFecha
Pivotes y pivote N:M	snake_case singular compuesto	apprentice_projects
Commits (Git)	prefijos tipo Convencional (feat, fix, refactor, docs, style)	fix: completar cierre definitivo del MVP web

Fuente: Revisión de código y `git log`.

6.17. Control de versiones

- **Herramienta:** Git sobre GitHub (origin: https://github.com/YDanB-312/ProyecTwin-MVP.git), rama principal master (única).
- **Historial:** 46 commits; la actividad se concentra entre septiembre y octubre de 2026. Últimos hitos registrados: consolidación de migraciones (eeff159), normalización de

controladores (54e184f), móvil F1–F6 (09abd95), documentación de auditoría (b6dab04) y normalización responsiva (3b15f3f, 13/10/2026).

- **Autores:** dos identidades de commit («Oscar» y «YDanB-312»). No hay ramas de características ni etiquetas (tags) de versión.
- **Estado del árbol al cierre de la auditoría:** existían cambios sin confirmar (archivos de despliegue nuevos —Dockerfile, docker/, railway.json—, ajustes en TrustProxies, routes/web.php y frontend/src/lib/api.js, y la migración sessions). Se documentan como trabajo en curso; **el manual describe el árbol de trabajo completo**, señalando donde algo aún no está confirmado en Git.
- **Recomendación:** antes de la entrega, confirmar (commit) el trabajo en curso, etiquetar la versión entregada y mantener la rama master desplegable en todo momento.

6.18. Procedimiento recomendado para modificar y mantener el código

Flujo general sugerido (alineado con las herramientas que el proyecto ya usa):

24. Crear una rama descriptiva (feat/…, fix/…) desde master.
25. Implementar el cambio respetando las convenciones (§ 6.16) y las reglas de dominio existentes.
26. Backend: ejecutar php artisan test (base de desarrollo); Frontend: npm run test:unit, npm run lint y, cuando aplique, los E2E (npm test).
27. Si el cambio altera el esquema: nueva migración (nunca editar migraciones ya aplicadas en producción) y verificación con migrate:fresh --seed en local.
28. Si toca el motor: actualizar pruebas de SimilitudServiceTest y recalibrar (similitud:recalibrar) en un entorno con datos representativos.
29. Ejecutar npm run build antes de desplegar; confirmar que el contenedor arranca y /up responde.
30. Confirmar el cambio con mensaje convencional y actualizar la documentación afectada (incluido este manual cuando corresponda).

Recetas puntuales verificadas contra la estructura real:

- **Agregar un endpoint:** crear ruta en backend/routes/api-v1.php dentro del grupo adecuado (autenticado y rol), implementar el método en el controlador con validación y scopes de alcance, y añadir la prueba Feature correspondiente.

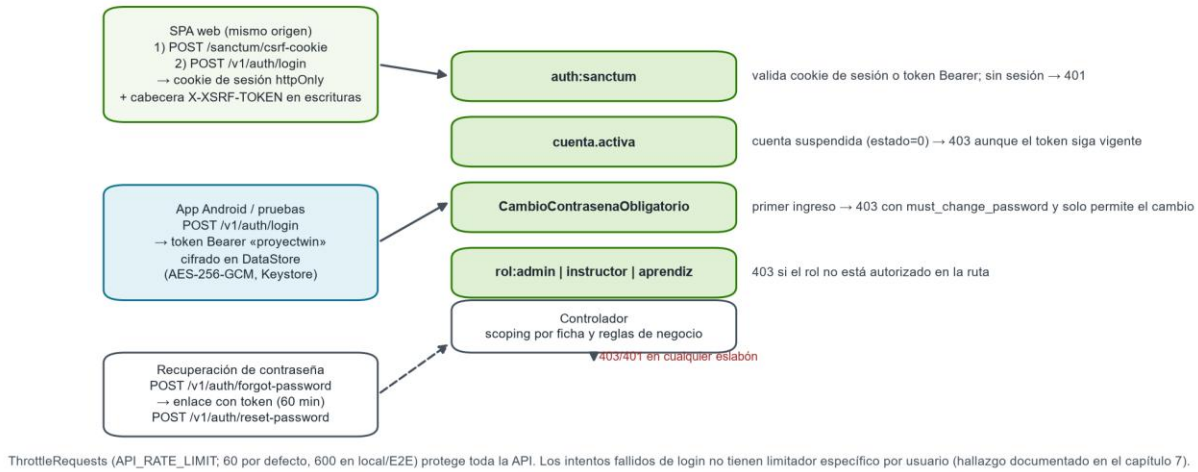
- **Agregar una página web:** crear la carpeta en frontend/src/pages/<rol>/… con Nombre.jsx e index.jsx, declarar la ruta con ProtectedRoute allowedRoles=[…] + SafeRoute en App.jsx, y añadir el enlace al Sidebar y Footer según el rol.
- **Cambiar el umbral del motor:** hacerlo desde /admin/config-similitud (queda auditado) y ejecutar el recálculo; no editar filas de motor_configs directamente en producción.
- **Cambiar reglas de visibilidad:** modificar el scope del modelo afectado (p. ej. scopeParaUsuario) y actualizar las pruebas de alcance (VisibilityTest, SimilarityScopeTest) para conservar la cobertura.

CAPÍTULO 7. SEGURIDAD Y CALIDAD

Este capítulo describe la seguridad implementada, las pruebas existentes y las ejecutadas durante la auditoría, los riesgos identificados y el plan de mejora. **Advertencia metodológica:** una revisión estática y funcional como esta **no equivale a una auditoría formal de certificación** ni a una prueba de penetración; las conclusiones se limitan a lo observado y ejecutado.

7.1. Arquitectura de seguridad

Figura 8. Cadena de autenticación y autorización (SPA y móvil) con los eslabones de defensa



Fuente: Elaboración propia a partir de `routes/api-v1.php`, `Kernel.php` y los middleware propios.

La seguridad se organiza en capas defensivas (Figura 8): (1) transporte — cookie httpOnly y mismo origen en web, HTTPS extremo a extremo en producción; (2) autenticación — Sanctum con dos modos; (3) estado de cuenta — el middleware `cuenta.activa` invalida tokens de cuentas suspendidas; (4) contraseña temporal — `CambioContrasenaObligatorio` bloquea toda la API hasta el primer cambio; (5) autorización — middleware `rol` más *scoping* por ficha en modelos y controladores; (6) superficie de datos — campos sensibles ocultos por defecto e inclusión por lista blanca; (7) tasa — `ThrottleRequests` global de la API; y (8) trazabilidad — bitácora inmutable de acciones sensibles.

7.2. Autenticación

- **Acceso por `username`** (no por correo), con recordarme opcional; el correo se reserva para la recuperación de contraseña (getEmailForPasswordReset).

- **Contraseñas con hash bcrypt** (Laravel); la contraseña temporal se cifra con Crypt (clave APP_KEY) y **solo se descifra al exportar el PDF de credenciales**, mientras must_change_password esté activo.
- **SPA:** cookie de sesión httpOnly + CSRF (/sanctum/csrf-cookie + X-XSRF-TOKEN); el JavaScript nunca almacena tokens.
- **Móvil:** token Sanctum emitido en el login, persistido cifrado con AES-256-GCM (clave en Android Keystore) y reintroducido por interceptor en cada petición.
- **Recuperación de contraseña:** broker nativo con token de 60 minutos y *throttling* de 60 segundos; la respuesta es genérica exista o no el correo. En entorno local el enlace puede devolverse a la interfaz (facilitador de desarrollo); en producción debe viajar solo por correo.
- **Cambio de correo o contraseña** exige la contraseña actual; al cambiar el correo se revocan los demás tokens del usuario; la contraseña nueva no puede repetir la anterior.

7.3. Autorización y roles

El modelo de autorización combina **roles** (aprendiz, instructor, admin) con **alcance por ficha** y **reglas de estado**. Los puntos de control verificados son:

- Middleware rol en 22 rutas (administración de usuarios, catálogos, motor, bitácora, credenciales, recálculo).
- Scopes de visibilidad en modelos: Project::paraUsuario (listado), paraDetalle (detalle con lectura ampliada del instructor en su programa), Similarity::paraUsuario/visibleDetalle (pares propios o autorizados) y Comment::paraUsuario.
- Regla «Classroom» (Project::puedeEscribir, § 6.7): participación solo dentro de la ficha activa; fuera de ella, solo lectura.
- Guardas de invariantes en controladores: no auto-degradarse, no dejar el sistema sin administradores activos (409), instructor con fichas no cambia de rol (409), no quitar al creador de su equipo (409), traslado de aprendices entre fichas bloqueado para instructores (422; solo admin).
- Privacidad entre pares: un aprendiz no ve propuestas, comentarios ni pasajes de compañeros; las contrapartes de similitud de otros alcances se abren en modo lectura sin datos personales.

Estas reglas cuentan con pruebas específicas (`PermisosTest`, `AuthorizationTest`, `VisibilityTest`, `SimilarityScopeTest`, `SimilarityPerspectiveTest`, `AccesoContraparteTest`, `InteraccionPropuestaTest`, `CierreAuditoriaTest`), todas superadas en la ejecución de esta auditoría (§ 7.10).

7.4. Validación de entradas

- Validación declarativa ($request->validate) en cada controlador: tipos, longitudes, rangos (umbral 0,05–0,95; meses 1–60; campos mínimos de propuesta), unicidad (correo, documento, número de ficha) y pertenencia (códigos de ficha, ids relacionados).
- Asignación masiva controlada: los modelos y controladores usan listas blancas; los campos internos (huella_envio, vigente, estado en reportes, notificaciones) no son aceptados desde el cliente (probado en NormalizacionSeguridadTest y CierreAuditoriaTest).
- Inclusión de relaciones con lista blanca por modelo (?included=), ignorando rutas no autorizadas de forma silenciosa y segura.
- Salida saneada por $hidden: documentos, username, must_change_password y metadatos de credenciales no se serializan salvo en endpoints administrativos que los revelan explícitamente.
- Bloqueo de reenvíos vacíos mediante huella de contenido (sha1) y transacciones con lockForUpdate en envío y en el motor.

7.5. Gestión de errores

El backend responde con códigos homogéneos (401/403/404/409/422/419/502, § 6.14) y mensajes en español orientados al usuario. En seguridad, el diseño «falla cerrado»: sin sesión → 401; sin permiso → 403; cuenta suspendida → 403 aunque el token siga vigente. La interfaz evita pantallas blancas con `SafeRoute`/`ErrorBoundary` y expone recuperación (volver al panel o reportar la falla). El registro de auditoría y el de historial están diseñados para **no romper la acción del usuario** ante fallos de escritura secundaria (fail-open controlado que la auditoría interna marcó como riesgo menor de observabilidad).

7.6. Protección de datos

- **Minimización:** las vistas de terceros (perfiles y contrapartes de similitud) muestran solo nombre, rol, foto y datos de contacto institucional.
- **Cifrado en reposo selectivo:** contraseñas (hash) y contraseñas temporales (cifrado simétrico con APP_KEY); el resto de los datos personales no se cifra a nivel de columna (decisión del MVP).
- **Cifrado en tránsito:** HTTPS (proveedor) con SESSION_SECURE_COOKIE recomendado en producción y TrustProxies configurado para terminar TLS en el proxy.

- **Credenciales institucionales:** los PDF con contraseña temporal deben manejarse como material sensible; el sistema audita su exportación, pero no controla su distribución posterior (riesgo R-08, § 7.13).
- **Datos de demostración:** el seeder crea personas ficticias (documentos 10000000XX, correos @sena.edu.co); no corresponden a personas reales.

7.7. Gestión de secretos

- backend/.env está excluido del control de versiones; solo se versiona .env.example sin valores sensibles.
- En producción, el arranque exige APP_KEY (el entrypoint falla con error explícito si falta) y las credenciales de base de datos llegan por variables del proveedor (Railway), no por archivos.
- APP_DEBUG=false y logs a stderr en el contenedor; no se expone phpinfo ni rutas de depuración.
- En esta auditoría **no se localizaron secretos reales** (claves, tokens o contraseñas de producción) en el repositorio; las únicas credenciales presentes son las de demostración del seeder, documentadas como tales.
- Riesgo residual: los PDF de credenciales y los correos son canales externos al control del sistema; se recomienda política institucional de manejo.

7.8. Dependencias y riesgos de seguridad

La auditoría ejecutó los verificadores de vulnerabilidades de ambos ecosistemas el 16/10/2026. Los resultados **difieren de las afirmaciones históricas de los README** (el frontend declaraba «0 vulnerabilidades»), porque se publicaron avisos nuevos con posterioridad; prevalece la verificación reciente:

Tabla 56. Resultado de los verificadores de dependencias (16/10/2026)

Herramienta	Resultado	Detalle principal	Acción recomendada
composer audit (backend)	42 avisos en 11 paquetes	league/commonmark (12, incl. alta), guzzlehttp/guzzle (9, incl. alta CVE-2026-69246), laravel/framework (4, incl. alta), symfony/mime (2, alta), symfony/http-foundation, symfony/mailer, symfony/routing, guzzlehttp/psr7, symfony/yaml, symfony/polyfill-intl-idn, league/flysystem	Actualizar dependencias con composer update controlado (probando la suite) o priorizar los parches de las severidades altas.
npm audit (frontend)	3 vulnerabilidades altas	brace-expansion, source-map-js y undici; todas en la cadena de herramientas de desarrollo/build; npm reporta corrección disponible (npm audit fix)	Ejecutar npm audit fix y revalidar build/lint/E2E.

Fuente: Salida real de `composer audit` y `npm audit` durante la auditoría (no se modificaron dependencias).

Contexto de explotabilidad: `guzzle` se usa como transporte de correo y `commonmark` para renderizado de Markdown en correos; si el sistema no procesa contenido de usuarios a través de esas vías, la explotación remota directa es improbable, pero el riesgo de cadena de suministro permanece y debe planificarse (riesgo R-02, § 7.13). No existe automatización de dependencias (Dependabot) ni CI que detecte avisos de forma continua.

7.9. Pruebas existentes

Tabla 57. Inventario de pruebas del repositorio

Suite	Herramienta	Composición	Qué cubre
Backend	PHPUnit 10	33 clases (31 Feature + 2 Unit), 244 métodos, 1.156 aserciones	Autenticación, autorización por rol y alcance, credenciales, fichas, propuestas (ciclo completo), motor y vigencia, comentarios, notificaciones, soporte, auditoría, atomicidad y regresiones de cierre de auditoría (BUG-001…).
Frontend unitarias	Vitest + Testing Library	17 archivos, 60 pruebas	Utilidades, validaciones, cliente HTTP, componentes base (formularios, tablas, badges, dials, paginación) y focus-trap.
Frontend E2E	Playwright (+ axe)	54 especificaciones (47 raíz + 7 agentes)	Flujos por rol, privacidad, permisos, ciclo de propuesta, similitudes, credenciales, responsividad (375/768), accesibilidad WCAG A/AA (claro y oscuro), navegación autónoma por agentes y casos negativos.
Móvil	—	Sin pruebas propias en el repositorio	La app Android no incluye suite automatizada (deuda técnica declarada).

Fuente: Inventario directo del repositorio (16/10/2026).

7.10. Pruebas ejecutadas durante la auditoría

Todas las ejecuciones siguientes se realizaron el 16/10/2026 sobre una **base de datos separada** (`proyectwin_mvp_manual`) para no alterar los datos del equipo, con el árbol de trabajo tal como se encontró:

Tabla 58. Verificaciones ejecutadas y resultados reales

#	Verificación	Comando / método	Resultado
1	Instalación backend	composer install	Correcta (paquetes descubiertos; 81 sugerencias de financiación, sin errores)
2	Instalación frontend	npm install	227 paquetes en ~32 s
3	Clave de aplicación	php artisan key:generate	Correcta
4	Esquema y datos	php artisan migrate:fresh --seed	18 migraciones DONE + seed (15 usuarios, 16 propuestas, 10 similitudes, 30 notificaciones)
5	Arranque API	php artisan serve + GET /up	200 — {"status":"ok"}
6	Arranque SPA	npm run dev + carga del portal	200 — interfaz operativa
7	Suite backend	php artisan test	**244 superadas / 0 fallidas** (1.156 aserciones, 35,93 s)
8	Unitarias frontend	npm run test:unit	**60 superadas** en 17 archivos (12,85 s)
9	Análisis estático	npm run lint	0 hallazgos
10	Integridad de datos	10 consultas SQL de verificación (§ 4.12)	0 incidencias en los 10 chequeos
11	Capturas de interfaz	Script Playwright propio (24 pantallas)	24 capturas reales (Capítulo 8)
12	Auditoría de dependencias	composer audit / npm audit	42 avisos / 3 altas (dev) — § 7.8
13	Suite E2E completa	npm run test:e2e	**No ejecutada** en esta auditoría (ver limitación siguiente)

#	Verificación	Comando / método	Resultado
14	Compilación Android	gradlew assembleDebug	**No ejecutada** (Docker y emulador no disponibles)

Fuente: Ejecución real de la auditoría (salidas registradas).

Limitación de la verificación E2E: la suite completa re-siembra la base antes de cada archivo y puede tardar decenas de minutos con reseeds; en esta auditoría se priorizó una verificación representativa (capturas navegadas por rol con el sistema real) y la ejecución íntegra de la suite de backend y de unitarias. La suite E2E queda documentada como existente y pendiente de re-ejecución completa por el equipo (Anexo F).

7.11. Resultados y evidencias

- Las capturas del Capítulo 8 demuestran que los flujos principales funcionan de extremo a extremo con la API real y datos sembrados (sin maquetas).
- La suite de backend cubre las regresiones de la auditoría interna previa: CierreAuditoriaTest referencia explícitamente BUG-001…027 y NEW-001…004, incluidos los dos hallazgos críticos de aquel informe (fuga de datos por included y traslados de aprendices entre fichas).
- Verificación independiente de los dos hallazgos críticos en el código actual: (1) GeneralUser::$hidden oculta documento, username y metadatos de credenciales; (2) el traslado de aprendices entre fichas por instructores responde 422 («Solo un administrador puede trasladarlo») en los tres caminos (store, agregarAprendiz, update). Ambos se clasifican como **corregidos y con prueba de regresión** al cierre de esta auditoría.
- Los conteos se obtuvieron por lectura directa y ejecución; donde difieren de los README (E2E y tests por archivo), prevalece esta verificación.

Conclusión de seguridad honesta: el sistema aplica prácticas sólidas de autenticación, autorización y trazabilidad para su alcance de MVP; **no se afirma que sea «seguro»** en términos absolutos. Existen riesgos abiertos de dependencias, ausencia de CI, ausencia de pruebas de penetración y decisiones de infraestructura pendientes (sesiones, copias de seguridad) que se detallan a continuación.

7.12. Limitaciones identificadas

- No se ejecutaron pruebas de penetración ni escaneo dinámico de vulnerabilidades.
- No se verificó el comportamiento tras un balanceador/proxy real (el rastro de IP en auditoría depende de TrustProxies).
- La accesibilidad (WCAG) se apoya en las especificaciones axe existentes; no se re-ejecutaron completas en esta auditoría.

- No hay pruebas de carga: los límites de rendimiento y los efectos del tamaño de la base (fotos en texto) no están medidos.
- La app móvil no fue compilada ni ejecutada en esta auditoría (análisis estático únicamente).
- El motor de similitud no fue evaluado con métricas de precisión/recall sobre un conjunto etiquetado real (existe la herramienta similitud:calibrar para ello, sin resultados versionados).

7.13. Riesgos técnicos y medidas de mitigación

Tabla 59. Registro de riesgos (observados y potenciales)

ID	Riesgo	Causa	Impacto	Prob.	Nivel	Evidencia	Mitigación
R-01	Dependencias con avisos de seguridad vigentes (backend y dev-tools)	Versiones fijadas por el lockfile frente a avisos publicados en 2026	Medio	Alta	Alto	composer audit (42), npm audit (3 altas)	Planificar composer update/npm audit fix con revalidación de la suite; activar Dependabot al crear CI.
R-02	Fin de soporte de Laravel 10 y PHP 8.1/8.2	El framework y la rama PHP declaradas salieron del ciclo de soporte (PHP 8.2 termina el 31/12/2026)	Alto	Alta	Alto	composer.json (^8.1, laravel ^10); php.net; laravel.com (rama actual 13.x)	Migrar a Laravel 11/12 y PHP 8.3+ (la imagen Docker ya usa 8.3); planificar ventana de migración.
R-03	Sesiones de archivo se pierden en cada redespliegue	SESSION_DRIVER=file con contenedor efímero	Medio	Alta	Medio	.env.example; migración sessions en curso	Adoptar SESSION_DRIVER=database (migración ya creada en el árbol de trabajo) o Redis gestionado.
R-04	Ausencia de integración continua	No hay flujos CI en el repositorio	Medio	Alta	Medio	Sin .github/workflows; verificaciones manuales	Crear pipeline (lint + tests backend + unit + build) en cada push.
R-05	Sin estrategia de copias de seguridad implementada	No hay scripts ni retención documentada	Alto	Media	Alto	§ 4.11 (procedimiento solo recomendado)	Volcados programados + prueba de restauración trimestral; verificar respaldos del plugin MySQL.
R-06	Errores de correo no visibles para el usuario final	Sin SMTP, el aviso de credenciales falla y solo queda credenciales_error	Bajo	Media	Bajo	CredencialesCorreo (diseño fail-safe)	Configurar SMTP en producción y monitorear el campo; mostrar el error al admin que reenvía.
R-07	Crecimiento de la base por fotos en base64	foto_url almacena data URLs	Medio	Media	Medio	general_users.foto_url; utils/foto.js (256 px)	Mover fotos a almacenamiento de objetos con URL firmada si el volumen crece.
R-08	Distribución insegura de PDF de credenciales	El PDF contiene contraseña temporal en claro	Alto	Baja	Medio	CredencialesPdf	Política institucional de entrega; acortar vigencia de temporales; auditar exportaciones (ya se audita).
R-09	Sin límite específico de intentos de login	El throttle es global por API, no por credencial	Medio	Media	Medio	Kernel.php (grupo api)	Añadir rate limiter dedicado al POST /auth/login y alertas de auditoría.
R-10	Borrado de ficha elimina filas de aprendices (CASCADE)	FK apprentices.id_class_group ON DELETE CASCADE	Medio	Baja	Bajo	Migración; hallazgo histórico BUG-007	Cambiar a SET NULL o nullificar antes de borrar (mejora planificada).
R-11	Doble detección concurrente del mismo par	Índice no único en similarities + buscar-y-crear	Bajo	Baja	Bajo	Recomputador (lock sobre motor_configs serializa)	Índice único parcial por par vigente si el motor escala.
R-12	Documentación interna desactualizada	README/CHANGELOG describen etapas previas	Bajo	Alta	Medio	§ 1.9; Anexo F	Actualizar README con los conteos reales y enlazar a este manual.
R-	APK sin	No hay signingConfig	Medio	Media	Medi	Movil/app/build.gradle.k	Definir firma por

ID	Riesgo	Causa	Impacto	Prob.	Nivel	Evidencia	Mitigación
13	configuración de firma en Gradle	declarado			o	ts	variables/keystore fuera del repositorio antes de distribuir.
R-14	Sin observabilidad centralizada	Logs locales; sin métricas ni alertas	Medio	Media	Medio	LOG_CHANNEL=stack; contenedor a stderr	Servicio de recolección de logs y alertas de errores; healthcheck ya existe.
R-15	Trabajo en curso sin confirmar en Git	Artefactos de despliegue y migración sessions sin commit al cierre	Bajo	Alta	Medio	git status del 16/10/2026	Confirmar y etiquetar antes de la entrega; revisar routes/web.php y TrustProxies modificados.

Fuente: Auditoría (observados) y análisis (potenciales). «Nivel» = combinación cualitativa de impacto y probabilidad.

7.14. Plan de mejora

Orden sugerido por relación impacto/esfuerzo, coherente con los riesgos anteriores:

31. **Confirmar y etiquetar** el trabajo en curso (despliegue, sessions) y actualizar README con conteos reales (R-12, R-15).
32. **Actualizar dependencias** con severidad alta y revalidar las suites (R-01).
33. **Crear CI mínimo** (lint + pruebas backend + unitarias + build) y activar Dependabot (R-04).
34. **Adoptar `SESSION_DRIVER=database`** y configurar SMTP real (R-03, R-06).
35. **Implantar copias de seguridad** programadas con prueba de restauración (R-05).
36. **Rate limiting específico de login** y alertas de auditoría sobre fallos repetidos (R-09).
37. **Plan de migración de versión** (Laravel 11/12 + PHP 8.3+), con ventana y pruebas de regresión (R-02).
38. **Endurecer borrados** (aprendices sin ficha al borrar ficha) y revisar la unicidad del par vigente (R-10, R-11).
39. **Observabilidad**: recolección de logs y alertas de errores (R-14).
40. **Firma de APK** y suite de pruebas para la app móvil (R-13).
41. **Re-ejecutar la suite E2E completa** y agregar pruebas de concurrencia (R-11, limitación § 7.10).

7.15. Control de calidad y normas ISO (análisis de pertinencia)

Las ediciones vigentes y las denominaciones se verificaron **directamente en el catálogo oficial de ISO el 16 de octubre de 2026** (fuente: iso.org, referencias completas en la bibliografía). Este apartado es un **análisis de pertinencia con criterios de evaluación propuestos**, no una auditoría ni una declaración de cumplimiento: se distingue, para cada norma, la práctica observada de la evidencia de cumplimiento.

Tabla 60. Normas de referencia: estado verificado y relación con el proyecto

Norma	Edición vigente verificada (16/10/2026)	Propósito general	Relación con ProyecTwin
ISO/IEC	ISO/IEC 27001:2022 (+ Amd	Requisitos de un sistema	La seguridad implementada (roles,

Norma	Edición vigente verificada (16/10/2026)	Propósito general	Relación con ProyecTwin
27001	1:2024)	de gestión de seguridad de la información (SGSI).	cifrado selectivo, auditoría, validación) se puede leer como prácticas alineadas a controles; el proyecto **no tiene un SGSI**.
ISO 14001	ISO 14001:2026 (retiró la edición 2015)	Requisitos de un sistema de gestión ambiental.	Sin relación funcional directa; pertinente solo como criterio organizacional/institucional si el SENA lo exige al proyecto (p. ej. consumo energético de la infraestructura).
ISO/IEC 25010	ISO/IEC 25010:2023 (modelo de calidad del producto; línea SQuaRE con 25019:2023 y 25002:2024)	Modelo de características de calidad de producto de software.	Aplicable como **marco de evaluación**: idoneidad, seguridad, fiabilidad, mantenibilidad, usabilidad, compatibilidad, rendimiento y portabilidad se pueden calificar con la evidencia disponible.
ISO/IEC 9126	Retirada (ISO/IEC 9126-1:2001, revisada por la línea 25010)	Modelo histórico de calidad del producto (2001).	Referencia **histórica**: su sucesión por ISO/IEC 25010:2023 debe citarse correctamente; no debe usarse como norma vigente.
ISO 37001	ISO 37001:2025 (retiró la edición 2016)	Sistemas de gestión antisoborno.	No aplica al alcance técnico del proyecto; se menciona por completitud institucional.
ISO 10005	Ediciones 1995 y 2005 retiradas; la edición de 2018 no pudo confirmarse en la ficha del catálogo durante esta consulta	Directrices para planes de calidad.	Útil si el equipo formaliza un plan de calidad del proyecto (versión, alcance, criterios de aceptación); verificar la edición vigente antes de citarla.
ISO 9001	ISO 9001:2026 (publicada el 07/10/2026; retiró la 9001:2015 el 16/09/2026)	Requisitos de un sistema de gestión de la calidad.	Marco de referencia para documentación, control de cambios y mejora continua; el proyecto **no está certificado**.
Familia ISO/IEC 33000	ISO/IEC 33002:2015 publicada y confirmada (revisión 2026-01-08); predecesora ISO/IEC 15504-2:2003 retirada	Evaluación de procesos de software (conceptos, requisitos y modelos de capacidad).	Permitiría evaluar la madurez de procesos (desarrollo, pruebas, configuración); requeriría una evaluación formal con evidencia histórica del equipo.
Familia ISO/IEC 27000	ISO/IEC 27000:2026 (edición 6; retiró la de 2018)	Visión general, conceptos y vocabulario de la familia de seguridad de la información.	Vocabulario de referencia para el apartado de seguridad; su uso no implica conformidad con ISO/IEC 27001.

Fuente: Catálogo oficial de ISO (iso.org), consulta directa 16/10/2026; referencias completas en la bibliografía.

Lectura crítica por norma (qué se podría evaluar, qué hay y qué falta):

- **ISO/IEC 27001:2022** — *Criterios evaluables:* control de acceso (roles, mínimo privilegio), criptografía (hash/cifrado selectivo), gestión de vulnerabilidades (audit de dependencias), registro de eventos (bitácora), gestión de incidentes (reportes de soporte). *Evidencias disponibles:* middlewares y scopes, $hidden, auditoría inmutable, salidas de composer/npm audit. *No comprobado:* SGSI formal, análisis de riesgos documentado, revisión por la dirección, continuidad de negocio. *Recomendación:* mantener este manual como evidencia técnica primaria si la institución decide formalizar un SGSI.
- **ISO/IEC 25010:2023** — *Criterios evaluables con la evidencia de este manual:* idoneidad funcional (trazabilidad RF, capítulos 3 y 8), fiabilidad (atomicidad, 244 pruebas), seguridad (capítulo 7), usabilidad (WCAG, especificaciones axe), compatibilidad (web/móvil/escritorio),

mantenibilidad (convenciones, lint, documentación), portabilidad (Docker). *No comprobado:* medición cuantitativa de eficiencia de desempeño y fiabilidad bajo carga. *Recomendación:* usar la tabla de requisitos (§ 3.1–3.2) como base de una futura evaluación por características.

- **ISO 9001:2026** — *Criterios evaluables:* control de la información documentada (este manual, README, guías internas), control de cambios (historial de fixes, Git), mejora continua (plan § 7.14). *No comprobado:* sistema de gestión de la calidad formal, auditorías internas de calidad. *Nota:* la edición 2026 es reciente (07/10/2026); su transición afecta a organizaciones certificadas, no a este proyecto.
- **Familia ISO/IEC 33000** — *Criterios evaluables:* proceso de desarrollo (evidencia en Git y estructura), proceso de pruebas (suites y ejecución de § 7.10), proceso de gestión de configuración (migraciones consolidadas, control de versiones). *No comprobado:* evaluación de capacidad por niveles con evaluadores competentes. *Recomendación:* considerar un autodiagnóstico ligero si se busca madurez de procesos.
- **Familia ISO/IEC 27000 y ISO 37001 / ISO 14001 / ISO 10005** — se documentan como **referencias institucionales**; no se identificó evidencia interna específica más allá del vocabulario de seguridad (27000) y la oportunidad de formalizar un plan de calidad (10005) con la estructura de este manual (objetivo, alcance, criterios de aceptación, control de versiones).

Declaración explícita: ninguna sección de este capítulo permite afirmar que ProyecTwin «cumple» una norma ISO en sentido certificable. Lo documentado son **prácticas observadas**, **evidencia disponible** y **aspectos no evaluados**, con recomendaciones para una eventual evaluación formal.

CAPÍTULO 8. INTERFACES Y FUNCIONALIDADES IMPLEMENTADAS

8.1. Metodología de verificación visual

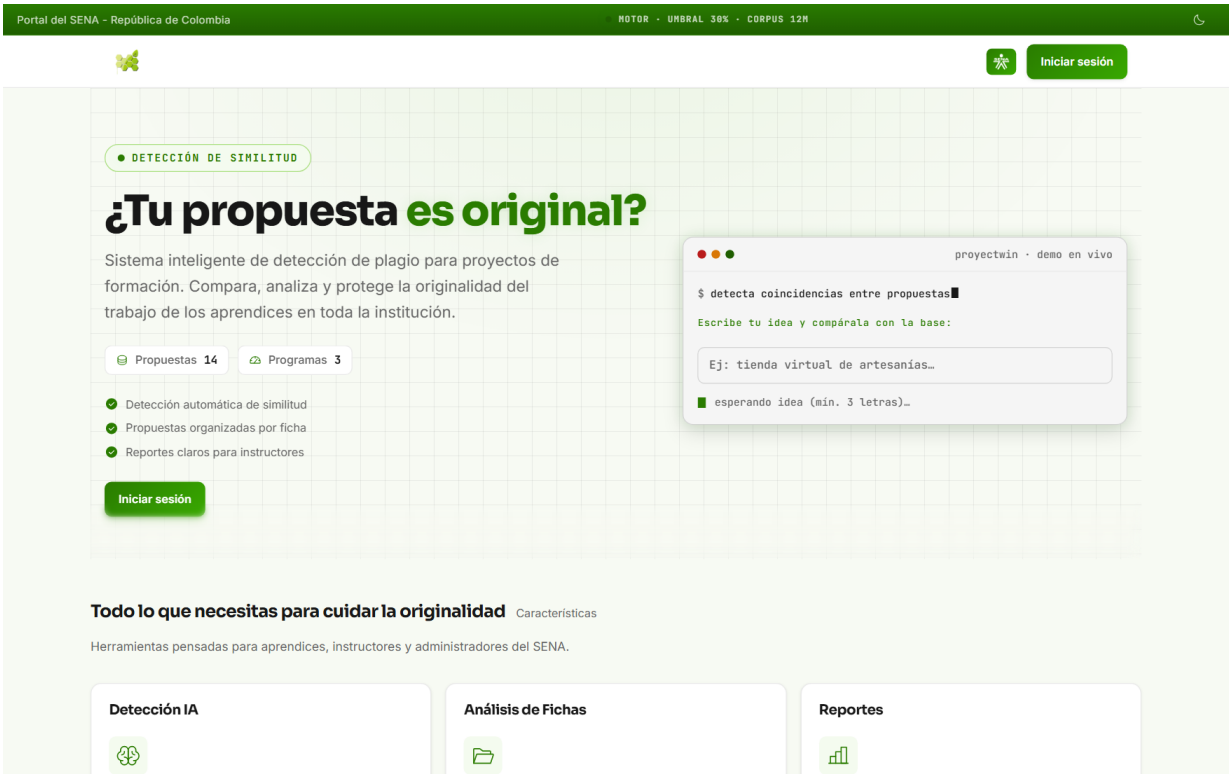
Todas las capturas de este capítulo se tomaron el **16 de octubre de 2026** sobre el **sistema en ejecución** (API en `127.0.0.1:8000` y SPA en `localhost:5173`), con la base de datos sembrada por el `DatabaseSeeder` del proyecto. Las capturas se generaron con Chromium (Playwright) a 1400×880 y factor de escala 1,25 (escritorio) y 390×844 (móvil), con movimiento reducido activado. Los datos que aparecen son **sintéticos** (personas y documentos de demostración del seeder, § 7.6); no se exponen credenciales ni datos reales. La app Android **no** aporta capturas porque no se compiló en esta auditoría (limitación declarada). Las capturas del manual de usuario ajeno (PopayanAllTour) **no** se usaron como evidencia de este sistema.

Cada módulo se documenta con: objetivo, ubicación en el código, componentes, datos de entrada/salida, validaciones, errores y estado de implementación. El estado «verificado» significa que la pantalla fue ejecutada y capturada en esta auditoría; la lógica asociada está además cubierta por las suites cuando se indica.

8.2. Módulo público (portal, acceso y recuperación)

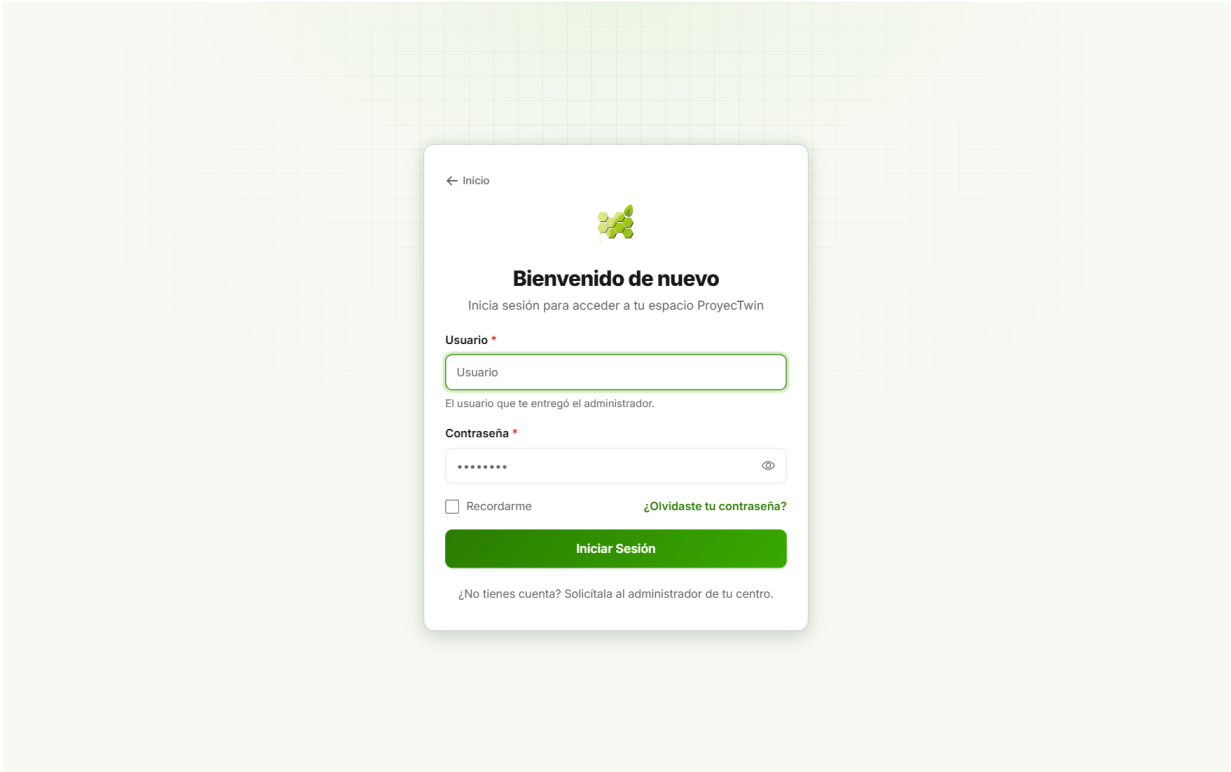
Objetivo: presentar el sistema sin sesión, ofrecer la demo pública del motor y gestionar el acceso. **Ubicación:** `frontend/src/pages/public/{Home,Login,RecuperarContrasena,RestablecerContrasena,PaginaNoEncontrada}`; API pública en `AuthController`, `MotorConfigController@resumen` y `SimilarityController@demo`. **Componentes:** `LandingLayout`, `LandingHeader`, `GovernmentBar`, `TextType`, `ConsoleCard`. **Datos:** la demo compara un texto (3–300 caracteres) contra el corpus aprobado y devuelve máximo 3 coincidencias con título y porcentaje (nunca autores). **Errores:** validación 422 del texto; el portal no requiere sesión.

Figura 9. Portal público de ProyecTwin con el resumen del motor y la demo de similitud



Fuente: Captura real del sistema en ejecución (16/10/2026).

Figura 10. Pantalla de inicio de sesión por nombre de usuario

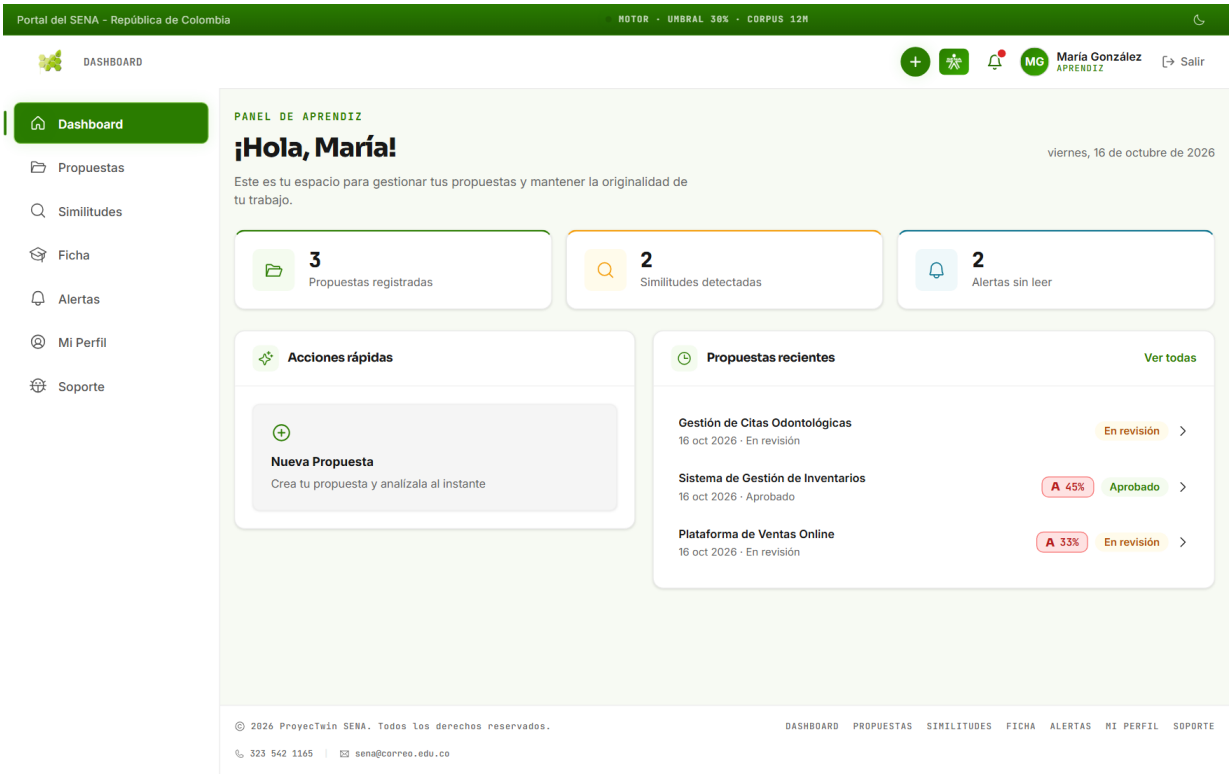


Fuente: Captura real del sistema en ejecución (16/10/2026).

8.3. Módulo del aprendiz

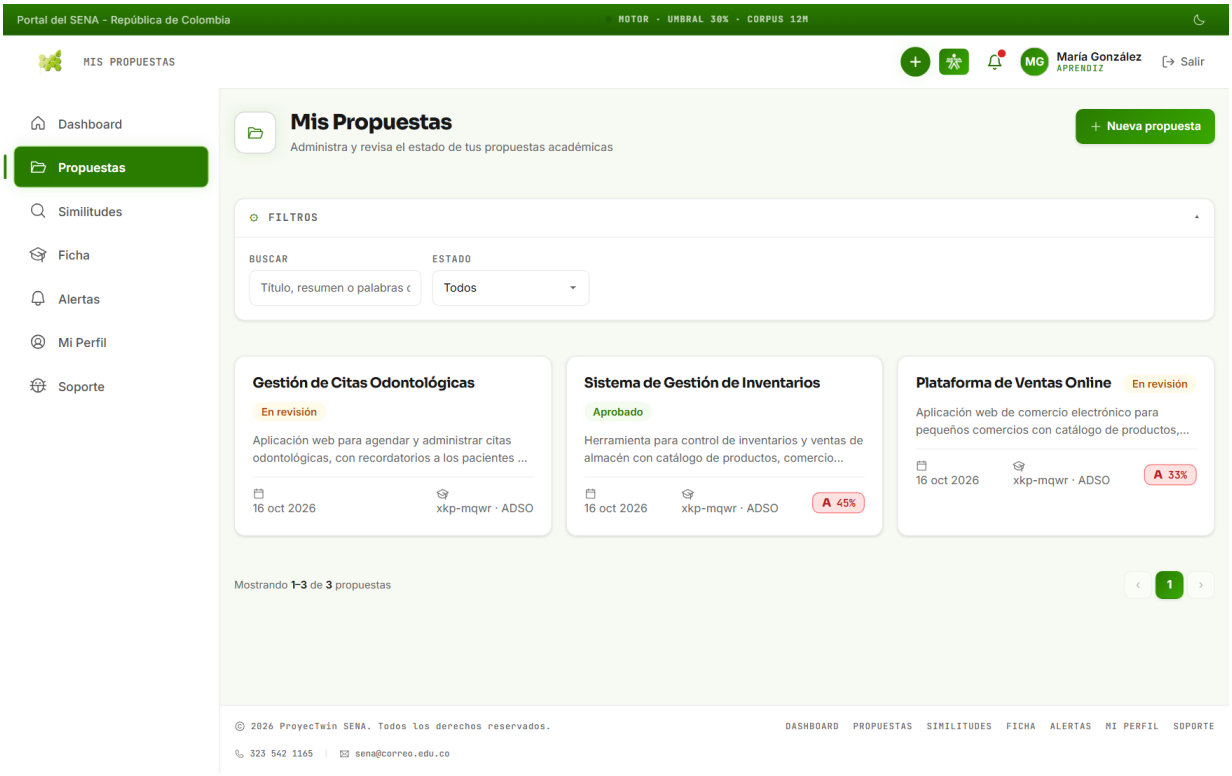
Objetivo: gestionar el ciclo completo de las propuestas propias (crear, enviar, corregir, seguir) dentro de una ficha activa. **Páginas:** 14 rutas en `frontend/src/pages/aprendiz/`. **Reglas clave:** solo se participa dentro de la ficha activa (regla «Classroom»); el aprobado queda bloqueado; solo se reenvía con cambios reales; las similitudes muestran únicamente pares propios. **Cobertura por pruebas:** `Propuestas/FlujoPropuestaTest`, `Ficha/ApprenticeFichaTest`, `Equipo/EquipoTest` y las especificaciones E2E homónimas.

Figura 11. Panel del aprendiz: métricas personales, acciones rápidas y propuestas recientes con su grado de similitud



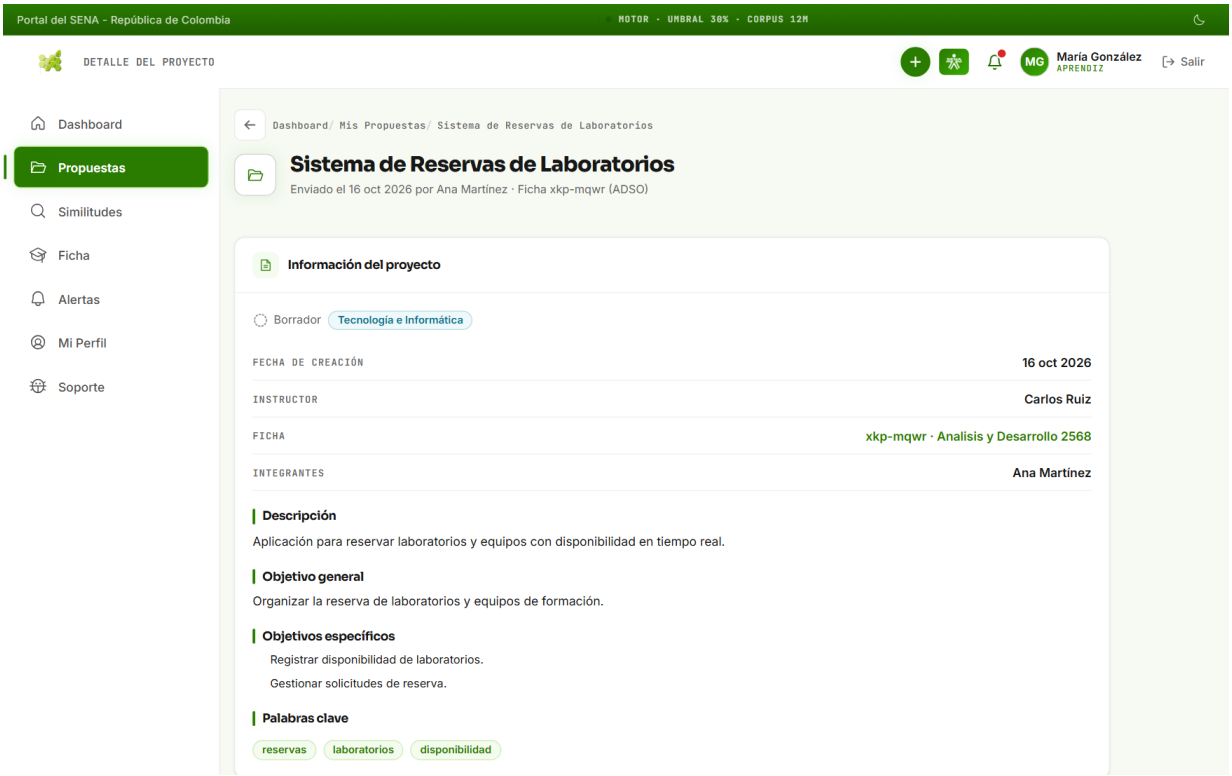
Fuente: Captura real (usuario de demostración «María González»).

Figura 12. Listado de propuestas del aprendiz con filtros por estado y grado de similitud



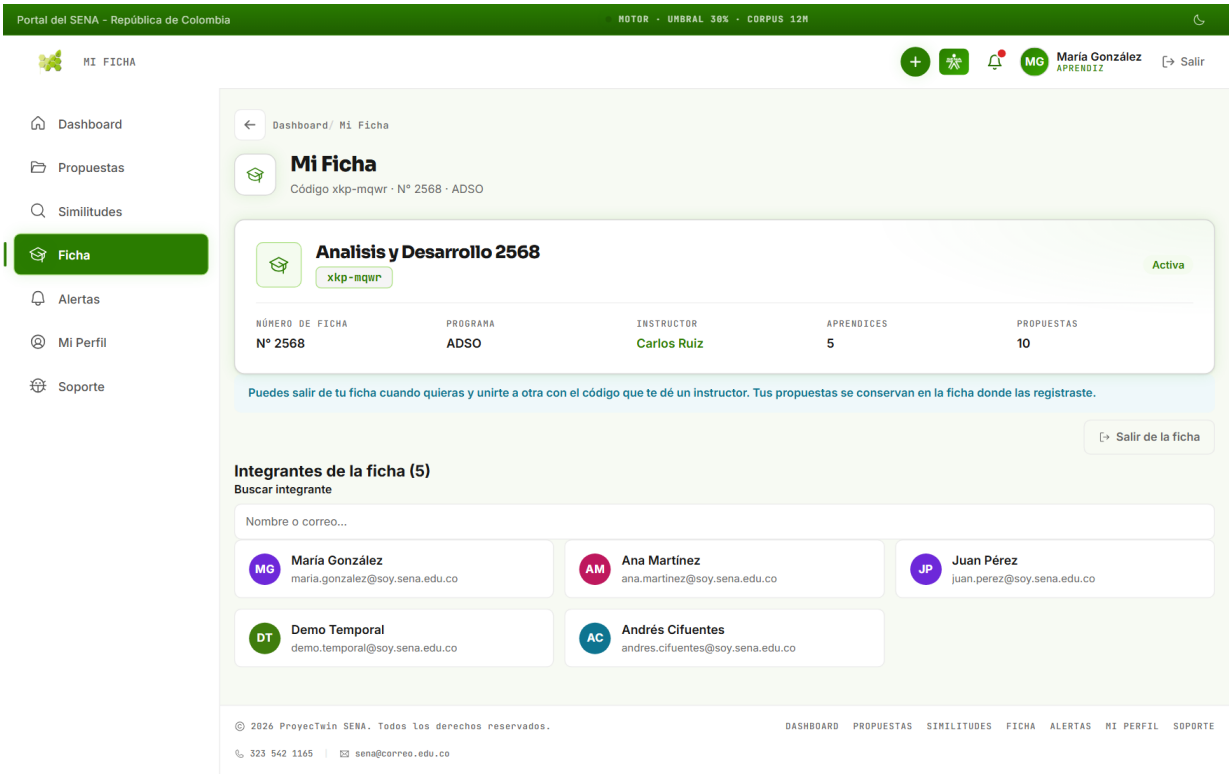
Fuente: Captura real del sistema en ejecución.

Figura 13. Detalle de una propuesta: datos, equipo, historial y acciones según estado



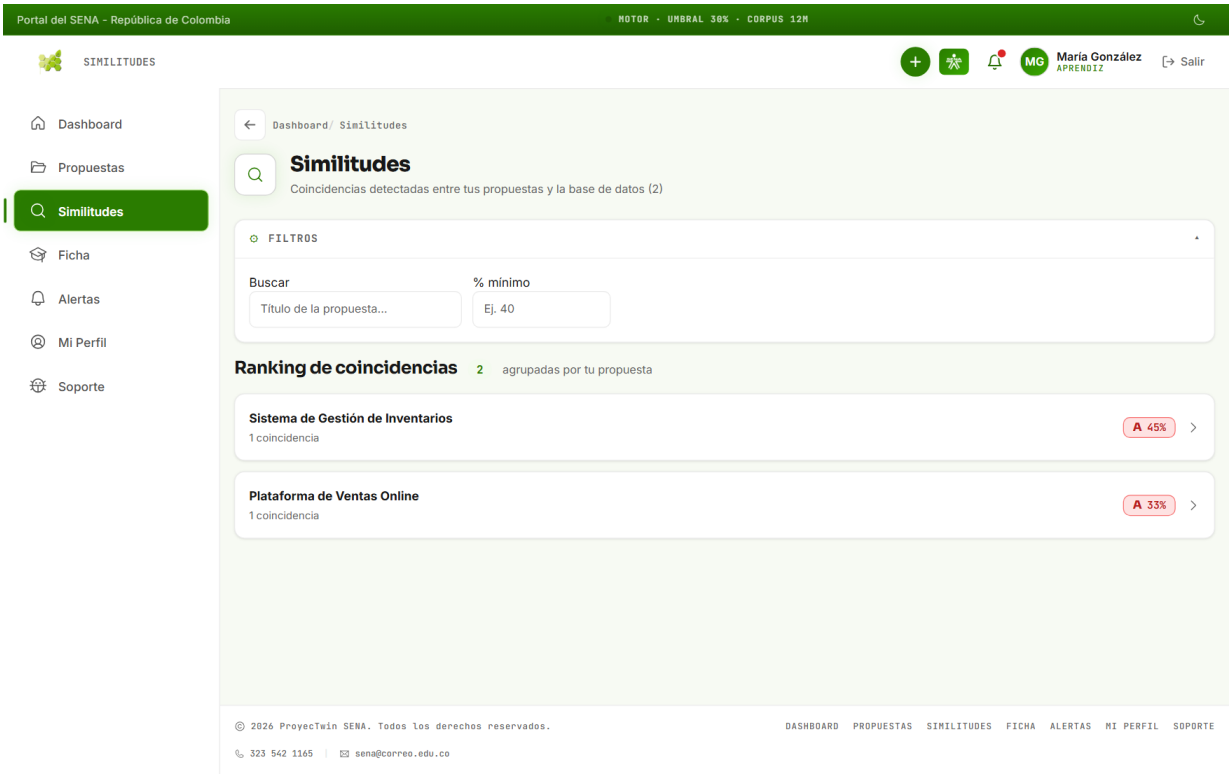
Fuente: Captura real del sistema en ejecución.

Figura 14. «Mi ficha»: unión por código, código propio y compañeros



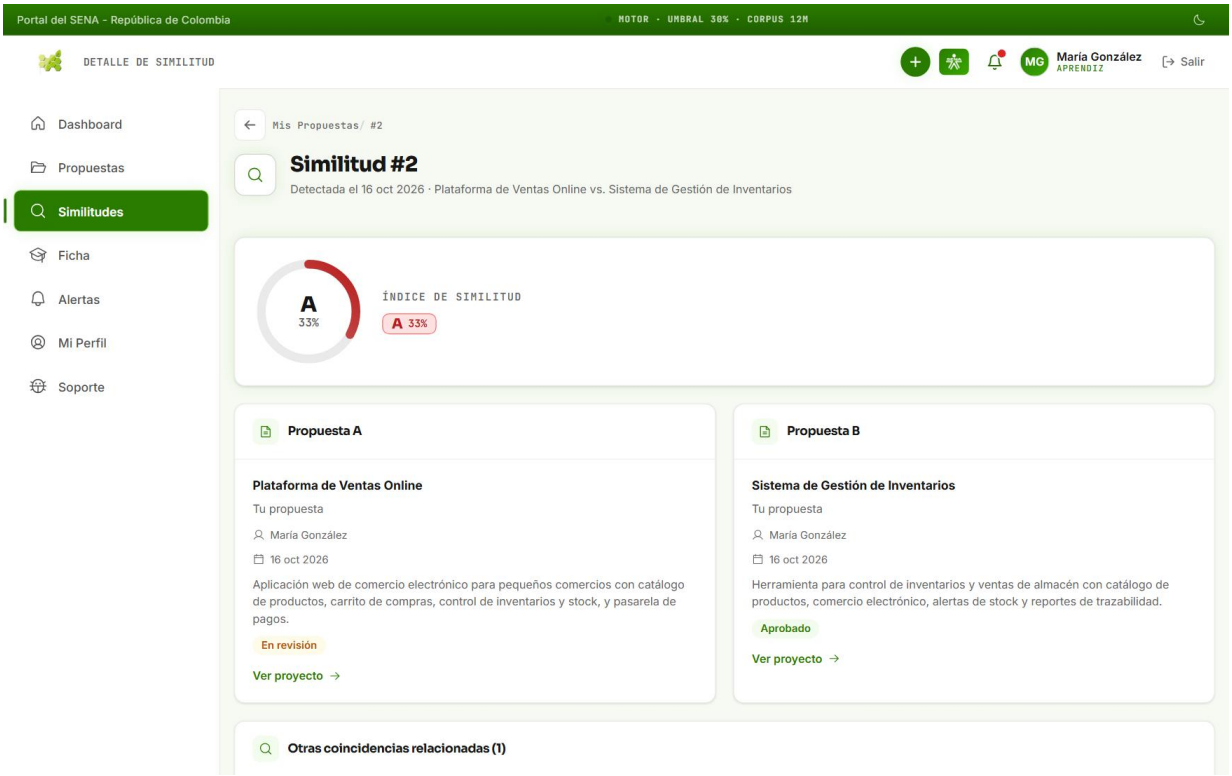
Fuente: Captura real del sistema en ejecución.

Figura 15. Mis similitudes: pares detectados con porcentaje y grado (el histórico se conserva)



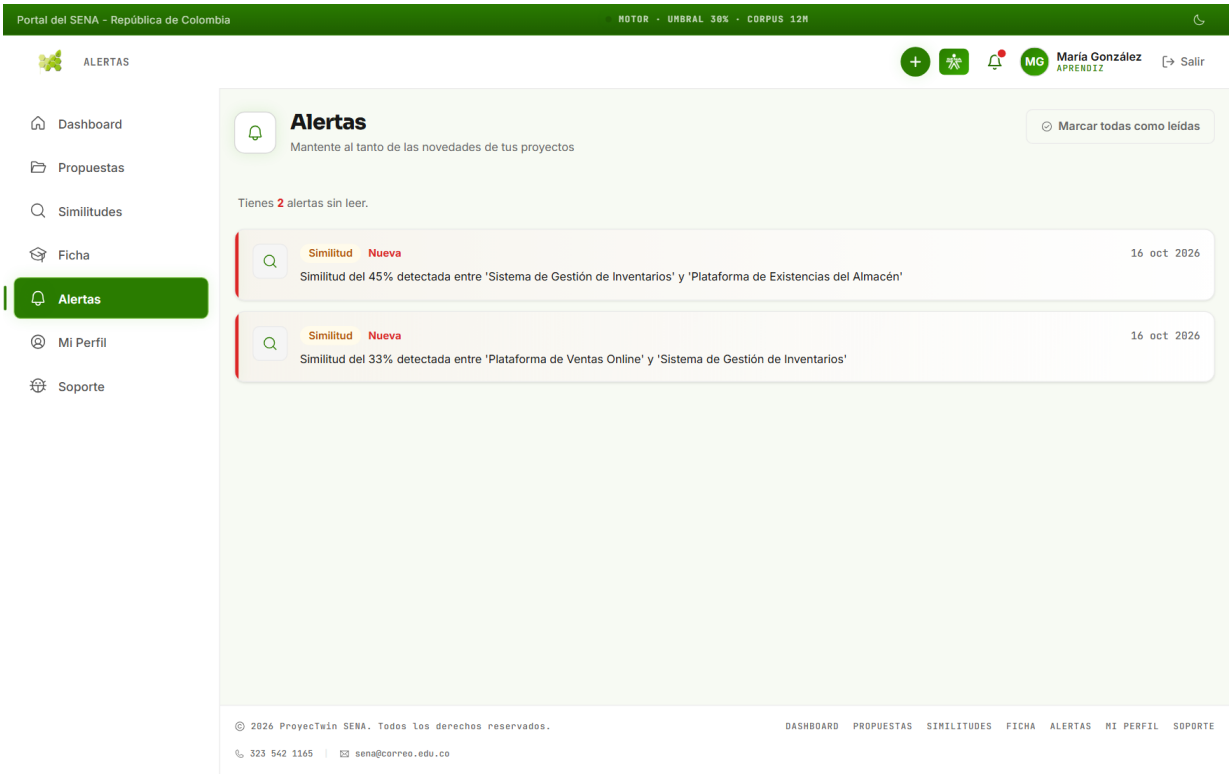
Fuente: Captura real del sistema en ejecución.

Figura 16. Detalle de similitud: comparación del par, señales del motor y pasajes coincidentes



Fuente: Captura real del sistema en ejecución.

Figura 17. Bandeja de alertas del aprendiz con estados de lectura

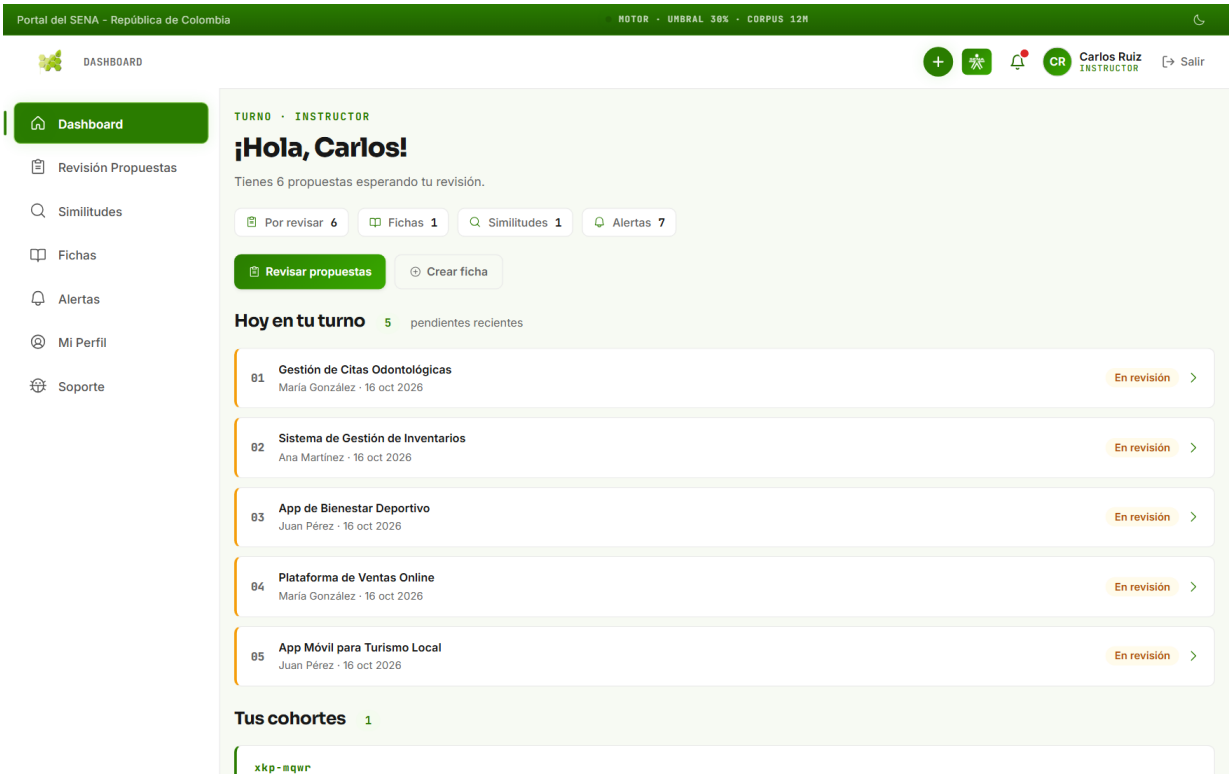


Fuente: Captura real del sistema en ejecución.

8.4. Módulo del instructor

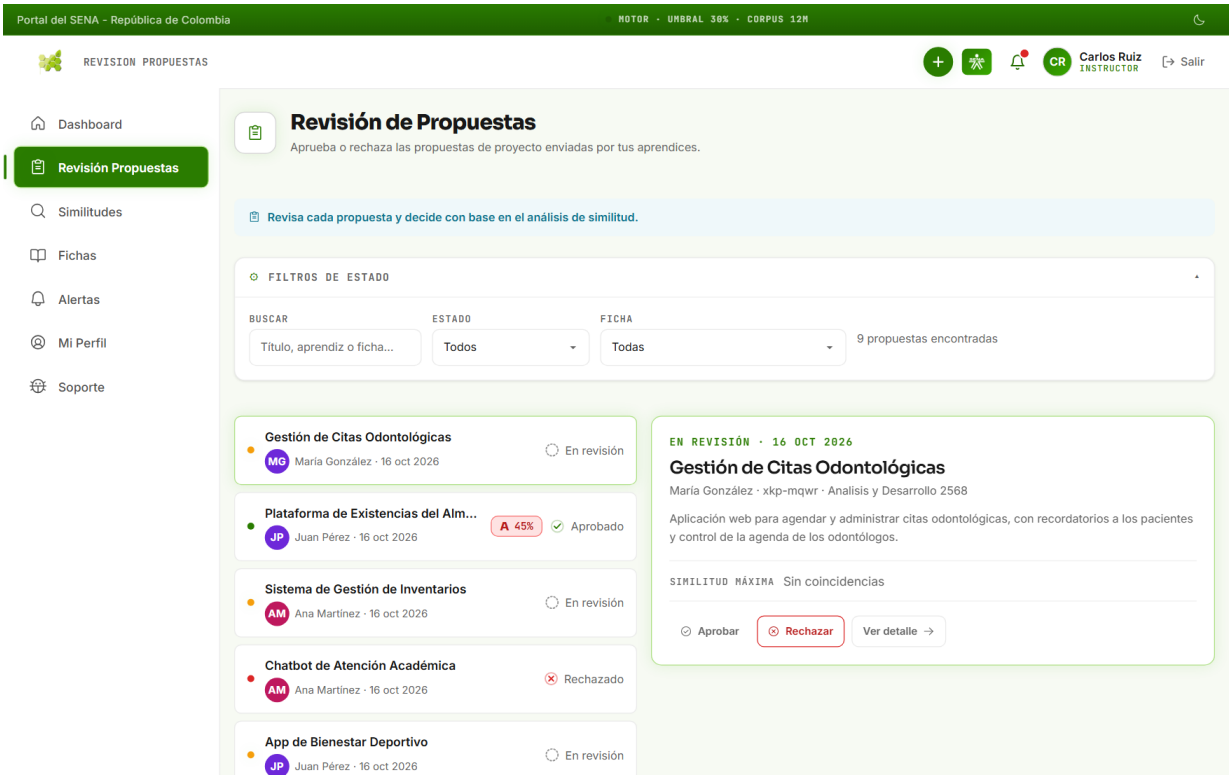
Objetivo: revisar las propuestas de sus fichas (aprobar/rechazar con observación), gestionar las fichas y su roster, y supervisar las similitudes de su alcance. **Páginas:** 11 rutas en `frontend/src/pages/instructor/`. **Reglas clave:** no edita aprobadas; el rechazo exige observación (guardada como comentario); la aprobación no re-ejecuta el motor (decisión verificada); puede crear fichas y dar de alta aprendices con credenciales. **Cobertura:** `instructor.spec.js`, `revision-rechazo.spec.js`, `ficha-edicion.spec.js` y las pruebas `PermisosTest`/`FichaAprendizTest`.

Figura 18. Panel del instructor: cola de revisión, fichas a cargo y actividad



Fuente: Captura real (usuario de demostración «Carlos Ruiz»).

Figura 19. Cola de revisión de propuestas con filtros por estado y ficha



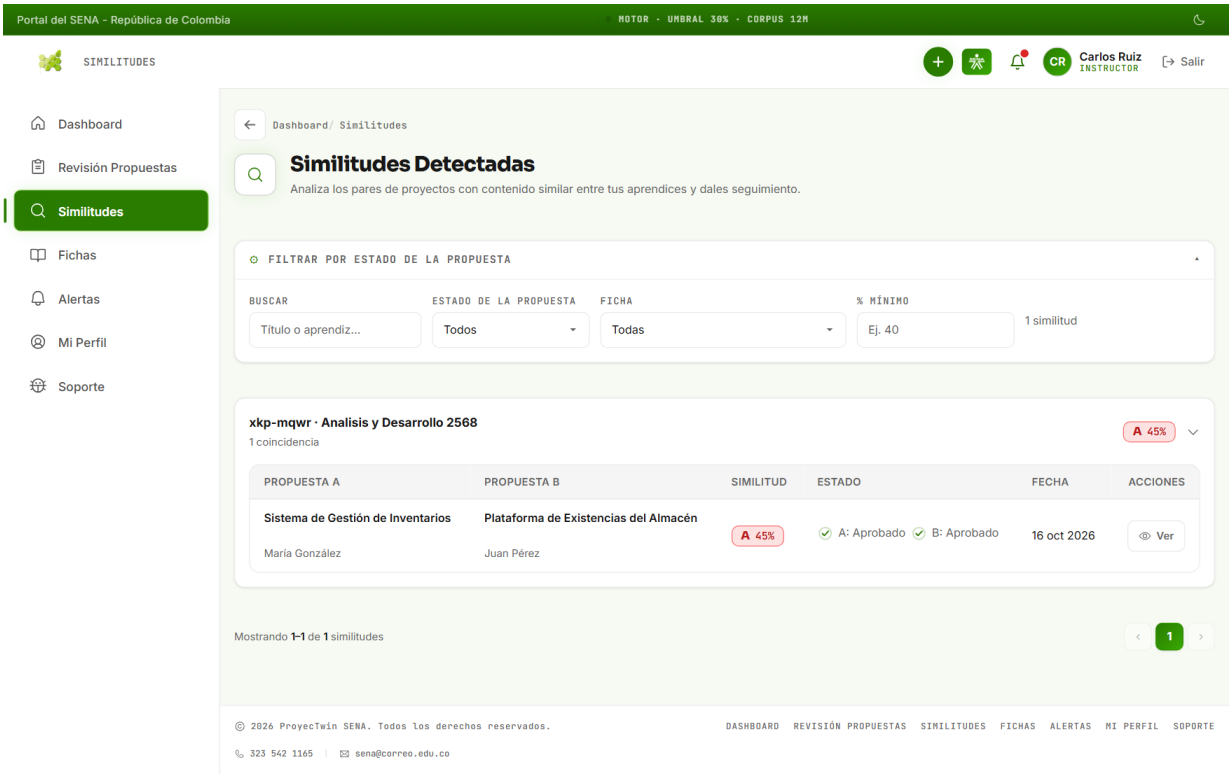
Fuente: Captura real del sistema en ejecución.

Figura 20. Detalle de propuesta para revisión: contenido completo, equipo y observaciones



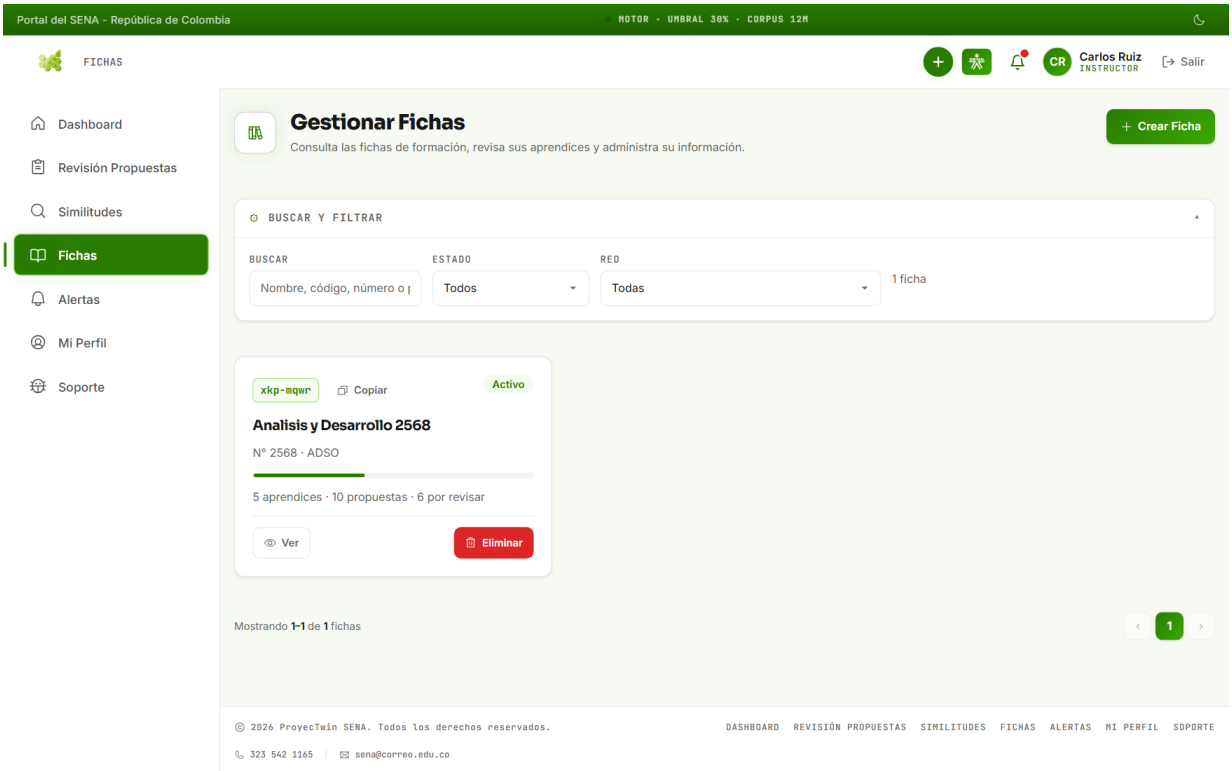
Fuente: Captura real del sistema en ejecución.

Figura 21. Similitudes del alcance del instructor agrupadas por ficha



Fuente: Captura real del sistema en ejecución.

Figura 22. Gestión de fichas del instructor (creación, edición y roster)

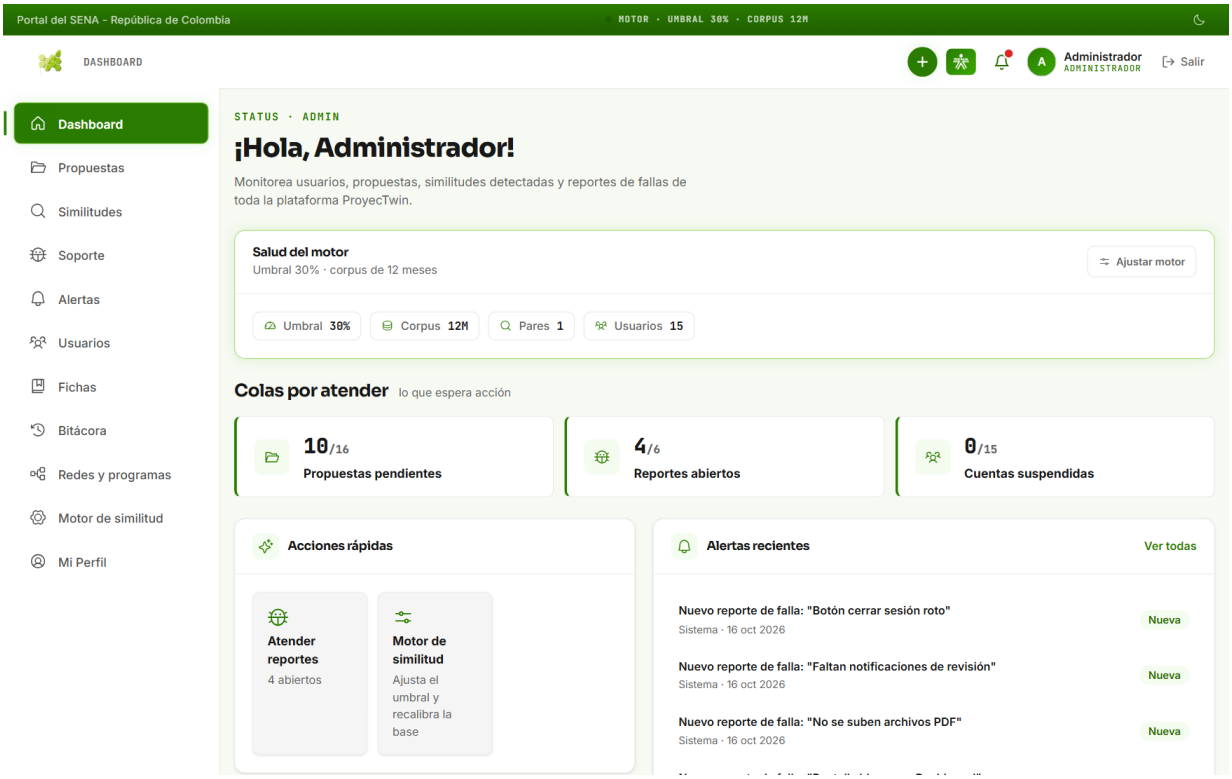


Fuente: Captura real del sistema en ejecución.

8.5. Módulo del administrador

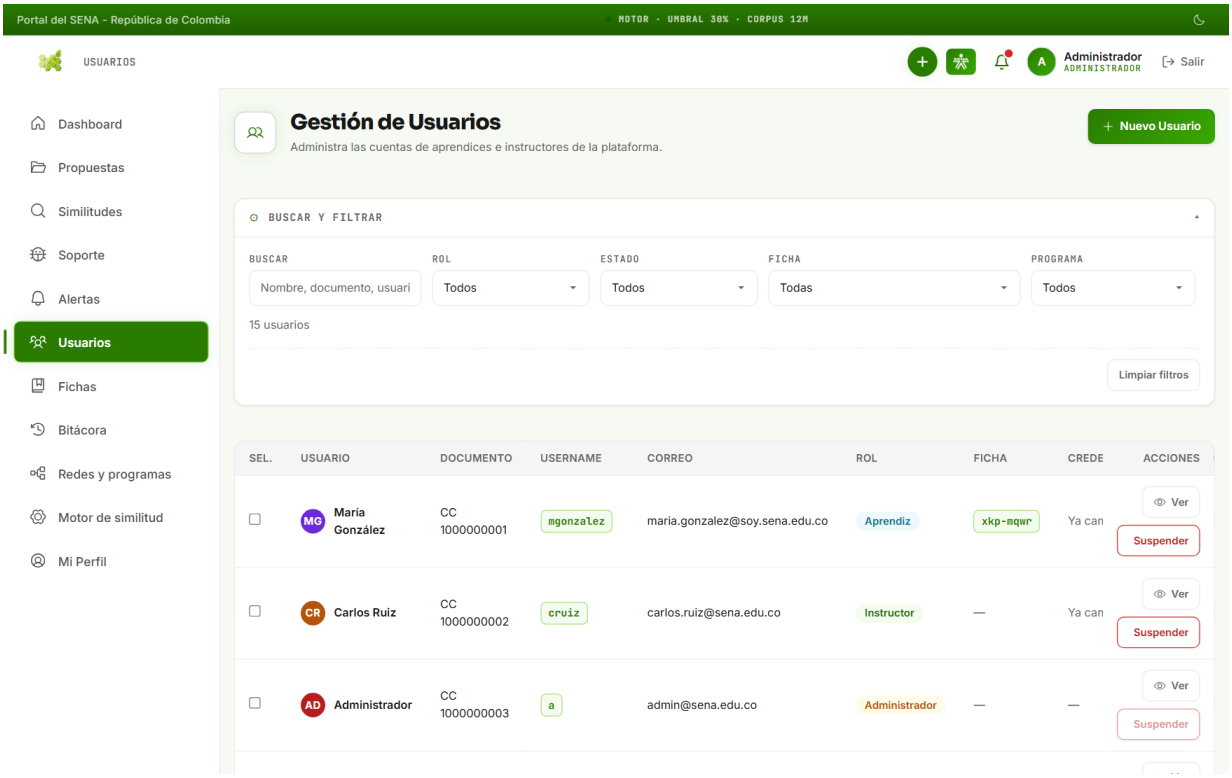
Objetivo: operar la plataforma: usuarios y credenciales, catálogos, fichas, propuestas, similitudes, motor, soporte y bitácora. **Páginas:** 16 rutas en `frontend/src/pages/admin/`. **Poderes singularizados:** único rol que exporta credenciales en PDF, configura el motor, recalcula el corpus, interviene propuestas aprobadas y consulta la bitácora. **Cobertura:** `admin*.spec.js` (6 especificaciones) y pruebas `DeleteGuardsTest`, `GeneralUserRolTest`, `CredencialesTest`.

Figura 23. Panel administrativo: métricas globales y colas de gestión



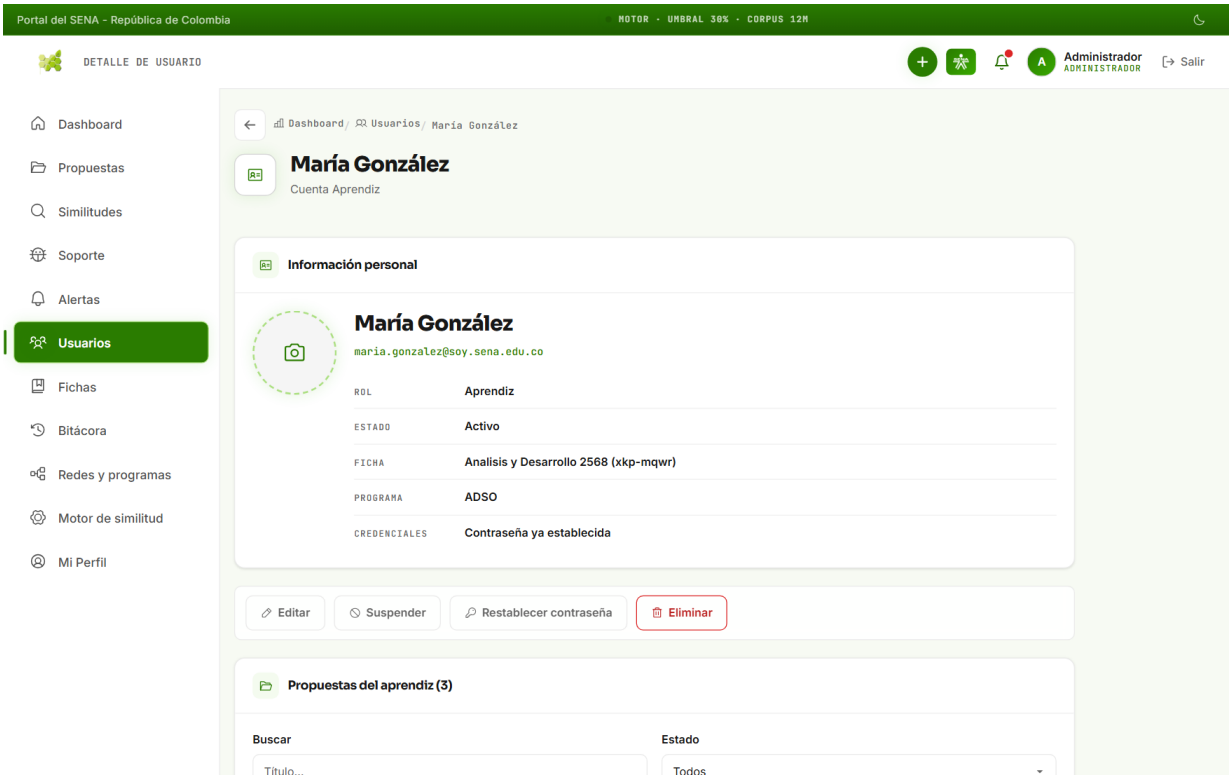
Fuente: Captura real (usuario de demostración «Administrador»).

Figura 24. Gestión de usuarios: búsqueda, filtros, estado y credenciales



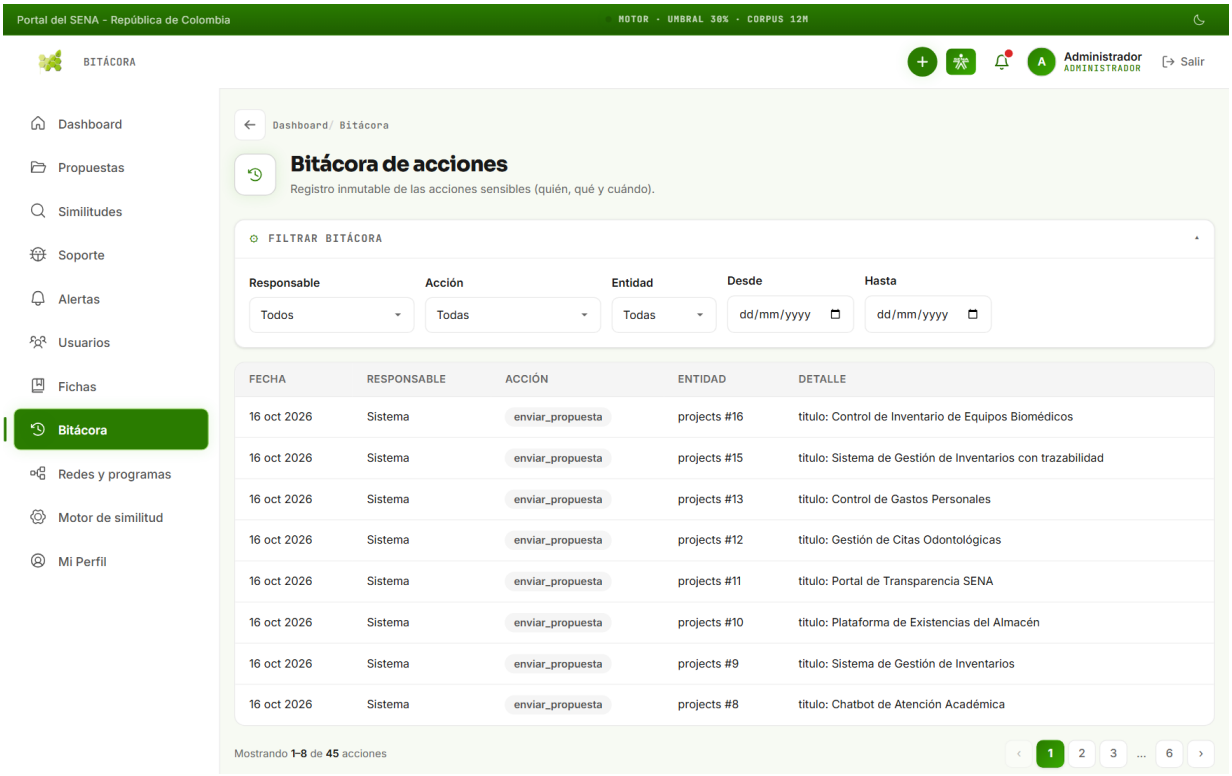
Fuente: Captura real; los datos mostrados son ficticios del seeder.

Figura 25. Detalle de usuario: perfil, poderes administrativos y actividad



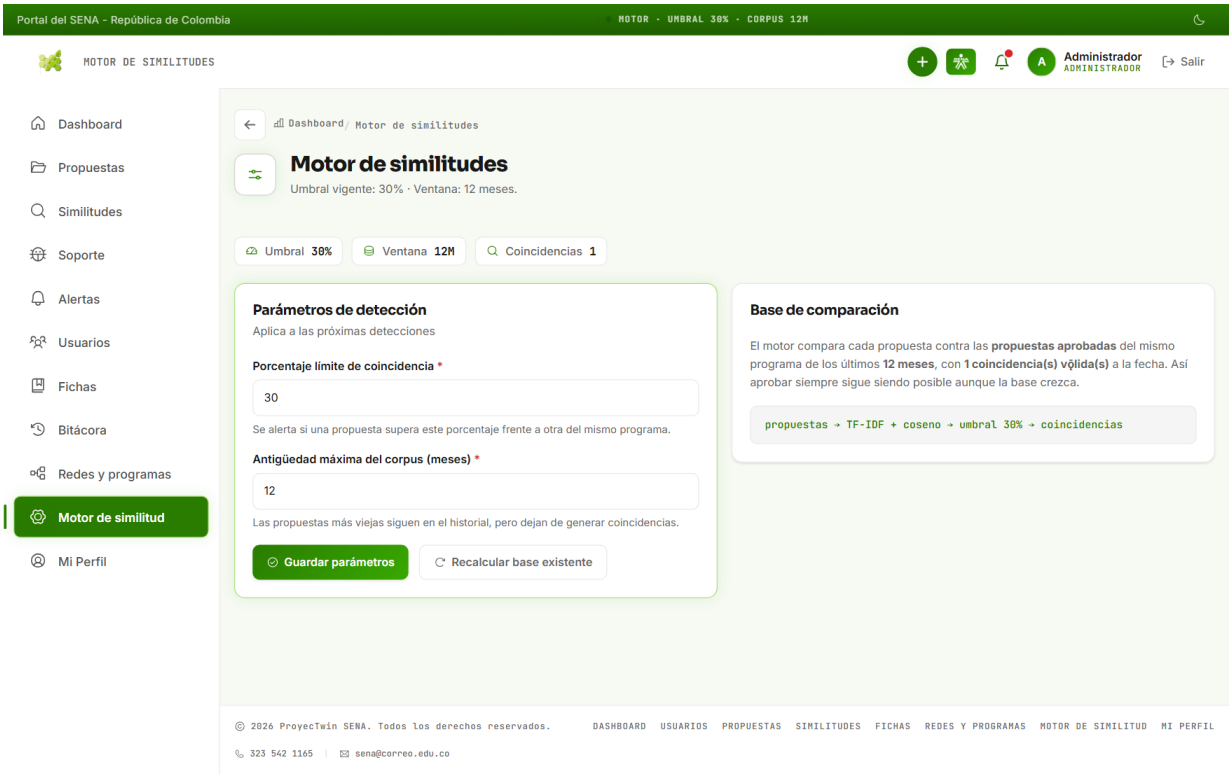
Fuente: Captura real del sistema en ejecución.

Figura 26. Bitácora de auditoría inmutable con filtros y paginación



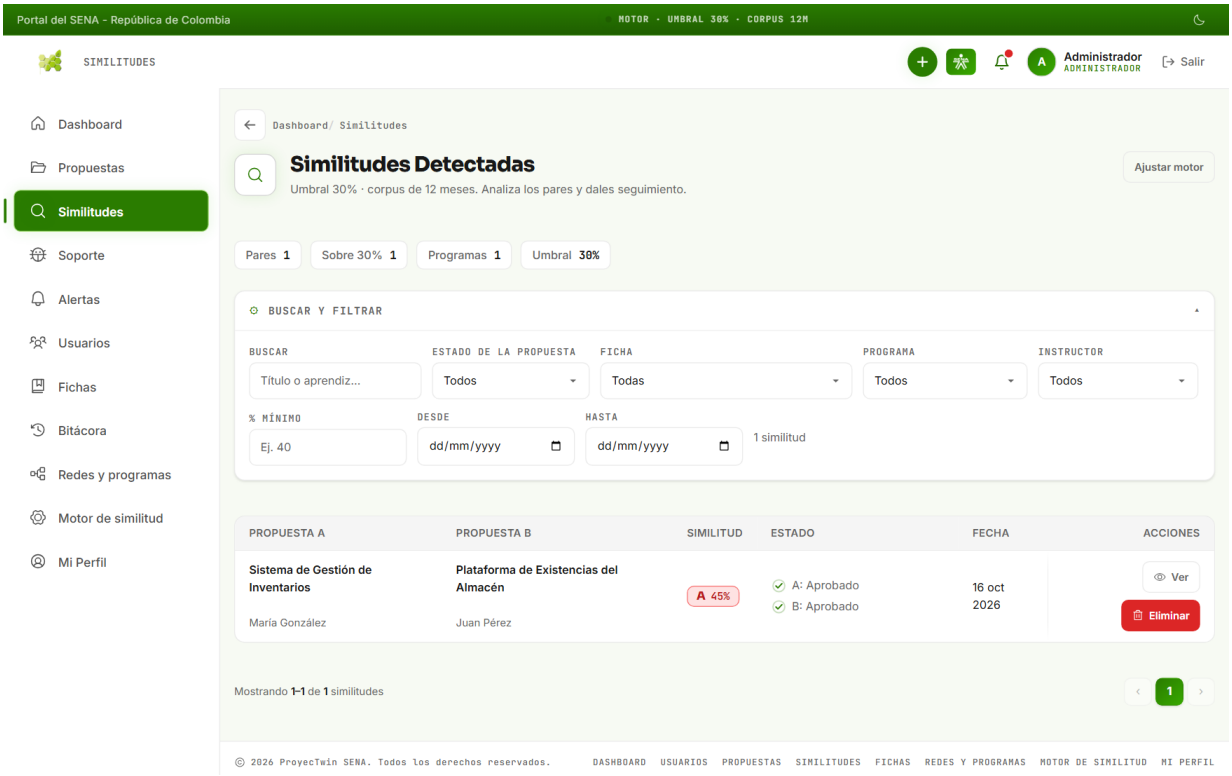
Fuente: Captura real del sistema en ejecución.

Figura 27. Configuración del motor de similitud (umbral y ventana del corpus)



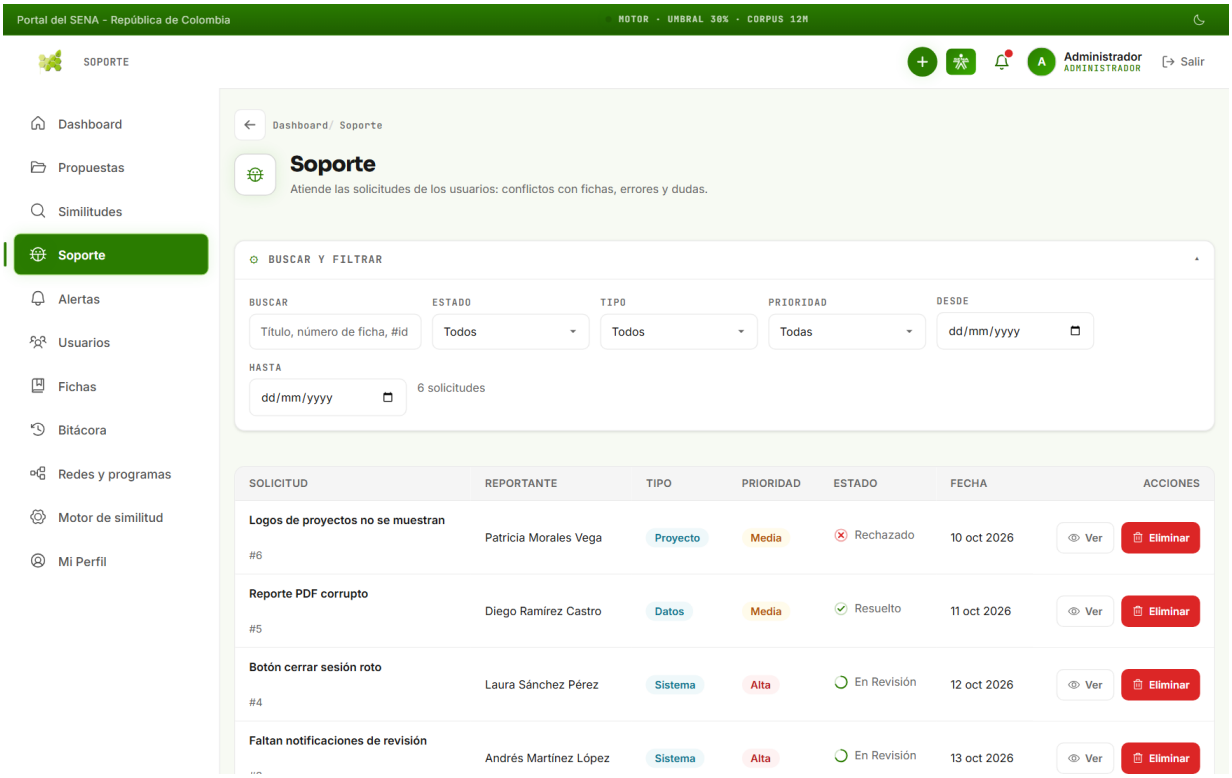
Fuente: Captura real del sistema en ejecución.

Figura 28. Supervisión global de similitudes con recálculo del corpus



Fuente: Captura real del sistema en ejecución.

Figura 29. Bandeja de soporte: reportes de falla y su ciclo de atención

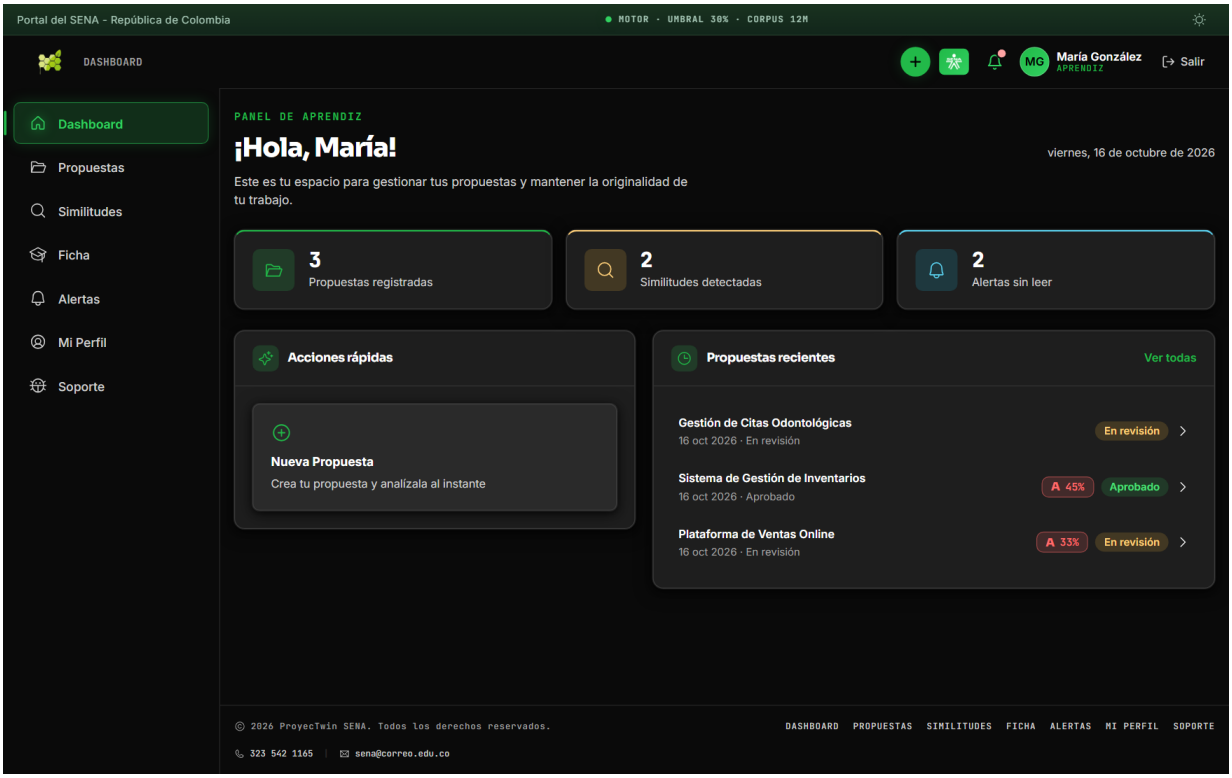


Fuente: Captura real del sistema en ejecución.

8.6. Modo oscuro y experiencia móvil

La interfaz incluye un modo oscuro persistente (cookie `theme`) y diseño adaptable. Las capturas siguientes evidencian ambos comportamientos; las especificaciones `accesibilidad-oscuro.spec.js` y `responsive.spec.js` cubren estas variantes en la suite E2E (no re-ejecutadas completas en esta auditoría).

Figura 30. Panel del aprendiz en modo oscuro (persistencia por cookie y contraste verificado por axe en la suite)



Fuente: Captura real del sistema en ejecución.

Figura 31. Portal público en vista móvil (390×844)



Fuente: Captura real del sistema en ejecución.

Figura 32. Listado de propuestas del aprendiz en vista móvil (tarjetas en lugar de tabla)



Fuente: Captura real del sistema en ejecución.

8.7. Aplicación Android (análisis estático)

La app `Movil/` replica los flujos de los tres roles con arquitectura MVVM (22 ViewModels, 12 repositorios, 14 interfaces de API) y navegación por grafos de rol. Usa los mismos endpoints `/v1` con token Bearer cifrado, tema Material con la identidad institucional (`ui/theme`) y pantallas compartidas (alertas, soporte, perfil, fichas). **No se capturaron pantallas ni se compiló el APK en esta auditoría** (requiere Android Studio/emulador); su estado se clasifica como *no verificado visualmente*, con análisis de código fuente y configuración. Detalle funcional: Anexo A y capítulo 6.

8.8. Resumen de estado por módulo

Tabla 61. Estado de implementación y verificación por módulo

Módulo	Estado	Evidencia principal
Portal público y demo del motor	Verificado	Capturas 01–02; ejecución del portal; endpoints públicos
Autenticación, recuperación y cambio obligatorio	Verificado	Login por tres roles ejecutado; CredencialesTest, PasswordRecoveryTest
Aprendiz: propuestas, equipo y ficha	Verificado	Capturas 03–06; FlujoPropuestaTest, EquipoTest, ApprenticeFichaTest
Aprendiz: similitudes y alertas	Verificado	Capturas 07–09; SimilarityScopeTest, SimilarityPerspectiveTest
Instructor: revisión, fichas y similitudes	Verificado	Capturas 11–15; PermisosTest, FichaAprendizTest
Administrador: usuarios, motor, bitácora y soporte	Verificado	Capturas 16–22; DeleteGuardsTest, AuditLogTest, SoporteTest
Modo oscuro y experiencia móvil web	Verificado (capturas) / suite E2E pendiente de re-ejecución	Capturas 10, 23–24; especificaciones existentes
Motor de similitud (backend)	Verificado	10 pares generados por el seed; SimilitudServiceTest; comando similitud:calcular
Credenciales PDF y correo	Verificado (generación) / parcial (correo real no probado sin SMTP)	CredencialesCorreoTest (con correo simulado)
App Android	No verificado visualmente en esta auditoría	Análisis de código y configuración
Despliegue Docker/Railway	No ejecutado en esta auditoría	Configuración verificada por lectura (Dockerfile, railway.json)

Fuente: Auditoría (16/10/2026).

REFERENCIAS BIBLIOGRÁFICAS

Fuentes consultadas durante la auditoría y la elaboración del manual (normas APA, séptima edición). Solo se listan fuentes efectivamente consultadas entre el 5 y el 16 de octubre de 2026; las normas se verificaron directamente en el catálogo oficial de ISO el 16/10/2026.

Normas técnicas (ISO)

- International Organization for Standardization. (2022). *ISO/IEC 27001:2022. Information security, cybersecurity and privacy protection — Information security management systems — Requirements* (Ed. 3; Amd 1:2024). https://www.iso.org/standard/27001
- International Organization for Standardization. (2026). *ISO 14001:2026. Environmental management systems — Requirements with guidance for use* (Ed. 4). https://www.iso.org/standard/14001
- International Organization for Standardization. (2023). *ISO/IEC 25010:2023. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (Ed. 2). https://www.iso.org/standard/78176.html
- International Organization for Standardization. (2024). *ISO/IEC 25010:2011 (retirada el 04/03/2024; reemplazada por ISO/IEC 25002:2024, ISO/IEC 25010:2023 e ISO/IEC 25019:2023)*. https://www.iso.org/standard/35733.html
- International Organization for Standardization. (2012). *ISO/IEC 9126-1:2001. Software engineering — Product quality — Part 1: Quality model (retirada el 13/09/2012; revisada por ISO/IEC 25010)*. https://www.iso.org/standard/22749.html
- International Organization for Standardization. (2025). *ISO 37001:2025. Anti-bribery management systems — Requirements with guidance for use* (Ed. 2). https://www.iso.org/standard/37001
- International Organization for Standardization. (2026). *ISO 9001:2026. Quality management systems — Requirements*. https://www.iso.org/standard/9001
- International Organization for Standardization. (2026). *ISO 9001:2015 (retirada el 16/09/2026; reemplazada por ISO 9001:2026)*. https://www.iso.org/standard/62085.html
- International Organization for Standardization. (2015). *ISO/IEC 33002:2015. Information technology — Process assessment — Requirements for performing process assessment*. https://www.iso.org/standard/54176.html
- International Organization for Standardization. (2026). *ISO/IEC 27000:2026. Information security, cybersecurity and privacy protection — Information security management systems — Overview* (Ed. 6). https://www.iso.org/standard/27000

- International Organization for Standardization. (2026). *ISO/TC 176/SC 2 — Quality systems: catálogo de normas publicadas y retiradas* (consulta del estado de ISO 10005). https://www.iso.org/committee/53896.html

Documentación oficial de las tecnologías

- Laravel. (2026). *Laravel 10.x Documentation*. https://laravel.com/docs/10.x
- Meta Open Source. (2026). *React — Quick Start (documentación oficial)*. https://react.dev/learn
- VoidZero Inc. y colaboradores de Vite. (2026). *Vite — Getting Started (documentación de la versión 8)*. https://vite.dev/guide/
- The PHP Group. (2026). *Supported Versions (ciclo de vida de las ramas de PHP)*. https://www.php.net/supported-versions.php
- Microsoft. (2026). *Playwright — Installation (documentación oficial)*. https://playwright.dev/docs/intro

Documentos internos del proyecto (fuentes primarias)

- ProyecTwin. (2026). *README del backend* [backend/README.md]. https://github.com/YDanB-312/ProyecTwin-MVP
- ProyecTwin. (2026). *README del frontend* [frontend/README.md]. https://github.com/YDanB-312/ProyecTwin-MVP
- ProyecTwin. (2026). *Arquitectura y recorrido de una petición* [docs/arquitectura.md]. https://github.com/YDanB-312/ProyecTwin-MVP
- ProyecTwin. (2026). *Guía de despliegue en Railway* [docs/deploy-railway.md]. https://github.com/YDanB-312/ProyecTwin-MVP
- ProyecTwin. (2026). *Normativa visual Web + Android* [docs/normativa-visual.md]. https://github.com/YDanB-312/ProyecTwin-MVP
- ProyecTwin. (2026). *Informe de auditoría integral del MVP* [AUDITORIA-INTEGRAL.md, 05/10/2026]. https://github.com/YDanB-312/ProyecTwin-MVP
- ProyecTwin. (2026). *Historial de correcciones* [HISTORIAL-FIXES.md]. https://github.com/YDanB-312/ProyecTwin-MVP
- ProyecTwin. (2026). *Guion de presentación* [docs/guion-presentacion.md]. https://github.com/YDanB-312/ProyecTwin-MVP

Nota: las fechas entre corchetes corresponden a la fecha del documento interno cuando consta; el contenido se verificó contra el árbol de trabajo del 16/10/2026 y las divergencias se señalan en el Anexo F.

ANEXO A. ÁRBOL DE ARCHIVOS RELEVANTES

Árbol resumido del repositorio (se excluyen dependencias instaladas, artefactos de build y archivos temporales). El estado corresponde al árbol de trabajo del 16/10/2026.

```
ProyecTwin-MVP/
├── backend/
│   ├── app/
│   │   ├── Console/Commands/  (BackfillInstructorProfiles, SimilitudCalibrar,
SimilitudRecalcular)
│   │   ├── Http/
│   │   │   ├── Controllers/Controller.php
│   │   │   ├── Controllers/Api/  (15 controladores: Auth, GeneralUser, Instructor,
Apprentice,
│   │   │   │                      ClassGroup, Project, Similarity, MotorConfig,
KnowledgeNetwork,
│   │   │   │                      TrainingProgram, Notification, BugReport,
ApprenticeProject,
│   │   │   │                      Comment, AuditLog)
│   │   │   ├── Middleware/  (Authenticate, CambioContrasenaObligatorio, CuentaActiva,
│   │   │   │                 RolMiddleware, EncryptCookies, RedirectIfAuthenticated,
TrimStrings,
│   │   │   │                 TrustHosts, TrustProxies, ValidateSignature, VerifyCsrfToken)
│   │   │   └── Kernel.php
│   │   ├── Models/  (Admin, Apprentice, ApprenticeProject, AuditLog, BugReport,
ClassGroup,
│   │   │             Comment, GeneralUser, Instructor, KnowledgeNetwork, MotorConfig,
│   │   │             Notification, Project, ProjectHistory, Similarity, TrainingProgram)
│   │   ├── Notifications/  (CredencialesUsuario, ResetPasswordNotification)
│   │   ├── Providers/  (AppServiceProvider, AuthServiceProvider, EventServiceProvider,
│   │   │                RouteServiceProvider)
│   │   ├── Services/  (AltaUsuario, CredencialesCorreo, NotificacionesService,
SimilitudService)
│   │   ├── Similarity/  (Lsa, Pasajes, Recomputador)
│   │   └── Support/  (Auditoria, BorradoCascada, Credenciales, CredencialesPdf,
│   │                  HistorialProyecto, Included)
│   ├── config/  (app, auth, cache, cors, database, filesystems, hashing, logging, mail,
queue,
│   │             sanctum, services, session, view)
│   ├── database/
│   │   ├── migrations/  (18 archivos, 2026_04_09 … 2026_10_16)
│   │   └── seeders/  (DatabaseSeeder, VolumenSeeder)
│   ├── public/index.php
│   ├── resources/views/pdf/credenciales.blade.php
│   ├── routes/  (api-v1.php, console.php, web.php)
│   ├── tests/  (Feature/ 31 archivos, Unit/ 2 archivos)
│   ├── composer.json / composer.lock / .env.example / phpunit.xml
│   └── README.md / README-ALCANCE.md
├── frontend/
│   ├── e2e/  (54 especificaciones *.spec.js, runners y agentes)
│   ├── public/  (images/Logo-ProyecTwin.png, images/logo-sena-blanco.png,
manifest.webmanifest)
│   ├── src/
│   │   ├── App.jsx / main.jsx / index.css
│   │   ├── components/  (54 componentes + guards, con *Base compartidos)
│   │   ├── constants/ contexts/ hooks/ layouts/ lib/ pages/ tokens/ utils/
│   │   └── test/setup.js
│   ├── docs/ALINEACION-FRONTEND-BACKEND.md / CHANGELOG.md
│   ├── index.html / package.json / package-lock.json
│   ├── vite.config.js / vitest.config.js / playwright.config.js /
```

```
playwright.volumen.config.js
│   └── README.md / .env.example
├── Movil/
│   ├── app/src/main/java/com/example/proyectwin/
│   │   ├── MainActivity.kt / ProyecTwinApp.kt
│   │   ├── data/api/  (interceptores, 14 interfaces API, DTOs)  data/model/  data/mapper/
│   │   ├── data/local/  (SessionManager, TokenCipher, ImagenUtils, DescargasArchivos)
│   │   ├── data/repository/ (12)  domain/repository/ (12)  domain/usecase/ (2)
│   │   ├── di/  (NetworkModule, RepositoryModule)
│   │   ├── navigation/  (AppNavigation, NavIntents, NotificationRoutes)
│   │   └── ui/  (screens ~40, viewmodel 22, components, theme, util)
│   ├── app/src/main/res/  (drawable logos, themes, xml de red y backup)
│   ├── app/build.gradle.kts / build.gradle.kts / settings.gradle.kts
│   ├── gradle/libs.versions.toml + wrapper
│   └── app/src/main/AndroidManifest.xml
├── docker/  (nginx.conf, entrypoint.sh)
├── docs/  (arquitectura.md, arquitectura.html, deploy-railway.md, guion-presentacion.md,
│          normativa-visual.md, diagrama*.drawio/excalidraw, arquitectura-figma.svg)
├── docs/manual-tecnico/  (este manual, sus figuras, capturas y scripts de generación)
├── Dockerfile / railway.json / .dockerignore / .railwayignore
├── AUDITORIA-INTEGRAL.md / HISTORIAL-FIXES.md
└── .git/
```

Anexo A — Archivo: Árbol del repositorio (16/10/2026).

ANEXO B. CATÁLOGO COMPLETO DE ENDPOINTS

Tabla 62. Catálogo completo de endpoints de la API (84 rutas)

Método	Ruta	Acción	Middleware (además del grupo api)	Ámbito
POST	/v1/auth/login	AuthController@login	—	Pública
POST	/v1/auth/forgot-password	AuthController@forgotPassword	—	Pública
POST	/v1/auth/reset-password	AuthController@resetPassword	—	Pública
GET	/v1/knowledge-networks	KnowledgeNetworkController@index	—	Pública
GET	/v1/knowledge-networks/{id}	KnowledgeNetworkController@show	—	Pública
GET	/v1/training-programs	TrainingProgramController@index	—	Pública
GET	/v1/training-programs/{id}	TrainingProgramController@show	—	Pública
GET	/v1/config-similitud	MotorConfigController@show	—	Pública
GET	/v1/public/resumen	MotorConfigController@resumen	—	Pública
POST	/v1/public/demo-similitud	SimilarityController@demo	CSRF exento	Pública
POST	/v1/auth/logout	AuthController@logout	auth:sanctum + cuenta.activa	Sesión
GET	/v1/auth/me	AuthController@me	auth:sanctum + cuenta.activa	Sesión
PUT	/v1/auth/email	AuthController@changeEmail	auth:sanctum + cuenta.activa	Sesión
PUT	/v1/auth/password	AuthController@changePassword	auth:sanctum + cuenta.activa	Sesión
PUT	/v1/config-similitud	MotorConfigController@update	AUTH + rol:admin	Autenticada
GET	/v1/audit-logs	AuditLogController@index	AUTH + rol:admin	Autenticada
GET	/v1/general-users	GeneralUserController@index	AUTH + rol:admin	Autenticada
POST	/v1/general-users	GeneralUserController@store	AUTH + rol:admin	Autenticada
POST	/v1/general-users/credenciales	GeneralUserController@credenciales	AUTH + rol:admin	Autenticada
POST	/v1/general-users/{id}/credenciales/reenviar	GeneralUserController@reenviarCredenciales	AUTH + rol:admin	Autenticada
POST	/v1/general-users/{id}/credenciales/restablecer	GeneralUserController@restablecerCredenciales	AUTH + rol:admin	Autenticada
GET	/v1/general-users/{id}/perfil	GeneralUserController@perfil	AUTH	Autenticada
GET	/v1/general-users/{id}	GeneralUserController@show	AUTH	Autenticada
PUT	/v1/general-users/{id}	GeneralUserController@update	AUTH	Autenticada
DELETE	/v1/general-users/{id}	GeneralUserController@destroy	AUTH + rol:admin	Autenticada
POST	/v1/knowledge-networks	KnowledgeNetworkController@store	AUTH + rol:admin	Autenticada
PUT	/v1/knowledge-networks/{id}	KnowledgeNetworkController@update	AUTH + rol:admin	Autenticada
DELETE	/v1/knowledge-networks/{id}	KnowledgeNetworkController@destroy	AUTH + rol:admin	Autenticada
POST	/v1/training-programs	TrainingProgramController@store	AUTH + rol:admin	Autenticada
PUT	/v1/training-programs/{id}	TrainingProgramController@update	AUTH + rol:admin	Autenticada
DELETE	/v1/training-programs/{id}	TrainingProgramController@destroy	AUTH + rol:admin	Autenticada
GET	/v1/instructors	InstructorController@index	AUTH	Autenticada
GET	/v1/instructors/{id}	InstructorController@show	AUTH	Autenticada
PUT	/v1/instructors/{id}	InstructorController@update	AUTH + rol:admin,instructor	Autenticada
POST	/v1/instructors	InstructorController@store	AUTH + rol:admin	Autenticada
DELETE	/v1/instructors/{id}	InstructorController@destroy	AUTH + rol:admin	Autenticada
GET	/v1/apprentices	ApprenticeController@index	AUTH	Autenticada

Método	Ruta	Acción	Middleware (además del grupo api)	Ámbito
GET	/v1/apprentices/me/ficha/codigo/{codigo}	ApprenticeController@fichaPorCodigo	AUTH + rol:aprendiz	Autenticada
POST	/v1/apprentices/me/ficha	ApprenticeController@unirmeAFicha	AUTH + rol:aprendiz	Autenticada
DELETE	/v1/apprentices/me/ficha	ApprenticeController@salirDeFicha	AUTH + rol:aprendiz	Autenticada
GET	/v1/apprentices/{id}	ApprenticeController@show	AUTH	Autenticada
POST	/v1/apprentices	ApprenticeController@store	AUTH + rol:admin,instructor	Autenticada
PUT	/v1/apprentices/{id}	ApprenticeController@update	AUTH + rol:admin,instructor	Autenticada
DELETE	/v1/apprentices/{id}	ApprenticeController@destroy	AUTH + rol:admin	Autenticada
GET	/v1/class-groups	ClassGroupController@index	AUTH	Autenticada
GET	/v1/class-groups/{id}/credenciales	ClassGroupController@credenciales	AUTH + rol:admin	Autenticada
GET	/v1/class-groups/{id}	ClassGroupController@show	AUTH	Autenticada
POST	/v1/class-groups	ClassGroupController@store	AUTH + rol:admin,instructor	Autenticada
POST	/v1/class-groups/{id}/aprendices	ClassGroupController@agregarAprendiz	AUTH + rol:admin,instructor	Autenticada
PUT	/v1/class-groups/{id}	ClassGroupController@update	AUTH + rol:admin,instructor	Autenticada
DELETE	/v1/class-groups/{id}	ClassGroupController@destroy	AUTH + rol:admin,instructor	Autenticada
GET	/v1/projects	ProjectController@index	AUTH	Autenticada
POST	/v1/projects	ProjectController@store	AUTH	Autenticada
POST	/v1/projects/{id}/enviar	ProjectController@enviar	AUTH	Autenticada
GET	/v1/projects/{id}/historial	ProjectController@historial	AUTH	Autenticada
GET	/v1/projects/{id}	ProjectController@show	AUTH	Autenticada
PUT	/v1/projects/{id}	ProjectController@update	AUTH	Autenticada
DELETE	/v1/projects/{id}	ProjectController@destroy	AUTH	Autenticada
POST	/v1/similarities/detect	SimilarityController@detect	AUTH	Autenticada
GET	/v1/similarities	SimilarityController@index	AUTH	Autenticada
GET	/v1/similarities/{id}	SimilarityController@show	AUTH	Autenticada
POST	/v1/similarities/recalculate	SimilarityController@recalculate	AUTH + rol:admin	Autenticada
POST	/v1/similarities	SimilarityController@store	AUTH + rol:admin	Autenticada
PUT	/v1/similarities/{id}	SimilarityController@update	AUTH + rol:admin	Autenticada
DELETE	/v1/similarities/{id}	SimilarityController@destroy	AUTH + rol:admin	Autenticada
GET	/v1/notifications	NotificationController@index	AUTH	Autenticada
POST	/v1/notifications	NotificationController@store	AUTH	Autenticada
GET	/v1/notifications/{id}	NotificationController@show	AUTH	Autenticada
PUT	/v1/notifications/{id}	NotificationController@update	AUTH	Autenticad

Método	Ruta	Acción	Middleware (además del grupo api)	Ámbito
				a
DELETE	/v1/notifications/{id}	NotificationController@destroy	AUTH	Autenticada
GET	/v1/bug-reports	BugReportController@index	AUTH	Autenticada
POST	/v1/bug-reports	BugReportController@store	AUTH	Autenticada
GET	/v1/bug-reports/{id}	BugReportController@show	AUTH	Autenticada
PUT	/v1/bug-reports/{id}	BugReportController@update	AUTH	Autenticada
DELETE	/v1/bug-reports/{id}	BugReportController@destroy	AUTH	Autenticada
GET	/v1/apprentice-projects	ApprenticeProjectController@index	AUTH	Autenticada
POST	/v1/apprentice-projects	ApprenticeProjectController@store	AUTH	Autenticada
GET	/v1/apprentice-projects/{id}	ApprenticeProjectController@show	AUTH	Autenticada
DELETE	/v1/apprentice-projects/{id}	ApprenticeProjectController@destroy	AUTH	Autenticada
GET	/v1/comments	CommentController@index	AUTH	Autenticada
POST	/v1/comments	CommentController@store	AUTH	Autenticada
GET	/v1/comments/{id}	CommentController@show	AUTH	Autenticada
PUT	/v1/comments/{id}	CommentController@update	AUTH	Autenticada
DELETE	/v1/comments/{id}	CommentController@destroy	AUTH	Autenticada

Fuente: `backend/routes/api-v1.php` (auditoría 16/10/2026). AUTH = auth:sanctum + cuenta.activa + CambioContrasenaObligatorio.

ANEXO C. MATRIZ DE TRAZABILIDAD AMPLIADA

Tabla 63. Trazabilidad requisito → evidencia en código → prueba → estado

ID	Evidencia principal (archivo)	Prueba / verificación	Estado
RF-01	backend/app/Http/Controllers/Api/AuthController.php; middleware CambioContrasenaObligatorio	CredencialesTest; login real de tres roles (auditoría)	Verificado
RF-02	AuthController::forgotPassword/resetPassword; config/auth.php (broker)	PasswordRecoveryTest	Verificado
RF-03	GeneralUserController; Support/BorradoCascada.php	DeleteGuardsTest, GeneralUserRolTest	Verificado
RF-04	Services/AltaUsuario.php; Support/CredencialesPdf.php; vista Blade del PDF	CredencialesTest, CredencialesCorreoTest	Verificado
RF-05	ClassGroupController; migración class_groups	FichaNumeroUnicoTest, FichaAprendizTest	Verificado
RF-06	ApprenticeController::unirmeAFicha/salirDeFicha	ApprenticeFichaTest	Verificado
RF-07	ProjectController; Project::puedeEscribir; HistorialProyecto	FlujoPropuestaTest, LogicaNegocioTest, InteraccionPropuestaTest	Verificado
RF-08	ApprenticeProjectController; pivote apprentice_projects	EquipoTest	Verificado
RF-09	SimilarityController@detect; Similarity/Recomputador.php; Services/SimilitudService.php	SimilitudServiceTest; 10 pares generados por el seed	Verificado
RF-10	Recomputador::archivar/recalcular; comando similitud:recalcular	SimilitudesVigentesTest	Verificado
RF-11	NotificationController; Services/NotificacionesService.php	AuthorizationTest (casos de notificación)	Verificado
RF-12	Support/Auditoria.php; AuditLogController; project_histories	AuditLogTest, CierreAuditoriaTest	Verificado
RF-13	BugReportController; notificaciones de soporte	SoporteTest	Verificado
RF-14	KnowledgeNetworkController, TrainingProgramController	ReglasNegocioFase2Test	Verificado
RF-15	MotorConfigController@resumen; SimilarityController@demo	home-demo.spec.js (E2E, existente)	Verificado en ejecución (demo pública navegada)
RF-16	Movil/app/src/main/java/... (22 ViewModels, 12 repositorios)	Sin suite propia; análisis estático	Parcialmente verificado
RF-17	MotorConfigController@update; motor_configs	ValidacionesTest (rangos); captura 20	Verificado
RF-18	GeneralUserController@perfil	VisibilityTest, AccesoContraparteTest	Verificado
RNF-01	Middlewares + scopes; Project::scopeParaUsuario	PermisosTest, AuthorizationTest	Verificado
RNF-02	GeneralUser::$hidden; makeVisible administrativo	CierreAuditoriaTest (BUG-001)	Verificado por código y prueba
RNF-03	audit_logs + project_histories	AuditLogTest	Verificado
RNF-04	DB::transaction + lockForUpdate	AtomicidadTest	Verificado
RNF-05	Tokens de diseño; especificaciones axe	accesibilidad*.spec.js (existentes, no re-ejecutadas)	Parcialmente verificado
RNF-06	DataTable modo tarjeta; layout móvil	responsive.spec.js; capturas móviles reales	Parcialmente verificado
RNF-07	lib/api.js (TTL 15 s, deduplicación); ?paginado=1	Revisión de código	Verificado por código
RNF-08	Dockerfile; docker/entrypoint.sh	Lectura de configuración (build no ejecutado)	Verificado por configuración
RNF-09	Suites de prueba y ESLint	§ 7.10 (244 + 60 + lint 0)	Verificado
RNF-10	Revocación de tokens en cambio de correo; auditorías	ChangeEmailTest	Verificado
RNF-11	ThrottleRequests:api; CuentaActiva	NormalizacionSeguridadTest	Verificado
RNF-12	Scopes de visibilidad y privacidad de pasajes	VisibilityTest, SimilarityScopeTest, privacidad.spec.js	Verificado

Fuente: Auditoría (archivos y pruebas reales).

ANEXO D. COMANDOS DE REFERENCIA

Tabla 64. Comandos verificados o derivados de la configuración del repositorio

Ámbito	Comando	Origen / verificación
Instalación backend	composer install	Verificado (auditoría)
Entorno backend	copy .env.example .env + php artisan key:generate	Verificado
Base de datos	CREATE DATABASE proyectwin_mvp CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;	Verificado
Esquema y datos	php artisan migrate:fresh --seed	Verificado (18 migraciones + seed)
Datos de volumen	php artisan db:seed --class=VolumenSeeder	README del backend
Servidor de desarrollo	php artisan serve --host=127.0.0.1 --port=8000	Verificado
Pruebas backend	php artisan test	Verificado (244/244)
Motor	php artisan similitud:recalcular · similitud:calibrar	Comandos del repositorio
Perfiles	php artisan perfiles:docentes	Comando del repositorio
Dependencias web	npm install	Verificado (227 paquetes)
Desarrollo web	npm run dev	Verificado (5173 + proxy)
Build web	npm run build · npm run preview	Scripts del repositorio
Pruebas unitarias	npm run test:unit	Verificado (60/60)
Lint	npm run lint	Verificado (0 hallazgos)
E2E	npm test (Playwright; reseedea la base)	Configuración del repositorio (no re-ejecutada completa)
E2E completo	npm run test:full	Runner e2e/run-full.mjs
Móvil	gradlew assembleDebug (desde Movil/)	Gradle del repositorio (no ejecutado)
Contenedor	docker build -t proyectwin . / docker run -p 8080:8080 --env-file …	Dockerfile + entrypoint (no ejecutado; Docker ausente)
Healthcheck	GET /up → {"status":"ok"}	Verificado en local y definido en railway.json
Respaldo BD	mysqldump … > respaldo.sql	Procedimiento recomendado (§ 4.11, no implementado en el repo)

Fuente: Verificaciones de la auditoría y archivos del repositorio.

ANEXO E. EVIDENCIAS DE VERIFICACIÓN DE LA AUDITORÍA

Resultados reales obtenidos el 16/10/2026 (detalle en § 7.10). Se listan además las 24 capturas de interfaz generadas, disponibles en la carpeta de documentación del proyecto:

Tabla 65. Resumen de evidencias ejecutadas

Evidencia	Detalle
Suite backend	244 pruebas superadas · 1.156 aserciones · 35,93 s · 0 fallidas
Unitarias frontend	17 archivos · 60 pruebas superadas · 12,85 s
Lint frontend	ESLint sin hallazgos
Arranque completo	API 200 en /up; SPA operativa en localhost:5173; base sembrada (15 usuarios, 16 propuestas, 10 similitudes)
Integridad de datos	10/10 consultas sin incidencias (§ 4.12)
Dependencias	composer audit: 42 avisos / 11 paquetes; npm audit: 3 altas (dev)
Capturas de interfaz	24 PNG (portal, 3 roles, modo oscuro y vista móvil) — nombres 01_…24_ en docs/manual-tecnico/capturas/
Figuras técnicas	8 diagramas propios en docs/manual-tecnico/figuras/ (arquitectura, secuencia, ER, motor, estados, despliegue, autenticación, casos de uso)
Scripts de generación	docs/manual-tecnico/generacion/ (capturas Playwright, figuras matplotlib y generador DOCX)

Fuente: Auditoría (16/10/2026).

ANEXO F. REGISTRO DE LIMITACIONES, DISCREPANCIAS Y DATOS NO CONFIRMADOS

Tabla 66. Registro de limitaciones y discrepancias

ID	Tema	Situación encontrada	Tratamiento en este manual
L-01	Fuente funcional ajena	El PDF adjunto («M.U- PopayanAllTour -Sdel.pdf») documenta el proyecto PopayanAllTour, sin relación con este repositorio; su código fuente no existe en el entorno.	Se excluyó como fuente funcional; la discrepancia queda registrada aquí y en los preliminares.
L-02	Datos institucionales	Ficha de formación, centro, nómina completa del equipo y roles individuales no constan en el repositorio.	La portada y la ficha técnica los declaran como no confirmados; no se inventaron.
L-03	Conteos de los README	El README del backend cita «218 tests» y el del frontend «225 passed / 1 skipped» y «33 tests en 7 archivos»; la verificación real dio 244 pruebas backend, 54 especificaciones E2E y 60 pruebas unitarias en 17 archivos.	Prevalece la verificación del 16/10/2026; los conteos de los README se señalan como desactualizados.
L-04	«0 vulnerabilidades» del README	El README del frontend afirma npm audit → 0 vulnerabilidades; la auditoría detectó 3 altas (dev).	Prevalece la ejecución reciente (§ 7.8); recomendación de actualización.
L-05	Suite E2E no re-ejecutada completa	Existen 54 especificaciones E2E; su ejecución completa (con reseeds) no se realizó en esta auditoría.	Documentada como existente; estado «pendiente de re-ejecución» (Anexo E, § 7.10).
L-06	Docker y despliegue no ejecutados	El equipo de auditoría no tiene Docker instalado; Railway no se desplegó.	Configuración verificada por lectura; comandos derivados de los archivos, señalados como no ejecutados.
L-07	App Android no compilada	No se generó APK ni capturas del dispositivo.	Análisis estático; estado «no verificado visualmente».
L-08	documentación histórica	README-ALCANCE.md, ALINEACION-FRONTEND-BACKEND.md, CHANGELOG.md y HISTORIAL-FIXES.md describen etapas anteriores (era mock) y llevan nota histórica parcial.	Usados solo como contexto; las afirmaciones vigentes se tomaron del código. El CHANGELOG no tiene entradas posteriores a la integración real.
L-09	Trabajo en curso sin confirmar	Artefactos de despliegue, ajustes en TrustProxies/routes/web.php/api.js y la migración sessions no estaban confirmados en Git al cierre.	Documentados como estado del árbol de trabajo; no se alteró ningún archivo del repositorio.
L-10	Estado de la auditoría previa vs. actualidad	AUDITORIA-INTEGRAL.md (05/10/2026) concluyó «NO LISTO» con 2 hallazgos críticos; los commits posteriores y las pruebas de regresión indican corrección.	Se documentó la resolución verificable (pruebas de regresión y código actual: § 7.11); no se re-ejecutó la auditoría previa completa.
L-11	ISO 10005	La edición de 2018 no pudo confirmarse directamente en el catálogo durante la consulta (las ediciones 1995 y 2005 figuran retiradas).	Se declara la incertidumbre y se recomienda verificar la edición vigente antes de citarla contractualmente.
L-12	Integridad de negocio bajo concurrencia	No se ejecutaron pruebas de concurrencia reales (envíos simultáneos).	Documentada como limitación (§ 4.12); las mitigaciones de código (lockForUpdate) se describen como tales, no como probadas.
L-13	Métricas de calidad del motor	No hay resultados versionados de precisión/recall del motor sobre un conjunto etiquetado.	Se describe el mecanismo de calibración existente sin afirmar métricas no medidas.
L-14	Entorno de la base de datos de verificación	Para proteger los datos del equipo se usó una base separada (proyectwin_mvp_manual); el .env de trabajo fue creado por la auditoría (archivo local excluido de Git).	Registrado aquí; la base original no fue modificada y no se comprometió ningún secreto.

Fuente: Auditoría (16/10/2026).

Declaración final de alcance: este manual describe el sistema a partir de su repositorio y de su ejecución local verificada. Las afirmaciones se limitan a la evidencia recogida; toda

afirmación clasificada como «parcialmente verificada», «inferida» o «no verificada» debe tratarse como pendiente hasta que el equipo del proyecto la confirme o complete.